IFM 3.1: Compilerbau (Winter 2026/27)

BC George & Carsten Gips (HSBI)

12. November 2026

Inhaltsverzeichnis

I. Syllabus 1

1. Syllabus 2
- 1.1. Kursbeschreibung . . . 2
- 1.2. Überblick Modulinhalte . . . 2
- 1.3. Team . . . 3
- 1.4. Kursformat . . . 3
- 1.5. Fahrplan . . . 3
- 1.6. Prüfungsform, Note und Credits . . . 4
  - 1.6.1. **Studienleistung**: “Portfolio” . . . 4
  - 1.6.2. **Gesamtnote**: Parcoursprüfung . . . 5
  - 1.6.3. Hinweise . . . 5
- 1.7. Materialien . . . 6
- 1.8. Förderungen und Kooperationen . . . 6
  - 1.8.1. Kooperation mit University of Alberta, Edmonton (Kanada) . . . 6
- 1.9. LICENSE . . . 6

II. Vorlesungsunterlagen 7

2. Überblick 8
- 2.1. Struktur eines Compilers . . . 8
  - 2.1.1. Sprachen verstehen, Texte transformieren . . . 8
  - 2.1.2. Compiler: Big Picture . . . 9
  - 2.1.3. Lexikalische Analyse: Wörter (“*Token*”) erkennen . . . 11
  - 2.1.4. Syntaxanalyse: Sätze erkennen . . . 11
  - 2.1.5. Vorschau: Parser implementieren . . . 12
  - 2.1.6. Semantische Analyse: Bedeutung erkennen . . . 13
  - 2.1.7. Zwischencode generieren . . . 13
  - 2.1.8. Code optimieren . . . 14
  - 2.1.9. Code generieren . . . 14
  - 2.1.10. Probleme . . . 14
  - 2.1.11. Wrap-Up . . . 15
- 2.2. Bandbreite der Programmiersprachen . . . 15
  - 2.2.1. 99 Bottles of Beer . . . 15
  - 2.2.2. Imperativ, Hardwarenah: C . . . 16

2.2.3. Imperativ, Objektorientiert: Java . . . 16
2.2.4. Logisch: Prolog . . . 17
2.2.5. Funktional: Haskell . . . 18
2.2.6. Brainfuck . . . 19
2.2.7. Programmiersprache Lox . . . 19
2.2.8. Wrap-Up . . . 20
2.3. Anwendungen . . . 20
2.3.1. Anwendung: Compiler . . . 21
2.3.2. Anwendung: Interpreter . . . 21
2.3.3. Anwendung: Virtuelle Maschinen . . . 21
2.3.4. Anwendung: C-Toolchain . . . 22
2.3.5. Anwendung: C++-Compiler . . . 22
2.3.6. Anwendung: Bugfinder . . . 22
2.3.7. Anwendung: Pandoc . . . 23
2.3.8. Was bringt mir das? . . . 24
2.3.9. Wrap-Up . . . 26

3. Reguläre Sprachen, kontextfreie Grammatiken und Sprachen, lexikalische und syntaktische Analyse 28
3.1. Reguläre Sprachen, Ausdrucksstärke (Teil 1) . . . 28
3.1.1. Motivation . . . 28
3.1.2. Endliche Automaten . . . 28
3.1.3. Reguläre Sprachen . . . 31
3.1.4. Wrap-Up . . . 33
3.2. Reguläre Sprachen, Ausdrucksstärke (Teil 2) . . . 33
3.2.1. Wiederholung . . . 33
3.2.2. Motivation . . . 34
3.2.3. Lexer . . . 34
3.2.4. Wrap-Up . . . 35
3.3. CFG . . . 35
3.3.1. Wiederholung . . . 36
3.3.2. Motivation . . . 36
3.3.3. Kontextfreie Grammatiken und Sprachen . . . 38
3.3.4. Syntaxanalyse . . . 38
3.3.5. Wrap-Up . . . 39
3.4. LL-Parser . . . 40
3.4.1. Wiederholung . . . 40
3.4.2. Motivation . . . 40
3.4.3. Syntaxanalyse . . . 40
3.4.4. LL(k)-Grammatiken . . . 41
3.4.5. Wrap-Up . . . 44

4. Sprache L-Int: Integer, Addition, Subtraktion 45
4.0.1. Teil 1: . . . 45
4.0.2. Teil 2: . . . 45

4.1. L-Int: Die Sprache Lispy . . . 45
4.1.1. Problem: Wie verstehen wir `1 + 2 * 3`? . . . 45
4.1.2. S-Expressions als syntaktische Grundform . . . 46
4.1.3. Lispy: Erste Sprachstufe L-Int . . . 48
4.1.4. Wrap-up . . . 49
4.2. L-Int: Konkrete und abstrakte Syntax . . . 50
4.2.1. Erinnerung Lispy in der Sprachstufe L-Int . . . 50
4.2.2. Konkrete Grammatik für L-Int . . . 51
4.2.3. Beispiel konkrete Syntax . . . 53
4.2.4. Konkrete Grammatik - Eigenschaften . . . 53
4.2.5. Konkrete vs. abstrakte Grammatik . . . 54
4.2.6. Abstrakte Grammatik für L-Int . . . 54
4.2.7. Beispiel abstrakte Syntax (AST) . . . 55
4.2.8. AST in Java: Definition . . . 56
4.2.9. AST in Java: Beispielkonstruktion . . . 57
4.2.10. Wrap-up . . . 57
4.3. L-Int: Von Ausdrücken zum Interpreter . . . 59
4.3.1. Von AST zu Werten: Idee . . . 59
4.3.2. Value-Hierarchie: Motivation . . . 59
4.3.3. Value-Hierarchie in Java . . . 60
4.3.4. Tree-Walking-Interpreter: Grundidee . . . 60
4.3.5. Interpreter für L-Int-Expressions in Java (Pattern Matching) . . . 61
4.3.6. Interpreter in Aktion . . . 63
4.3.7. Wrap-up . . . 64
4.4. L-Int: Lexer - Handcodierte Implementierung . . . 65
4.4.1. Lexer: Erzeugen eines Token-Stroms aus einem Zeichenstrom . . . 65
4.4.2. Modellierung der Token . . . 66
4.4.3. Grundstruktur eines Lexers . . . 67
4.4.4. Partielle Beispielimplementierung Lexer für L-Int . . . 68
4.4.5. Hilfsfunktionen im Lexer: Zeichenhandling . . . 69
4.4.6. Typische Muster für die Definition von Token . . . 70
4.4.7. Fehler bei der lexikalischen Analyse . . . 71
4.4.8. Wrap-up . . . 72
4.5. L-Int: LL-Parser mit Recursive Descent . . . 75
4.5.1. LL-Parser mit Recursive Descent: Ziel . . . 75
4.5.2. Grundidee LL-Parser mit Recursive Descent . . . 76
4.5.3. Alternative Subregeln . . . 77
4.5.4. Grundstruktur eines RD-Parsers . . . 78
4.5.5. Partielle Beispielimplementierung RD-Parser für L-Int . . . 78
4.5.6. Details Parser für L-Int: `parseAddExpr()` . . . 80
4.5.7. Hilfsfunktionen im Parser: Tokenhandling . . . 81
4.5.8. Auflösen von (links-) rekursiven Grammatiken . . . 82
4.5.9. Lispy-Pipeline komplett . . . 84

4.5.10. Wrap-up . . . 84

5. Sprache L-Expr: erweiterte Ausdrücke, Vorrangregeln 87
5.0.1. Inhalte . . . 87
5.1. L-Expr: Multiplikation/Division und unäres Minus . . . 87
5.1.1. Rückblick: L-Int . . . 87
5.1.2. Erweiterung zu L-Expr: Multiplikation und Division . . . 88
5.1.3. Abstrakte Syntax für L-Expr . . . 89
5.1.4. Interpreter: Erinnerung an L-Int . . . 89
5.1.5. Interpreter L-Expr: Erben von L-Int und Ergänzen der neuen Funktionalität . . . 90
5.1.6. Wrap-up . . . 92
5.2. L-Expr: Vorrang und Assoziativität . . . 93
5.2.1. Java-artige Infix-Syntax: Vorrangregeln . . . 94
5.2.2. Assoziativität (Klammerung) . . . 94
5.2.3. Vorrang und Assoziativität: Gestufte Grammatik-Regeln . . . 94
5.2.4. Wrap-up . . . 95
5.3. L-Expr: ANTLR-Grammatik und Vorrangregeln . . . 97
5.3.1. Von klassischer CFG zur ANTLR-Grammatik . . . 98
5.3.2. Erinnerung: Konkrete Syntax für L-Expr . . . 98
5.3.3. ANTLR-Grammatik für L-Expr (Parser-Teil) . . . 98
5.3.4. ANTLR-Grammatik für L-Expr (Lexer-Teil) . . . 101
5.3.5. Arbeitsweise von ANTLR . . . 102
5.3.6. Verhalten des Lexers . . . 103
5.3.7. Berücksichtigung von Präzedenz im Parser . . . 103
5.3.8. Direkte Linksrekursion und ANTLR4 - automatische Präzedenz . . . 104
5.3.9. Ausblick . . . 105
5.3.10. Wrap-up . . . 105
5.4. L-Expr: ANTLR-Visitoren und Rückgabewerte . . . 108
5.4.1. Erinnerung: ANTLR-Toolchain: Von String zum Parse-Tree . . . 109
5.4.2. ANTLR: Generierte Klassen . . . 109
5.4.3. AST-Builder: Beispiel Int-Literal und Addition . . . 110
5.4.4. ANTLR-Pipeline komplett . . . 112
5.4.5. Default-Implementierung: Wann reichen die Default-`visitXYZ()`-Methoden? . . . 113
5.4.6. Wrap-up . . . 114

6. Sprache L-Var: Variablen, Statments, nested Scopes 117
6.0.1. Inhalt . . . 117

7. Sprache L-If: Datentyp Boolean, Vergleiche, Kontrollstrukturen (if/else, while) 118
7.0.1. Inhalte . . . 118

8. Sprache L-Fun: Funktionen (Definition, Aufruf) 119
8.0.1. Inhalte . . . 119

9. Sprache L-Class: Klassen (Felder, Methoden, Objekte) 120
9.0.1. Inhalte . . . 120

10. Sprache L-Inherit: Einfachvererbung und dynamischer Dispatch 121
10.0.1. Inhalte . . . 121

11. Sprache L-Self: native Klassen (inkl. Vererbung) 122
11.0.1. Inhalte . . . 122

III. Praktikum 123

12. Blatt 01: Reguläre Sprachen 125
12.1. A1.1: Sprachen von regulären Ausdrücken . . . 125
12.2. A1.2: Bezeichner in Programmiersprachen . . . 125
12.3. A1.3: Gleitkommazahlen in Programmiersprachen . . . 126
12.4. A1.4: Mailadressen? . . . 126
12.5. A1.5: Der zweitletzte Buchstabe . . . 126
12.6. A1.6: Sprache einer regulären Grammatik . . . 126

13. Blatt 02: CFG 127
13.1. A2.1: Akzeptierte Sprache . . . 127
13.2. A2.2: Kontextfreie Sprache . . . 127
13.3. A2.3: Kontextfreie Grammatik . . . 128
13.4. A2.4: PDA . . . 128

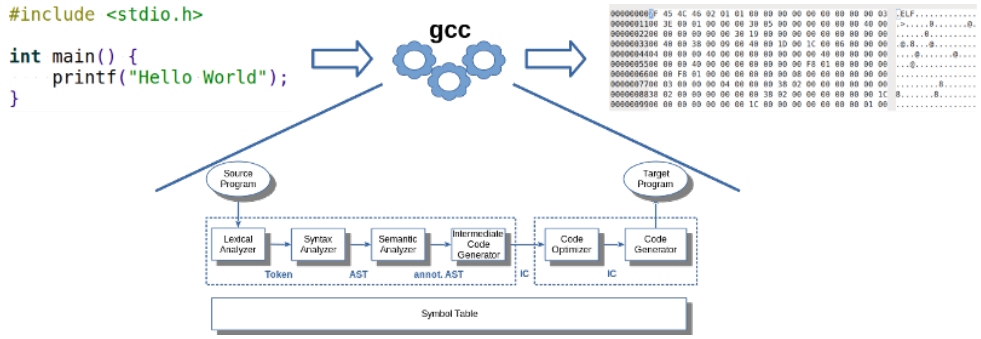
14. Blatt 03: L-Int und C-Int (Integer, Addition, Subtraktion) 129
14.1. A3.1: L-Int: Subtraktion verstehen und ergänzen (*) . . . 129
14.1.1. 1. Sprache und Semantik nachvollziehen . . . 129
14.1.2. 2. Auswertung der Subtraktion skizzieren . . . 130
14.2. A3.2: L-Int: Funktion `skipIrrelevant()` im Lexer ergänzen (**) . . . 130
14.3. A3.3: C++ ausprobieren: Compiler und ein erstes Programm (*) . . . 131
14.3.1. Schritt 1: Compiler überprüfen und installieren . . . 131
14.3.2. Schritt 2: Erstes Programm übersetzen und ausführen . . . 132
14.3.3. Mini-Waschzettel: `hello.cpp` im Vergleich zu L-Int / C-Int / Java . . . 132
14.4. A3.4: C-Int: Addition und Subtraktion für Integer in C++ (**) . . . 133
14.4.1. Waschzettel: In C++ sieht es so aus … . . . 133
14.4.2. Schritt 1: Java-Projekt für C-Int einrichten . . . 134
14.4.3. Schritt 2: Konkrete Syntax für C-Int festlegen . . . 134
14.4.4. Schritt 3: Abstrakte Syntax für C-Int definieren . . . 135
14.4.5. Schritt 4: Interpreter-Pipeline für C-Int implementieren . . . 135
14.4.6. Schritt 5: Tests für C-Int schreiben . . . 135
14.4.7. Schritt 6: C-Int-Ergebnisse mit nativem C++ vergleichen . . . 136
14.5. A3.5: Vertiefung: Interpreter für L-Int mit Visitor-Pattern (***) . . . 136

15. Blatt 04: L-Expr und C-Expr (Multiplikation/Division, Vorrang und ANTLR) 138
15.1. A4.1: L-Expr: Division ergänzen (*) . . . 138
15.2. A4.2: C-Expr: Vorrangregeln in einer gestuften Grammatik (**) . . . 140
15.2.1. Waschzettel: In C++ sieht es so aus … . . . 140
15.2.2. Schritt 1: C-Expr ausprobieren: Vorrang und unäres Minus live testen . . . 140
15.2.3. Schritt 2: Konkrete Syntax für C-Expr (gestufte Variante) . . . 140
15.2.4. Schritt 3: ANTLR-Grammatiken für C-Expr . . . 141
15.2.5. Schritt 4: Abstrakte Syntax für C-Expr . . . 141
15.2.6. Schritt 5: Parsen von C-Expr und bauen des AST . . . 141
15.2.7. Schritt 6: Interpreter-Pipeline für C-Expr . . . 142
15.3. A4.3: Vertiefung: ANTLR und Stolperfallen (***) . . . 142

Teil I.

Syllabus

1. Syllabus



1.1. Kursbeschreibung

Der Compiler ist das wichtigste Werkzeug in der Informatik. In der Königsdisziplin der Informatik schließt sich der Kreis, hier kommen die unterschiedlichen Algorithmen und Datenstrukturen und Programmiersprachenkonzepte zur Anwendung.

In diesem Modul geht es um ein grundlegendes Verständnis für die wichtigsten Konzepte im Compilerbau. Wir schauen uns dazu relevante aktuelle Tools und Frameworks an und setzen diese bei der Erstellung eines kleinen Compiler-Frontends für C++ ein.

1.2. Überblick Modulinhalte

1. Lexikalische Analyse: Scanner/Lexer
    - Reguläre Sprachen
    - Generierung mit ANTLR
2. Syntaxanalyse: Parser
    - Kontextfreie Grammatiken (CFG)
    - LL-Parser (Recursive-Descent-Parser)
    - Generierung mit ANTLR
3. Semantische Analyse: Symboltabellen, Name-Resolving, Type-Checking
    - Namen und Scopes
    - Typen, Klassen, Polymorphie

4. Interpreter: AST-Traversierung
5. C++ als zu verarbeitende Programmiersprache

1.3. Team

- BC George
- Carsten Gips (Sprechstunde nach Vereinbarung)
- Alesia Herbertz, Vivien Traue, Jonathan Hauer (Tutor:innen)

1.4. Kursformat

Vorlesung (2 SWS)	Praktikum (2 SWS)
Mo, 15:45 - 17:15 Uhr (Zoom)	G1: Mi, 11:30 - 13:00 Uhr (Zoom)
	G2: Mi, 14:00 - 15:30 Uhr (Zoom)
	G3: Mi, 11:30 - 13:00 Uhr (Präsenz D327)
	G4: Mi, 14:00 - 15:30 Uhr (Präsenz D327)

Durchführung der Vorlesung als *Flipped Classroom* (Carsten) bzw. als *reguläre Vorlesung* (BC). **Zugangsdaten Zoom siehe ILIAS**.

1.5. Fahrplan

Monat	Woche vom	Vorlesung (Mo)	Praktikum (Mi)	Edmonton/Minden-Meetings
Oktober	12.10	Orga || Überblick | Sprachen | Anwendungen	-	
	19.10.	Reguläre Sprachen 1	B01	
	26.10.	Reguläre Sprachen 2 || CFG	B02	
November	02.11.	LL-Parser (Theorie)	- (**Extra Sprechstunde 04.11., 13-14 Uhr, online**)	**Di, 03.11., 17:00 - 18:00 Uhr (online): ANTLR + Live-Coding**
	09.11.	L-Int (Teil 1): Intro L-Int | Grammatik | Interpreter	**Station 1**	
	16.11.	L-Int (Teil 2): Lexer | RD-Parser	B03: C-Int	

Monat	Woche vom	Vorlesung (Mo)	Praktikum (Mi)	Edmonton/Minden-Meetings
	23.11.	L-Expr: Intro L-Expr | Vorrang | ANTLR Grammatik | ANTLR Visitor	B04: C-Expr	
Dezember	30.11.	L-Var	**Station 2**	**Mo, 30.11., 17:00 - 18:00 Uhr (online): Minden Presentations**
	07.12.	L-If	C-Var, C-If	**Mo, 07.12., 17:00 - 18:00 Uhr (online): Edmonton Presentations**
	14.12.	L-Fun	C-If, C-Fun	
	21.12.	**Weihnachtspause**	-	
	28.12.	**Weihnachtspause**	-	
Januar	04.01.	L-Class	**Station 3**	
	11.01.	L-Inherit	C-Class, C-Inherit	
	18.01.	L-Self	C-Self, Snake	
	25.01.	Rückblick	**Projektvorstellung** (Video)	

1.6. Prüfungsform, Note und Credits

Parcoursprüfung plus Studienleistung (Portfolio), 5 ECTS

1.6.1. Studienleistung: “Portfolio”

Die Studienleistung ist eine unbenotete Leistung und setzt sich aus mehreren Komponenten zusammen:

1. Pro Person: Teilnahme an **mind. zwei Edmonton/Minden-Terminen** mit aktiver Beteiligung und Abgabe eines ausreichenden **Post Mortems** (pro Meeting, je Person)
    - Termin 1: Dienstag, 03.11., 17:00 - 18:00 Uhr (online)
    - Termin 2: Montag, 30.11., 17:00 - 18:00 Uhr (online)
    - Termin 3: Montag, 07.12., 17:00 - 18:00 Uhr (online)
    - Abgabe der Post Mortems (s.u.) zu den Edmonton-Meetings jeweils bis Montag 09:00 Uhr in der Folgewoche im ILIAS

2. Pro Team: **Abschluss-Video-Vortrag** zum erfolgreich bearbeiteten **Snake-Mini-Projekt** (letztes Blatt) am Semesterende
    - Termin: Mittwoch, 27.01., in Praktikumszeit (*Slots werden noch bekannt gegeben*)
    - Video: 10 Minuten Dauer (pro Team)
    - Anschließend kurzes Q&A (Fragen zum Snake-Mini-Projekt, pro Team)
    - Vorführung des Videos und die Q&A findet pro Team statt, Anwesenheit erforderlich
    - Abgabe des Videos und der Lösung zum Snake-Mini-Projekt bis Montag, 25.01., 09:00 Uhr im ILIAS
    - Teilleistung jeder Person im Team muss erkennbar sein

1.6.2. Gesamtnote: Parcoursprüfung

Die Modul-Note ergibt sich aus der Leistung in der Parcoursprüfung.

Sie können die Prüfung in der ersten oder in der zweiten Prüfungsphase ablegen. Die Stationen der Parcoursprüfung sind je nach Prüfungsphase unterschiedlich gestaltet und in sich geschlossen (kein Übertrag):

- **Prüfungsphase I**: **Vier Stationen** (digitale E-Assessments im B40 mit je 30 Minuten Dauer), **beste drei Ergebnisse ergeben die Note**
    - Station 1: Mittwoch, 11.11., in Praktikumszeit (12:30-13:00, 13:15-13:45, 14:00-14:30)
    - Station 2: Mittwoch, 02.12., in Praktikumszeit (*Slots werden noch bekannt gegeben*)
    - Station 3: Mittwoch, 06.01., in Praktikumszeit (*Slots werden noch bekannt gegeben*)
    - Station 4: Im ersten Prüfungszeitraum (*Termin wird vom Prüfungsamt bekanntgegeben*)
- **Prüfungsphase II**: **Digitale Klausur** im B40, Dauer 120 Minuten (*Termin wird vom Prüfungsamt bekanntgegeben*), **Klausurergebnis bestimmt die Note**

Fragetypen-Demo im ILIAS

1.6.3. Hinweise

- Die Parcoursprüfung und die Studienleistung sind zwei unabhängige Prüfungsleistungen und werden separat im CAT angemeldet. Die Studienleistung schaltet die Credits für das Modul frei, die Parcoursprüfung ergibt die Modulnote. Die Studienleistung ist **keine** Vorleistung für die Parcoursprüfung.
- Die Bearbeitung der Aufgaben erfolgt im Team.
- Ein Team umfasst 3 Personen.
- Im Praktikum beginnen wir gemeinsam mit der Bearbeitung der Übungsblätter, diskutieren über Lösungsansätze und erarbeiten Abnahmekriterien. Die Lösung soll anschließend teamweise fertiggestellt werden und kann auf Wunsch im nächsten Praktikum von Ihnen vorgestellt werden.
- Wir bauen schrittweise über die Übungsblätter hinweg einen Interpreter für einen Mini-C++-Dialekt auf. Sie benötigen diese schrittweise erarbeiteten Bausteine für das erfolgreiche Bearbeiten des Snake-Mini-Projekts (letztes Blatt) und damit das Bestehen der Studienleistung.
- “Erfolgreiche Bearbeitung” umfasst die Bearbeitung aller Aufgaben im Zusammenhang des Snake-Mini-Projekts. Die intensive Beschäftigung mit den Aufgaben muss erkennbar sein.
- Die Teilnahme am Praktikum ist freiwillig, wird aber deutlich empfohlen.

- Eine Bewertung einzelner Übungsblätter findet nicht statt.
- Die Post Mortems sind pro Edmonton-Meeting und individuell zu erstellen und abzugeben.
- Das Video zum Snake-Mini-Projekt ist pro Team zu erstellen und einmal abzugeben unter Angabe der Teammitglieder.
- "Aktive Beteiligung" umfasst Anwesenheit und sachbezogene Beiträge; Anwesenheit/Beteiligung werden dokumentiert.

- **Post Mortem**: Jede Person beschreibt individuell(!) die Teilnahme an den Edmonton/Minden-Meetings zurückblickend mit mind. 150 bis max. 400 Wörtern (Nutzlast! Überschriften und Links zählen nicht mit). Gehen Sie dabei aussagekräftig und nachvollziehbar auf folgende Punkte ein:

  1. **Zusammenfassung**: Was wurde auf dem Meeting besprochen?
  2. **Details**: Kurze Beschreibung besonders interessanter Aspekte.
  3. **Reflexion**: Was war der schwierigste Teil? Wie haben Sie dieses Problem gelöst?
  4. **Reflexion**: Was haben Sie gelernt oder (besser) verstanden?

  Die Post Mortems geben Sie bitte pro Person bis spätestens zur jeweiligen Deadline im ILIAS ab.

1.7. Materialien

1. "**Crafting Interpreters**". Nystrom, R., Genever Benning, 2021. ISBN 978-0-9905829-3-9. Online.
2. "**Introduction to Compiler Design**". Mogensen, T.A., Springer International, 2024. ISBN 978-3-031-46460-7.

1.8. Förderungen und Kooperationen

1.8.1. Kooperation mit University of Alberta, Edmonton (Kanada)

Über das Projekt "We CAN virtuOWL" der Fachhochschule Bielefeld ist im Frühjahr 2021 eine Kooperation mit der University of Alberta (Edmonton/Alberta, Kanada) im Modul "Compilerbau" gestartet.

Wir freuen uns, auch in diesem Semester wieder drei gemeinsame Sitzungen für beide Hochschulen anbieten zu können. (Diese Termine werden in englischer Sprache durchgeführt.)

1.9. LICENSE



Unless otherwise noted, this work by BC George, Carsten Gips and contributors is licensed under CC BY-SA 4.0. See the credits for a detailed list of contributing projects.

Teil II.

Vorlesungsunterlagen

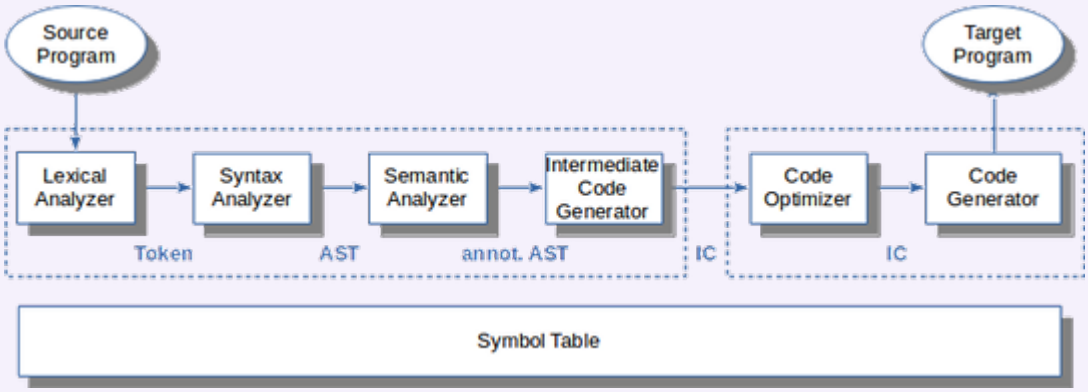
2. Überblick

Was ist ein Compiler? Welche Bausteine lassen sich identifizieren, welche Aufgaben haben diese?

2.1. Struktur eines Compilers

TL;DR

Compiler übersetzen (formalen) Text in ein anderes Format.
Typischerweise kann man diesen Prozess in verschiedene Stufen/Phasen einteilen. Dabei verarbeitet jede Phase den Output der vorangegangenen Phase und erzeugt ein (kompakteres) Ergebnis, welches an die nächste Phase weitergereicht wird. Dabei nimmt die Abstraktion von Stufe zu Stufe zu: Der ursprüngliche Input ist ein Strom von Zeichen, daraus wird ein Strom von Wörtern (Token), daraus ein Baum (Parse Tree), Zwischencode (IC), …



Die gezeigten Phasen werden traditionell unterschieden. Je nach Aufgabe können verschiedene Stufen zusammengefasst werden oder sogar gar nicht auftreten.

Videos

Vorlesung [YT], [HSBI]

2.1.1. Sprachen verstehen, Texte transformieren

> The cat runs quickly.

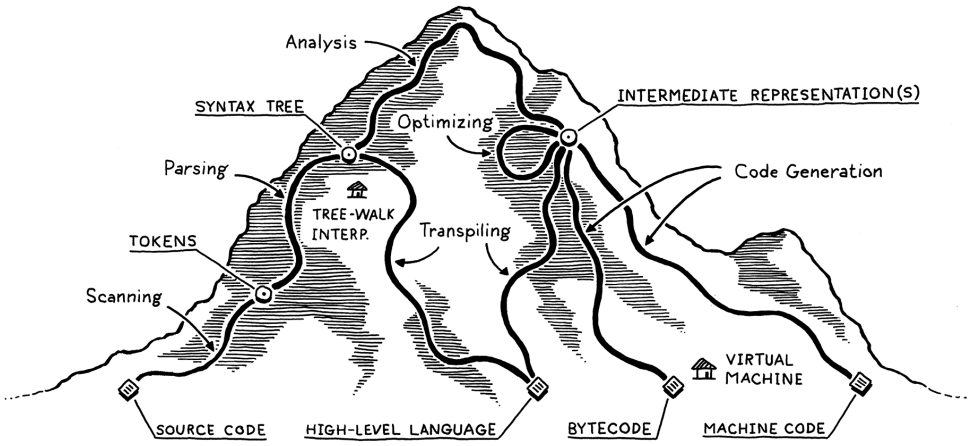
=> Struktur? Bedeutung?

Wir können hier (mit steigender Abstraktionsstufe) unterscheiden:

- Sequenz von Zeichen
- Wörter: Zeichenketten mit bestimmten Buchstaben, getrennt durch bestimmte andere Zeichen; Wörter könnten im Wörterbuch nachgeschlagen werden
- Sätze: Anordnung von Wörtern nach einer bestimmten Grammatik, Grenze: Satzzeichen

  Hier (vereinfacht): Ein Satz besteht aus Subjekt und Prädikat. Das Subjekt besteht aus einem oder keinen Artikel und einem Substantiv. Das Prädikat besteht aus einem Verb und einem oder keinem Adverb.
- Sprache: Die Menge der in einer Grammatik erlaubten Sätze

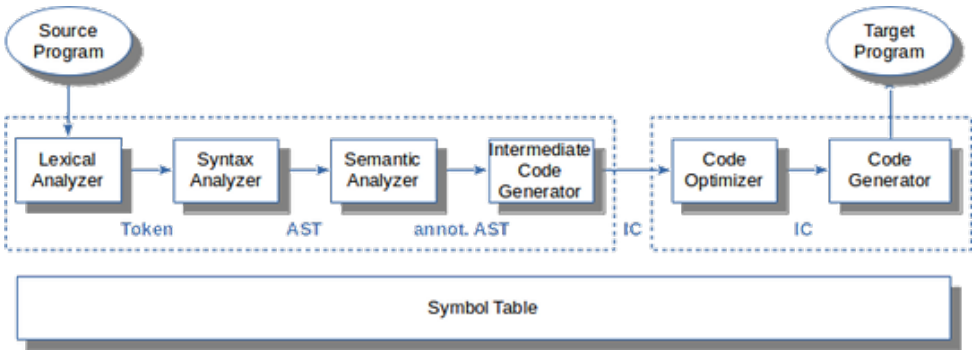
2.1.2. Compiler: Big Picture



Quelle: A Map of the Territory (mountain.png) by Bob Nystrom on Github.com (MIT)

Begriffe und Phasen

Die obige Bergsteige-Metapher kann man in ein nüchternes Ablaufdiagramm mit verschiedenen Stufen und den zwischen den Stufen ausgetauschten Artefakten übersetzen:



2.1.2.1. Frontend, Analyse

Die ersten Stufen eines Compilers, die mit der **Analyse** des Inputs beschäftigt sind. Dies sind in der Regel der Scanner, der Parser und die semantische Analyse.

- Scanner, Lexer, Tokenizer, Lexikalische Analyse

  Zerteilt den Zeichenstrom in eine Folge von Wörtern. Mit regulären Ausdrücken kann definiert werden, was Klassen gültiger Wörter (“Token”) sind. Ein Token hat i.d.R. einen Namen und einen Wert.

- Parser, Syntaxanalyse

  Der Parser erhält als Eingabe die Folge der Token und versucht mit Hilfe einer Grammatik zu bestimmen, ob es sich bei der Tokensequenz um gültige Sätze im Sinne der Grammatik handelt. Hier gibt es viele Algorithmen, die im Wesentlichen in die Klassen “top-down” und “bottom-up” fallen.

- Semantische Analyse, Kontexthandling

  In den vorigen Stufen wurde eher lokal gearbeitet. Hier wird über den gesamten Baum und die Symboltabelle hinweg geprüft, ob beispielsweise Typen korrekt verwendet wurden, in welchen Scope ein Name gehört etc. Mit diesen Informationen wird der AST angereichert.

- Symboltabellen

  Datenstrukturen, um Namen, Werte, Scopes und weitere Informationen zu speichern. Die Symboltabellen werden vor allem beim Parsen befüllt und bei der semantischen Analyse gelesen, aber auch der Lexer benötigt u.U. diese Informationen.

2.1.2.2. Backend, Synthese

Die hinteren Stufen eines Compilers, die mit der **Synthese** der Ausgabe beschäftigt sind. Dies sind in der Regel verschiedene Optimierungen und letztlich die Code-Generierung

- Codegenerierung

  Erzeugung des Zielprogramms aus der (optimierten) Zwischendarstellung. Dies ist oft Maschinencode, kann aber auch C-Code oder eine andere Ziel-Sprache sein.

- Optimierung

  Diverse Maßnahmen, um den resultierenden Code kleiner und/oder schneller zu gestalten.

- Symboltabellen

  Datenstrukturen, um Namen, Werte, Scopes und weitere Informationen zu speichern. Die Symboltabellen werden vor allem beim Parsen befüllt und bei der semantischen Analyse gelesen, aber auch der Lexer benötigt u.U. diese Informationen.

2.1.2.3. Weitere Begriffe

- Parse Tree, Concrete Syntax Tree

Repräsentiert die Struktur eines Satzes, wobei jeder Knoten dem Namen einer Regel der Grammatik entspricht. Die Blätter bestehen aus den Token samt ihren Werten.

- AST, (Abstract) Syntax Tree

  Vereinfachte Form des Parse Tree, wobei der Bezug auf die Element der Grammatik (mehr oder weniger) weggelassen wird.

- Annotierter AST

  Anmerkungen am AST, die für spätere Verarbeitungsstufen interessant sein könnten: Typ-Informationen, Optimierungsinformationen, …

- Zwischen-Code, IC

  Zwischensprache, die abstrakter ist als die dem AST zugrunde liegenden Konstrukte der Ausgangssprache. Beispielsweise könnten `while`-Schleifen durch entsprechende Label und Sprünge ersetzt werden. Wie genau dieser Zwischen-Code aussieht, muss der Compilerdesigner entscheiden. Oft findet man den Assembler-ähnlichen "3-Adressen-Code".

- Sprache

  Eine Sprache ist eine Menge gültiger Sätze. Die Sätze werden aus Wörtern gebildet, diese wiederum aus Zeichenfolgen.

- Grammatik

  Eine Grammatik beschreibt formal die Syntaxregeln für eine Sprache. Jede Regel in der Grammatik beschreibt dabei die Struktur eines Satzes oder einer Phrase.

2.1.3. Lexikalische Analyse: Wörter ("*Token*") erkennen

Die lexikalische Analyse (auch *Scanner* oder *Lexer* oder *Tokenizer* genannt) zerteilt den Zeichenstrom in eine Folge von Wörtern ("*Token*"). Die geschieht i.d.R. mit Hilfe von *regulären Ausdrücken*.

Dabei müssen unsinnige/nicht erlaubte Wörter erkannt werden.

Überflüssige Zeichen (etwa Leerzeichen) werden i.d.R. entfernt.

```
1  sp = 100;
```

```
1  <ID, sp>, <OP, =>, <INT, 100>, <SEM>
```

Anmerkung: In der obigen Darstellung werden die Werte der Token ("*Lexeme*") zusammen mit den Token "gespeichert". Alternativ können die Werte der Token auch direkt in der Symboltabelle gespeichert werden und in den Token nur der Verweis auf den jeweiligen Eintrag in der Tabelle.

2.1.4. Syntaxanalyse: Sätze erkennen

In der Syntaxanalyse (auch *Parser* genannt) wird die Tokensequenz in gültige Sätze unterteilt. Dazu werden in der Regel *kontextfreie Grammatiken* und unterschiedliche Parsing-Methoden (*top-down*, *bottom-up*) genutzt.

Dabei müssen nicht erlaubte Sätze erkannt werden.

```
<ID, sp>, <OP, =>, <INT, 100>, <SEM>
```

```
statement : assign SEM ;
assign : ID OP INT ;
```

```
                    statement                    =
                    /       \                  / \
                assign      SEM              sp  100
              /   |   \      |
            ID    OP  INT    ;
            |     |    |
            sp    =   100
```

Mit Hilfe der Produktionsregeln der Grammatik wird versucht, die Tokensequenz zu erzeugen. Wenn dies gelingt, ist der Satz (also die Tokensequenz) ein gültiger Satz im Sinne der Grammatik. Dabei sind die Token aus der lexikalischen Analyse die hier betrachteten Wörter!

Dabei entsteht ein sogenannter *Parse-Tree* (oder auch "*Syntax Tree*"; in der obigen Darstellung der linke Baum). In diesen Bäumen spiegeln sich die Regeln der Grammatik wider, d.h. zu einem Satz kann es durchaus verschiedene Parse-Trees geben.

Beim *AST* ("*Abstract Syntax Tree*") werden die Knoten um alle später nicht mehr benötigten Informationen bereinigt (in der obigen Darstellung der rechte Baum).

Anmerkung: Die Begriffe werden oft nicht eindeutig verwendet. Je nach Anwendung ist das Ergebnis des Parsers ein AST oder ein Parse-Tree.

Anmerkung: Man könnte statt `OP` auch etwa ein `ASSIGN` nutzen und müsste dann das "=" nicht extra als Inhalt speichern, d.h. man würde die Information im Token-Typ kodieren.

2.1.5. Vorschau: Parser implementieren

```
stat : assign | ifstat | ... ;
assign : ID '=' expr ';' ;
```

```
void stat() {
    switch (<<current token>>) {
        case ID : assign(); break;
        case IF : ifstat(); break;
        ...
        default : <<raise exception>>
    }
}
void assign() {
    match(ID);
    match('=');
    expr();
    match(';');
}
```

Der gezeigte Parser ist ein sogenannter “LL(1)”-Parser und geht von oben nach unten vor, d.h. ist ein Top-Down-Parser.

Nach dem Betrachten des aktuellen Tokens wird entschieden, welche Alternative vorliegt und in die jeweilige Methode gesprungen.

Die `match()`-Methode entspricht dabei dem Erzeugen von Blättern, d.h. hier werden letztlich die Token der Grammatik erkannt.

2.1.6. Semantische Analyse: Bedeutung erkennen

In der semantischen Analyse (auch *Context Handling* genannt) wird der AST zusammen mit der Symboltabelle geprüft. Dabei spielen Probleme wie Scopes, Namen und Typen eine wichtige Rolle.

Die semantische Analyse ist direkt vom Programmierparadigma der zu übersetzenden Sprache abhängig, d.h. müssen wir beispielsweise das Konzept von Klassen verstehen?

Als Ergebnis dieser Phase entsteht typischerweise ein *annotierter AST*.

```
{
    int x = 42;
    {
        int x = 7;
        x += 3;    // ???
    }
}
```

```
                                                    = {type: real, loc: tmp1}
sp = 100;                                          / \
                                                  /   \
                                               sp     inttofloat
                                        {type: real,       |
                                         loc: var b}      100
```

2.1.7. Zwischencode generieren

Aus dem annotierten AST wird in der Regel ein Zwischencode (“*Intermediate Code*”, auch “IC”) generiert. oft findet man hier den Assembler-ähnlichen “3-Adressen-Code”, in manchen Compilern wird als IC aber auch der AST selbst genutzt.

```
                  = {type: real, loc: tmp1}
                 / \
                /   \
             sp     inttofloat
      {type: real,       |
       loc: var b}      100
```

=> `t1 = inttofloat(100)`

2.1.8. Code optimieren

An dieser Stelle verlassen wir das Compiler-Frontend und begeben uns in das sogenannte *Backend*. Die Optimierung des Codes kann sehr unterschiedlich ausfallen, beispielsweise kann man den Zwischencode selbst optimieren, dann nach sogenanntem "Targetcode" übersetzen und diesen weiter optimieren, bevor das Ergebnis im letzten Schritt in Maschinencode übersetzt wird.

Die Optimierungsphase ist sehr stark abhängig von der Zielhardware. Hier kommen fortgeschrittene Mengen- und Graphalgorithmen zur Anwendung. Die Optimierung stellt den wichtigsten Teil aktueller Compiler dar.

Aus zeitlichen und didaktischen Gründen werden wir in dieser Veranstaltung den Fokus auf die Frontend-Phasen legen und die Optimierung nur grob streifen.

```
t1 = inttofloat(100) => t1 = 100.0
```

```
x = y*0; => x = 0;
```

2.1.9. Code generieren

- Maschinencode:

```
1  STD  t1, 100.0
```

- Andere Sprache:
  - Bytecode
  - C
  - …

2.1.10. Probleme

```
1  5*4+3
```

AST?

Problem: Vorrang von Operatoren

- Variante 1: `+(*(5, 4), 3)`
- Variante 2: `*(5, +(4, 3))`

```
1  stat : expr ';'
2       | ID '(' ')' ';'
3       ;
4  expr : ID '(' ')'
5       | INT
6       ;
```

2.1.11. Wrap-Up

- Compiler übersetzen Text in ein anderes Format

- Typische Phasen:
  1. Lexikalische Analyse
  2. Syntaxanalyse
  3. Semantische Analyse
  4. Generierung von Zwischencode
  5. Optimierung des (Zwischen-) Codes
  6. Codegenerierung

Zum Nachlesen

- Aho u. a. (2023): Kapitel 1 Introduction
- Grune u. a. (2012): Kapitel 1 Introduction

Lernziele

- k2: Ich kann die Struktur eines Compilers und die verschiedenen Phasen und deren Aufgaben erklären

2.2. Bandbreite der Programmiersprachen

TL;DR

Am Beispiel des Abzählreims “99 Bottles of Beer” werden (ganz kurz) verschiedene Programmiersprachen betrachtet. Jede der Sprachen hat ihr eigenes Sprachkonzept (Programmierparadigma) und auch ein eigenes Typ-System sowie ihre eigene Strategie zur Speicherverwaltung und Abarbeitung.
Auch wenn die Darstellung längst nicht vollständig ist, macht sie doch deutlich, dass Compiler teilweise sehr unterschiedliche Konzepte “verstehen” müssen.

Videos

Vorlesung [YT], [HSBI]

2.2.1. 99 Bottles of Beer

> 99 bottles of beer on the wall, 99 bottles of beer. Take one down and pass it around, 98 bottles of beer on the wall.
>
> 98 bottles of beer on the wall, 98 bottles of beer. Take one down and pass it around, 97 bottles of beer on the wall.
>
> […]

> 2 bottles of beer on the wall, 2 bottles of beer. Take one down and pass it around, 1 bottle of beer on the wall.
>
> 1 bottle of beer on the wall, 1 bottle of beer. Take one down and pass it around, no more bottles of beer on the wall.
>
> No more bottles of beer on the wall, no more bottles of beer. Go to the store and buy some more, 99 bottles of beer on the wall.

Quelle: Abzählreim "99 Bottles of Beer" nach "Lyrics of the song 99 Bottles of Beer" on 99-bottles-of-beer.net

2.2.2. Imperativ, Hardwarenah: C

```c
#define MAXBEER (99)
void chug(int beers);
main() {
   register beers;
   for(beers = MAXBEER; beers; chug(beers--))  puts("");
   puts("\nTime to buy more beer!\n");
}
void chug(register beers) {
   char howmany[8], *s;
   s = beers != 1 ? "s" : "";
   printf("%d bottle%s of beer on the wall,\n", beers, s);
   printf("%d bottle%s of beeeeer . . . ,\n", beers, s);
   printf("Take one down, pass it around,\n");
   if(--beers) sprintf(howmany, "%d", beers); else strcpy(howmany, "No more");
   s = beers != 1 ? "s" : "";
   printf("%s bottle%s of beer on the wall.\n", howmany, s);
}
```

Quelle: "Language C" by Bill Wein on 99-bottles-of-beer.net

- Imperativ
- Procedural
- Statisches Typsystem
- Resourcenschonend, aber "unsicher": Programmierer muss wissen, was er tut
- Relativ hardwarenah
- Einsatz: Betriebssysteme, Systemprogrammierung

2.2.3. Imperativ, Objektorientiert: Java

```java
class bottles {
    public static void main(String args[]) {
        String s = "s";
        for (int beers=99; beers>-1;) {
            System.out.print(beers + " bottle" + s + " of beer on the wall, ");
```

```
6               System.out.println(beers + " bottle" + s + " of beer, ");
7               if (beers==0) {
8                   System.out.print("Go to the store, buy some more, ");
9                   System.out.println("99 bottles of beer on the wall.\n");
10                  System.exit(0);
11              } else
12                  System.out.print("Take one down, pass it around, ");
13              s = (--beers == 1)?"":"s";
14              System.out.println(beers + " bottle" + s + " of beer on the wall.\n
                    ");
15          }
16      }
17  }
```

Quelle: "Language Java" by Sean Russell on 99-bottles-of-beer.net

- Imperativ
- Objektorientiert
- Multi-Threading
- Basiert auf C/C++
- Statisches Typsystem
- Automatische Garbage Collection
- "Sichere" Architektur: Laufzeitumgebung fängt viele Probleme ab
- Architekturneutral: Nutzt Bytecode und eine JVM
- Einsatz: High-Level All-Purpose Language

2.2.4. Logisch: Prolog

```
1  bottles :-
2      bottles(99).
3
4  bottles(1) :-
5      write('1 bottle of beer on the wall, 1 bottle of beer,'), nl,
6      write('Take one down, and pass it around,'), nl,
7      write('Now they are all gone.'), nl,!.
8  bottles(X) :-
9      write(X), write(' bottles of beer on the wall,'), nl,
10     write(X), write(' bottles of beer,'), nl,
11     write('Take one down and pass it around,'), nl,
12     NX is X - 1,
13     write(NX), write(' bottles of beer on the wall.'), nl, nl,
14     bottles(NX).
```

Quelle: "Language Prolog" by M@ on 99-bottles-of-beer.net

- Deklarativ
- Logisch: Definition von Fakten und Regeln; eingebautes Beweissystem

- Einsatz: Theorem-Beweisen, Natural Language Programming (NLP), Expertensysteme, …

2.2.5. Funktional: Haskell

```haskell
bottles 0 = "no more bottles"
bottles 1 = "1 bottle"
bottles n = show n ++ " bottles"

verse 0   = "No more bottles of beer on the wall, no more bottles of beer.\n"
          ++ "Go to the store and buy some more, 99 bottles of beer on the wall.
              "

verse n   = bottles n ++ " of beer on the wall, " ++ bottles n ++ " of beer.\n"
          ++ "Take one down and pass it around, " ++ bottles (n-1)
                                                  ++ " of beer on the wall.\n"

main      = mapM (putStrLn . verse) [99,98..0]
```

Quelle: "Language Haskell" by Iavor on 99-bottles-of-beer.net

- Deklarativ
- Funktional
- Lazy, pure
- Statisches Typsystem
- Typinferenz
- Algebraische Datentypen, Patternmatching
- Einsatz: Compiler, DSL, Forschung

2.2.6. Brainfuck



Quelle: Screenshot of "Language Brainfuck" by Michal Wojciech Tarnowski on 99-bottles-of-beer.net

- Imperativ
- Feldbasiert (analog zum Band der Turingmaschine)
- 8 Befehle: Zeiger und Zellen inkrementieren/dekrementieren, Aus- und Eingabe, Sprungbefehle

2.2.7. Programmiersprache Lox

```
fun fib(x) {
    if (x == 0) {
        return 0;
    } else {
        if (x == 1) {
            return 1;
        } else {
            fib(x - 1) + fib(x - 2);
        }
    }
}

var wuppie = fib;
wuppie(4);
```

- Die Sprache “Lox” finden Sie hier: craftinginterpreters.com/the-lox-language.html
- C-ähnliche Syntax
- Imperativ, objektorientiert, Funktionen als *First Class Citizens*, Closures

- Dynamisch typisiert
- Garbage Collector
- Statements und Expressions
- (Kleine) Standardbibliothek eingebaut

Die Sprache ähnelt stark anderen modernen Sprachen und ist gut geeignet, um an ihrem Beispiel Themen wie Scanner/Parser/AST, Interpreter, Object Code und VM zu studieren :)

2.2.8. Wrap-Up

- Compiler übersetzen formalen Text in ein anderes Format

- Berücksichtigung von unterschiedlichen
  - Sprachkonzepten (Programmierparadigmen)
  - Typ-Systemen
  - Speicherverwaltungsstrategien
  - Abarbeitungsstrategien

Zum Nachlesen

- Aho u. a. (2023): Kapitel 1 Introduction
- Grune u. a. (2012): Kapitel 1 Introduction

Lernziele

- k1: Ich kenne verschiedene Beispiele für verschiedene Programmiersprachen und Paradigmen

2.3. Anwendungen

TL;DR

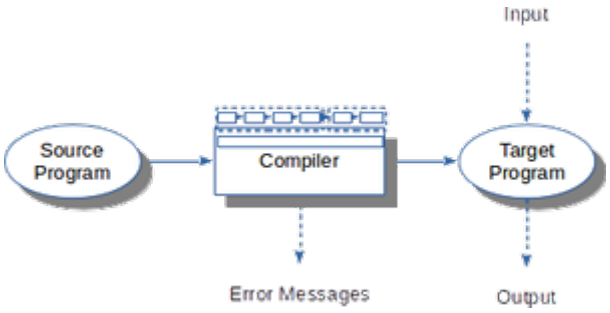
Es gibt verschiedene Anwendungsmöglichkeiten für Compiler. Je nach Bedarf wird dabei die komplette Toolchain durchlaufen oder es werden Stufen ausgelassen. Häufig genutzte Varianten sind dabei:

- “Echte” Compiler: Übersetzen Sourcecode nach ausführbarem Maschinencode
- Interpreter: Interaktive Ausführung von Sourcecode
- Virtuelle Maschinen als Zwischending zwischen Compiler und Interpreter
- Transpiler: Übersetzen formalen Text nach formalem Text
- Analysetools: Parsen den Sourcecode, werten die Strukturen aus

Videos

Vorlesung [YT], [HSBI]

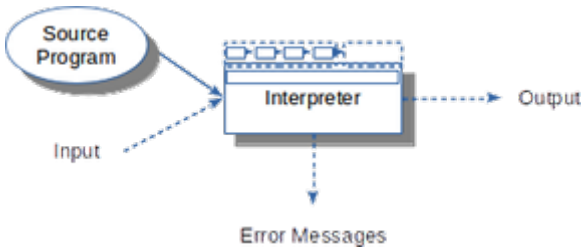
2.3.1. Anwendung: Compiler



Wie oben diskutiert: Der Sourcecode durchläuft alle Phasen des Compilers, am Ende fällt ein ausführbares Programm heraus. Dieses kann man starten und ggf. mit Inputdaten versehen und erhält den entsprechenden Output. Das erzeugte Programm läuft i.d.R. nur auf einer bestimmten Plattform.

Beispiele: gcc, clang, …

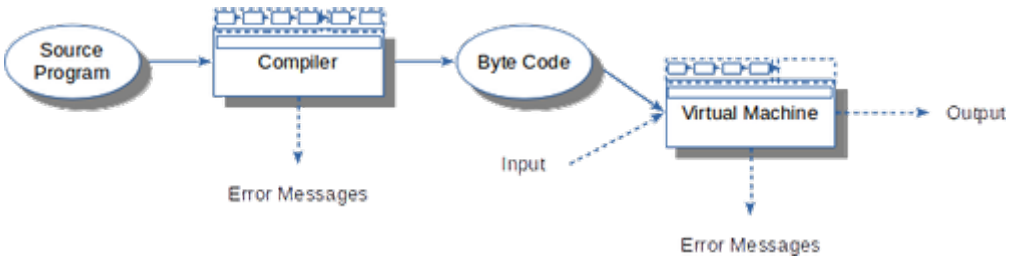
2.3.2. Anwendung: Interpreter



Beim Interpreter durchläuft der Sourcecode nur das Frontend, also die Analyse. Es wird kein Code erzeugt, stattdessen führt der Interpreter die Anweisungen im AST bzw. IC aus. Dazu muss der Interpreter mit den Eingabedaten beschickt werden. Typischerweise hat man hier eine “Read-Eval-Print-Loop” (*REPL*).

Beispiele: Python

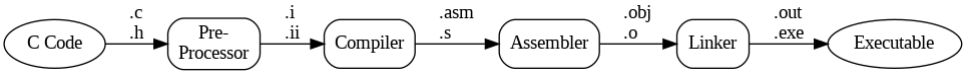
2.3.3. Anwendung: Virtuelle Maschinen



Hier liegt eine Art Mischform aus Compiler und Interpreter vor: Der Compiler übersetzt den Quellcode in ein maschinenunabhängiges Zwischenformat (“Byte-Code”). Dieser wird von der virtuellen Maschine (“VM”) gelesen und ausgeführt. Die VM kann also als Interpreter für Byte-Code betrachtet werden.

Beispiel: Java mit seiner JVM

2.3.4. Anwendung: C-Toolchain

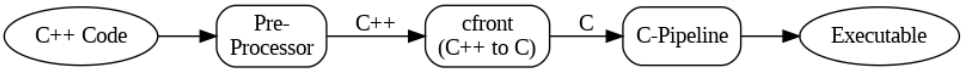


Erinnern Sie sich an die LV “Systemprogrammierung” im dritten Semester :-)

Auch wenn es so aussieht, als würde der C-Compiler aus dem Quelltext direkt das ausführbare Programm erzeugen, finden hier dennoch verschiedene Stufen statt. Zuerst läuft ein Präprozessor über den Quelltext und ersetzt alle `#include` und `#define` etc., danach arbeitet der C-Compiler, dessen Ausgabe wiederum durch einen Assembler zu ausführbarem Maschinencode transformiert wird.

Beispiele: gcc, clang, …

2.3.5. Anwendung: C++-Compiler



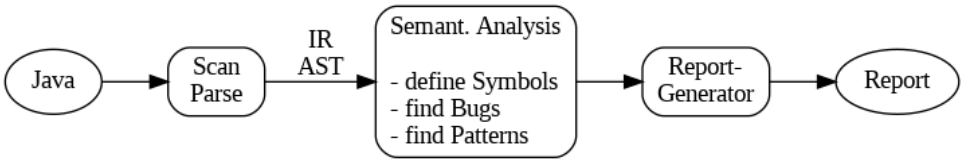
C++ hat meist keinen eigenen (vollständigen) Compiler :-)

In der Regel werden die C++-Konstrukte durch `cfront` nach C übersetzt, so dass man anschließend auf die etablierten Tools zurückgreifen kann.

Dieses Vorgehen werden Sie relativ häufig finden. Vielleicht sogar in Ihrem Projekt …

Beispiel: g++

2.3.6. Anwendung: Bugfinder



Tools wie FindBugs analysieren den (Java-) Quellcode und suchen nach bekannten Fehlermustern. Dazu benötigen sie nur den Analyse-Teil eines Compilers!

Auf dem AST kann dann nach vorab definierten Fehlermustern gesucht werden (Stichwort “Graphmatching”). Dazu fällt die semantische Analyse entsprechend umfangreicher aus als normal.

Zusätzlich wird noch eine Reporting-Komponente benötigt, da die normalen durch die Analysekette erzeugten Fehlermeldungen nicht helfen (bzw. sofern der Quellcode wohlgeformter Code ist, würden ja keine Fehlermeldungen durch die Analyseeinheit generiert).

Beispiele: SpotBugs, Checkstyle, ESLint, …

2.3.7. Anwendung: Pandoc

Pandoc ist ein universeller und modular aufgebauter Textkonverter, der mit Hilfe verschiedener *Reader* unterschiedliche Textformate einlesen und in ein Zwischenformat (hier JSON) transformieren kann. Über verschiedene *Writer* können aus dem Zwischenformat dann Dokumente in den gewünschten Zielformaten erzeugt werden.

Die Reader entsprechen der Analyse-Phase und die Writer der Synthese-Phase eines Compilers. Anstelle eines ausführbaren Programms (Maschinencode) wird ein anderes Textformat erstellt/ausgegeben.

Beispielsweise wird aus diesem Markdown-Schnipsel …

```
Dies ist ein Satz mit
*  einem Stichpunkt, und
*  einem zweiten Stichpunkt.
```

… dieses Zwischenformat erzeugt, …

```
{"blocks":[{"t":"Para","c":[{"t":"Str","c":"Dies"},{"t":"Space"},
           {"t":"Str","c":"ist"},{"t":"Space"},{"t":"Str","c":"ein"},
           {"t":"Space"},{"t":"Str","c":"Satz"},{"t":"Space"},
           {"t":"Str","c":"mit"}]},
           {"t":"BulletList","c":[[{"t":"Plain","c":[{"t":"Str","c":"einem"},{"
               t":"Space"},{"t":"Str","c":"Stichpunkt,"},{"t":"Space"},{"t":"Str
               ","c":"und"}]}],[{"t":"Plain","c":[{"t":"Str","c":"einem"},{"t":"
               Space"},{"t":"Str","c":"zweiten"},{"t":"Space"},{"t":"Str","c":"
               Stichpunkt."}]}]]}],"pandoc-api-version":[1,17,0,4],"meta":{}}
```

… und daraus schließlich dieser TeX-Code.

```
Dies ist ein Satz mit
\begin{itemize}
\tightlist
\item einem Stichpunkt, und
\item einem zweiten Stichpunkt.
\end{itemize}
```

Im Prinzip ist Pandoc damit ein Beispiel für Compiler, die aus einem formalen Text nicht ein ausführbares Programm erzeugen (Maschinencode), sondern einen anderen formalen Text. Dieser werden häufig auch “Transpiler” genannt.

Weitere Beispiele:

- Lexer-/Parser-Generatoren: ANTLR, Flex, Bison, …: formale Grammatik nach Sourcecode

- CoffeeScript: CoffeeScript (eine Art “JavaScript light”) nach JavaScript
- Emscripten: C/C++ nach LLVM nach WebAssembly (tatsächlich kann LLVM-IR auch direkt als Input verwendet werden)
- Fitnesse: Word/Wiki nach ausführbare Unit-Tests

2.3.8. Was bringt mir das?

Beschäftigung mit dem schönsten Thema in der Informatik ;-)

2.3.8.1. Auswahl einiger Gründe für den Besuch des Moduls “Compilerbau”

- Erstellung eigener kleiner Interpreter/Compiler
  - Einlesen von komplexen Daten
  - DSL als Brücke zwischen Stakeholdern
  - DSL zum schnelleren Programmieren (denken Sie etwa an CoffeeScript …)
- Wie funktionieren FindBugs, Lint und ähnliche Tools?
  - Statische Codeanalyse: Dead code elimination
- Language-theoretic Security: LangSec
- Verständnis für bestimmte Sprachkonstrukte und -konzepte (etwa `virtual` in C++)
- Vertiefung durch Besuch “echter” Compilerbau-Veranstaltungen an Uni möglich :-)
- Wie funktioniert:
  - ein Python-Interpreter?
  - das Syntaxhighlighting in einem Editor oder in Doxygen?
  - ein Hardwarecompiler (etwa VHDL)?
  - ein Text-Formatierer (TeX, LaTeX, …)?
  - CoffeeScript oder Emscripten?
- Wie kann man einen eigenen Compiler/Interpreter basteln, etwa für
  - MiniJava (mit C-Backend)
  - Brainfuck
  - Übersetzung von JSON nach XML
- Um eine profundes Kenntnis von Programmiersprachen zu erlangen, ist eine Beschäftigung mit ihrer Implementierung unerlässlich.
- Viele Grundtechniken der Informatik und elementare Datenstrukturen wie Keller, Listen, Abbildungen, Bäume, Graphen, Automaten etc. finden im Compilerbau Anwendung. Dadurch schließt sich in gewisser Weise der Kreis in der Informatikausbildung …
- Aufgrund seiner Reife gibt es hervorragende Beispiele von formaler Spezifikation im Compilerbau.
- Mit dem Gebiet der formalen Sprachen berührt der Compilerbau interessante Aspekte moderner Linguistik. Damit ergibt sich letztlich eine Verbindung zur KI …
- Die Unterscheidung von Syntax und Semantik ist eine grundlegende Technik in fast allen formalen Systeme.

2.3.8.2. Parser-Generatoren (Auswahl)

Diese Tools könnte man beispielsweise nutzen, um seine eigene Sprache zu basteln.

- ANTLR (ANother Tool for Language Recognition) is a powerful parser generator for reading, processing, executing, or translating structured text or binary files: github.com/antlr/antlr4
- Grammars written for ANTLR v4; expectation that the grammars are free of actions: github.com/antlr/grammars-v4
- An incremental parsing system for programmings tools: github.com/tree-sitter/tree-sitter
- Flex, the Fast Lexical Analyzer - scanner generator for lexing in C and C++: github.com/westes/flex
- Bison is a general-purpose parser generator that converts an annotated context-free grammar into a deterministic LR or generalized LR (GLR) parser employing LALR(1) parser tables: gnu.org/software/bison
- Parser combinators for binary formats, in C: github.com/UpstandingHackers/hammer
- Eclipse Xtext is a language development framework: github.com/eclipse/xtext

2.3.8.3. Statische Analyse, Type-Checking und Linter

Als Startpunkt für eigene Ideen. Oder Verbessern/Erweitern der Projekte …

- Pluggable type-checking for Java: github.com/typetools/checker-framework
- SpotBugs is FindBugs' successor. A tool for static analysis to look for bugs in Java code: github.com/spotbugs/spotbugs
- An extensible cross-language static code analyzer: github.com/pmd/pmd
- Checkstyle is a development tool to help programmers write Java code that adheres to a coding standard: github.com/checkstyle/checkstyle
- JaCoCo - Java Code Coverage Library: github.com/jacoco/jacoco
- Sanitizers: memory error detector: github.com/google/sanitizers
- JSHint is a tool that helps to detect errors and potential problems in your JavaScript code: github.com/jshint/jshint
- Haskell source code suggestions: github.com/ndmitchell/hlint
- Syntax checking hacks for vim: github.com/vim-syntastic/syntastic

2.3.8.4. DSL (Domain Specific Language)

- NVIDIA Material Definition Language SDK: github.com/NVIDIA/MDL-SDK
- FitNesse – The Acceptance Test Wiki: github.com/unclebob/fitnesse

Hier noch ein Framework, welches auf das Erstellen von DSL spezialisiert ist:

- Eclipse Xtext is a language development framework: github.com/eclipse/xtext

2.3.8.5. Konverter von X nach Y

- Emscripten: An LLVM-to-JavaScript Compiler: github.com/kripken/emscripten
- "Unfancy JavaScript": github.com/jashkenas/coffeescript

- Universal markup converter: github.com/jgm/pandoc
- Übersetzung von JSON nach XML

2.3.8.6. Odds and Ends

- How to write your own compiler: staff.polito.it/silvano.rivoira/HowToWriteYourOwnCompiler.htm
- Building a modern functional compiler from first principles: github.com/sdiehl/write-you-a-haskell
- Language-theoretic Security: LangSec
- Generierung von automatisierten Tests mit Esprima: heise.de/-4129726
- Eigener kleiner Compiler/Interpreter, etwa für
  - MiniJava mit C-Backend oder sogar LLVM-Backend
  - Brainfuck

2.3.8.7. Als weitere Anregung: Themen der Mini-Projekte im W17

- Java2UMLet
- JavaDoc-to-Markdown
- Validierung und Übersetzung von Google Protocol Buffers v3 nach JSON
- svg2tikz
- SwaggerLang – Schreiben wie im Tagebuch
- Markdown zu LaTeX
- JavaDocToLaTeX
- MySQL2REDIS-Parser

2.3.9. Wrap-Up

- Compiler übersetzen formalen Text in ein anderes Format

- Nicht alle Stufen kommen immer vor => unterschiedliche Anwendungen
  - “Echte” Compiler: Sourcecode nach Maschinencode
  - Interpreter: Interaktive Ausführung
  - Virtuelle Maschinen als Zwischending zwischen Compiler und Interpreter
  - Transpiler: formaler Text nach formalem Text
  - Analysetools: Parsen den Sourcecode, werten die Strukturen aus

Zum Nachlesen

- Aho u. a. (2023): Kapitel 1 Introduction
- Grune u. a. (2012): Kapitel 1 Introduction

Lernziele

- k1: Ich kenne verschiedene Anwendungen für Compiler durch Einsatz bestimmter Stufen der Compiler-Pipeline

3. Reguläre Sprachen, kontextfreie Grammatiken und Sprachen, lexikalische und syntaktische Analyse

In der lexikalischen Analyse soll ein Lexer (auch “Scanner”) den Zeichenstrom in eine Folge von Token zerlegen. Zur Spezifikation der Token werden in der Regel reguläre Ausdrücke verwendet.

In der syntaktischen Analyse arbeitet ein Parser mit dem Tokenstrom, der vom Lexer kommt. Mit Hilfe einer Grammatik wird geprüft, ob gültige Sätze im Sinne der Sprache/Grammatik gebildet wurden. Der Parser erzeugt dabei den Parse-Tree. Man kann verschiedene Parser unterscheiden, beispielsweise die LL- und die LR-Parser.

3.1. Reguläre Sprachen, Ausdrucksstärke (Teil 1)

Weitere Unterlagen

- Annotierte Folien: Reguläre Sprachen, Ausdrucksstärke (Teil 1)

3.1.1. Motivation

3.1.1.1. Was muss ein Compiler wohl als erstes tun?

Hier entsteht ein Tafelbild.

3.1.1.2. Themen für heute

- Lexer
- Endliche Automaten
- Reguläre Sprachen

3.1.2. Endliche Automaten

3.1.2.1. Alphabete

Def.: Ein **Alphabet** Σ ist eine endliche, nicht-leere Menge von Symbolen. Die Symbole eines Alphabets heißen *Buchstaben*.

Def.: Ein **Wort** w *über einem Alphabet* Σ ist eine endliche Folge von Symbolen aus Σ. ϵ ist das leere Wort. Die *Länge* $|w|$ eines Wortes w ist die Anzahl von Buchstaben, die es enthält (Kardinalität).

Def.: Eine **Sprache** L *über einem Alphabet* Σ ist eine Menge von Wörtern über diesem Alphabet. Sprachen können endlich oder unendlich viele Wörter enthalten.

3.1.2.2. State machine

Hier entsteht ein Tafelbild.

3.1.2.3. Deterministische endliche Automaten

Bestimmte State machines:

- Eingaben bestimmen Zustandsübergänge
- Zustandsübergänge sind eindeutig
- Es gibt Anfang(szustand) und End(zuständ)e

3.1.2.4. Wie definieren wir das formal?

Hier entsteht ein Tafelbild.

3.1.2.5. Def.: Deterministischer endlicher Automat

Def.: Ein **deterministischer endlicher Automat** (DFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) **Übergangsfunktion** $(Q \times \Sigma) \to Q$, δ kann partiell sein
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

3.1.2.6. Beispiel

Hier entsteht ein Tafelbild.

3.1.2.7. Eingabewörter statt Buchstaben

Def.: Wir definieren $\delta^* : (Q \times \Sigma^*) \to Q$: induktiv wie folgt:

- Basis: $\delta^*(q, \epsilon) = q \, \forall q \in Q$
- Induktion: $\delta^*(q, a_1, \ldots, a_n) = \delta(\delta^*(q, a_1, \ldots, a_{n-1}), a_n)$

Def.: Ein DFA akzeptiert ein Wort $w \in \Sigma^*$ genau dann, wenn $\delta^*(q_0, w) \in F$.

3.1.2.8. Beispiel

Hier entsteht ein Tafelbild.

3.1.2.9. Nichtdeterministische endliche Automaten

Hier entsteht ein Tafelbild.

3.1.2.10. Def.: Nichtdeterministischer Automat

Def.: Ein **nichtdeterministischer endlicher Automat** (NFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) **Übergangsfunktion** $(Q \times \Sigma) \to Q$
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

3.1.2.11. Akzeptierte Sprachen

Def.: Sei A ein DFA oder ein NFA. Dann ist **L(A)** die von A akzeptierte Sprache, d. h.

$L(A) = \{\text{Wörter } w \mid \delta^*(q_0, w) \in F\}$

3.1.2.12. Wozu NFAs im Compilerbau?

Pattern Matching (Erkennung von Schlüsselwörtern, Bezeichnern, …) geht mit NFAs.

NFAs sind so nicht zu programmieren, aber:

Satz: Eine Sprache L wird von einem NFA akzeptiert $\Leftrightarrow$ L wird von einem DFA akzeptiert.

D. h. es existieren Algorithmen zur

- Umwandlung von NFAs in DFAS
- Minimierung von DFAs

3.1.3. Reguläre Sprachen

3.1.3.1. Reguläre Ausdrücke definieren Sprachen

Def.: Induktive Definition von **regulären Ausdrücken** (regex) und der von ihnen repräsentierten Sprache **L**:

- Basis:
  - ϵ und $\emptyset$ sind reguläre Ausdrücke mit $L(\epsilon) = \{\epsilon\}$, $L(\emptyset) = \emptyset$
  - Sei a ein Symbol $\Rightarrow$ a ist ein regex mit $L(a) = \{a\}$
- Induktion: Seien $E,\ F$ reguläre Ausdrücke. Dann gilt:
  - $E + F$ ist ein regex und bezeichnet die Vereinigung $L(E + F) = L(E) \cup L(F)$
  - EF ist ein regex und bezeichnet die Konkatenation $L(EF) = L(E)L(F)$
  - E^* ist ein regex und bezeichnet die Kleene-Hülle $L(E^*) = (L(E))^*$
  - (E) ist ein regex mit $L((E)) = L(E)$

Vorrangregeln der Operatoren für reguläre Ausdrücke: *, Konkatenation, +

3.1.3.2. Beispiel

Hier entsteht ein Tafelbild.

3.1.3.3. Wichtige Identitäten

Satz: Sei A ein DFA $\Rightarrow \exists$ regex R mit $L(A) = L(R)$.

Satz: Sei E ein regex $\Rightarrow \exists$ DFA A mit $L(E) = L(A)$.

3.1.3.4. Formale Grammatiken

Hier entsteht ein Tafelbild.

3.1.3.5. Formale Definition formaler Grammatiken

Def.: Eine *formale Grammatik* ist ein 4-Tupel $G = (N, T, P, S)$ aus

- N: endliche Menge von **Nichtterminalen**
- T: endliche Menge von **Terminalen**, $N \cap T = \emptyset$
- $S \in N$: **Startsymbol**
- P: endliche Menge von **Produktionen** der Form

$X \to Y$ mit $X \in (N \cup T)^* N (N \cup T)^*, Y \in (N \cup T)^*$

3.1.3.6. Ableitungen

Def.: Sei $G = (N, T, P, S)$ eine Grammatik, sei $\alpha A \beta$ eine Zeichenkette über $(N \cup T)^*$ und sei $A \to \gamma$ eine Produktion von G.

Wir schreiben: $\alpha A \beta \Rightarrow \alpha \gamma \beta$ ($\alpha A \beta$ leitet $\alpha \gamma \beta$ ab).

Def.: Wir definieren die Relation $\overset{*}{\Rightarrow}$ induktiv wie folgt:

- Basis: $\forall \alpha \in (N \cup T)^* \alpha \overset{*}{\Rightarrow} \alpha$ (Jede Zeichenkette leitet sich selbst ab.)
- Induktion: Wenn $\alpha \overset{*}{\Rightarrow} \beta$ und $\beta \Rightarrow \gamma$ dann $\alpha \overset{*}{\Rightarrow} \gamma$

Def.: Sei $G = (N, T, P, S)$ eine formale Grammatik. Dann ist $L(G) = \{$Wörter w über $T \mid S \overset{*}{\Rightarrow} w\}$ die von G erzeugte Sprache.

3.1.3.7. Beispiel

Hier entsteht ein Tafelbild.

3.1.3.8. Reguläre Grammatiken

Def.: Eine **reguläre (oder type-3-) Grammatik** ist eine formale Grammatik mit den folgenden Einschränkungen:

- Alle Produktionen sind entweder von der Form
  - $X \to aY$ mit $X \in N, a \in T, Y \in N$ (*rechtsreguläre* Grammatik) oder
  - $X \to Ya$ mit $X \in N, a \in T, Y \in N$ (*linksreguläre* Grammatik)
- $X \to \epsilon$ ist erlaubt

3.1.3.9. Beispiel

Hier entsteht ein Tafelbild.

3.1.3.10. Reguläre Sprachen

Satz: Die von endlichen Automaten akzeptiert Sprachklasse, die von regulären Ausdrücken beschriebene Sprachklasse und die von regulären Grammatiken erzeugte Sprachklasse sind identisch und heißen **reguläre Sprachen**.

3.1.4. Wrap-Up

3.1.4.1. Wrap-Up

- Definition und Aufgaben von Lexern
- DFAs und NFAs
- Reguläre Ausdrücke
- Reguläre Grammatiken
- Zusammenhänge zwischen diesen Mechanismen und Lexern, bzw. Lexergeneratoren

Zum Nachlesen

- Aho u. a. (2023): Abschnitt 2.6 und Kapitel 3
- Torczon und Cooper (2012): Kapitel 2
- Parr (2014)

Lernziele

- k1: Ich kenne DFAs
- k1: Ich kenne reguläre Ausdrücke
- k1: Ich kenne reguläre Grammatiken
- k2: Ich kann die Zusammenhänge und Gesetzmäßigkeiten bzgl. der oben genannten Konstrukte an einem Beispiel erklären
- k3: Ich kann für eine Fragestellung DFAs, reguläre Ausdrücke, reguläre Grammatiken entwickeln
- k3: Ich kann einen DFA entwickeln, der alle Schlüsselwörter, Namen und weitere Symbole einer Programmiersprache akzeptiert

3.2. Reguläre Sprachen, Ausdrucksstärke (Teil 2)

Weitere Unterlagen

- Annotierte Folien: Reguläre Sprachen, Ausdrucksstärke (Teil 2)

3.2.1. Wiederholung

3.2.1.1. Endliche Automaten, reguläre Ausdrücke, reguläre Grammatiken, reguläre Sprachen

- Wie sind DFAs und NFAs definiert?
- Was sind reguläre Ausdrücke?
- Was sind formale und reguläre Grammatiken?
- In welchem Zusammenhang stehen all diese Begriffe?

3.2.2. Motivation

3.2.2.1. Was haben reguläre Sprachen mit Compilern zu tun?

Hier entsteht ein Tafelbild.

3.2.2.2. Themen für heute

- Reguläre Sprachen
- Lexer
- Grenzen regulärer Sprachen

3.2.2.3. Wozu reguläre Sprachen im Compilerbau?

Reguläre Ausdrücke

- definieren Schlüsselwörter und alle weiteren Symbole einer Programmiersprache, z. B. den Aufbau von Gleitkommazahlen
- werden (oft von einem Generator) in DFAs umgewandelt
- sind die Basis des *Scanners* oder *Lexers*

3.2.3. Lexer

3.2.3.1. Ein Lexer ist mehr als ein DFA

Ein **Lexer**

- kann aus regulären Ausdrücken automatisch generiert werden
- wandelt mittels DFAs aus regulären Ausdrücken die Folge von Zeichen der Quelldatei in eine Folge von sog. Token um
- bekommt als Input eine Liste von Paaren aus regulären Ausdrücken und Tokennamen, z. B. (“while”, WHILE)
- Kommentare und Strings müssen richtig erkannt werden. (Schachtelungen)
- liefert Paare von Token und deren Werte, sofern benötigt, z. B. (WHILE, _), oder (IDENTIFIER, “radius”) oder (INTEGERZAHL, “334”)

3.2.3.2. Wofür reichen reguläre Sprachen nicht?

Für z. B. alle Sprachen, in deren Wörtern Zeichen über eine Konstante hinaus gezählt werden müssen. Diese Sprachen lassen sich oft mit Variablen im Exponenten beschreiben, die unendlich viele Werte annehmen können.

- $a^i b^{2*i}$ ist nicht regulär

- $a^i b^{2*i}$ für $0 \leq i \leq 3$ ist regulär

- Wo finden sich die oben genannten Variablen bei einem DFA wieder?
- Warum ist die erste Sprache oben nicht regulär, die zweite aber?

3.2.3.3. Wie geht es weiter?

Ein **Parser**

- führt mit Hilfe des Tokenstreams vom Lexer die Syntaxanalyse durch
- basiert auf einer sog. kontextfreien Grammatik, deren Terminale die Token sind
- liefert die syntaktische Struktur in Form eines Ableitungsbaums (**syntax tree**, **parse tree**), bzw. einen **AST** (abstract syntax tree) ohne redundante Informationen im Ableitungsbaum (z. B. Semikolons)
- liefert evtl. Fehlermeldungen

3.2.4. Wrap-Up

3.2.4.1. Wrap-Up

- Definition und Aufgaben von Lexern
- Zusammenhänge zwischen diesen Mechanismen und Lexern, bzw. Lexergeneratoren
- Grenzen regulärer Sprachen

Zum Nachlesen

- Aho u. a. (2023): Abschnitt 2.6 und Kapitel 3
- Torczon und Cooper (2012): Kapitel 2
- Parr (2014)

Lernziele

- k1: Ich kenne die Aufgaben eines Lexers
- k1: Ich kenne die Zusammenhänge zwischen DFAs, regulären Ausdrücken, regulären Grammatiken und Lexern
- k2: Ich kann für eine Beispielsprache begründen, warum sie nicht mit einem der oben genannten Mechanismen beschrieben werden kann

3.3. CFG

Weitere Unterlagen

- Annotierte Folien: CFG

3.3.1. Wiederholung

3.3.1.1. Endliche Automaten, reguläre Ausdrücke, reguläre Grammatiken, reguläre Sprachen

- Was ist ein Lexer?
- In welchem Zusammenhang stehen Lexer und reguläre Sprachen?
- Was können Lexer nicht?

3.3.2. Motivation

3.3.2.1. Was brauchen wir jetzt?

3.3.2.2. Themen für heute

- PDAs: mächtiger als DFAs, NFAs
- kontextfreie Grammatiken und Sprachen: mächtiger als reguläre Grammatiken und Sprachen
- DPDAs und deterministisch kontextfreie Grammatiken: die Grundlage der Syntaxanalyse im Compilerbau
- Syntaxanalyse

3.3.2.3. Einordnung: Erweiterung der Automatenklasse DFA, um komplexere Sprachen als die regulären akzeptieren zu können

Wir spendieren den DFAs einen möglichst einfachen, aber beliebig großen, Speicher, um zählen und matchen zu können. Wir suchen dabei konzeptionell die “kleinstmögliche” Erweiterung, die die akzeptierte Sprachklasse gegenüber DFAs vergrößert.

- Der konzeptionell einfachste Speicher ist ein Stack. Wir haben keinen wahlfreien Zugriff auf die gespeicherten Werte.
- Es soll eine deterministische und eine indeterministische Variante der neuen Automatenklasse geben.
- In diesem Zusammenhang wird der Stack auch Keller genannt.

3.3.2.4. Kellerautomaten (Push-Down-Automata, PDAs)

Def.: Ein Kellerautomat (PDA) $P = (Q,\ \Sigma,\ \Gamma,\ \delta,\ q_0,\ \bot,\ F)$ ist ein Septupel mit:

Q:	eine endliche Menge von Zuständen
Σ:	eine endliche Menge von Eingabesymbolen
Γ:	ein endliches Kelleralphabet
δ:	die Übergangsfunktion $\delta : \mathrm{Q} \times \Sigma \cup \{\epsilon\} \times \Gamma \to 2^{Q \times \Gamma^*}$
q_0:	der Startzustand
$\bot \in \Gamma$:	der anfängliche Kellerinhalt, symbolisiert den leeren Keller ($\bot$ = bottom)
$F \subseteq Q$:	die Menge von Endzuständen

Abbildung 3.1.: Definition eines PDAs

Ein PDA ist per Definition nichtdeterministisch und kann spontane Zustandsübergänge durchführen.

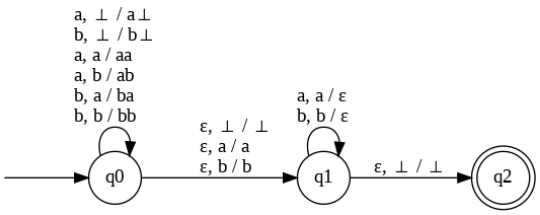
3.3.2.5. Was kann man damit akzeptieren?

Strukturen mit paarweise zu matchenden Symbolen.

Bei jedem Zustandsübergang wird ein Zeichen (oder ϵ) aus der Eingabe gelesen, ein Symbol von Keller genommen. Diese und das Eingabezeichen bestimmen den Folgezustand und eine Zeichenfolge, die auf den Stack gepackt wird. Dabei wird ein Symbol, (z. B. eines, das später mit einem Eingabesymbol zu matchen ist,) auf den Stack gepackt. Soll das automatisch vom Stack genommene Symbol auf dem Stack bleiben, muss es wieder gepusht werden.

3.3.2.6. Beispiel

Ein PDA für $L = \{ww^R \mid w \in \{a, b\}^*\}$:



3.3.2.7. Noch ein Beispiel

Hier entsteht ein Tafelbild.

3.3.2.8. Deterministische PDAs

Def. Ein PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, \bot, F)$ ist *deterministisch* :$\Leftrightarrow$

- $\delta(q, a, X)$ hat höchstens ein Element für jedes $q \in Q, a \in \Sigma$ oder $(a = \epsilon$ und $X \in \Gamma)$.
- Wenn $\delta(q, a, x)$ nicht leer ist für ein $a \in \Sigma$, dann muss $\delta(q, \epsilon, x)$ leer sein.

Deterministische PDAs werden auch *DPDAs* genannt.

3.3.2.9. Der kleine Unterschied

Satz: Die von DPDAs akzeptierten Sprachen sind eine echte Teilmenge der von PDAs akzeptierten Sprachen.

Die regulären Sprachen sind eine echte Teilmenge der von DPDAs akzeptierten Sprachen.

3.3.3. Kontextfreie Grammatiken und Sprachen

3.3.3.1. Kontextfreie Grammatiken

Def. Eine *kontextfreie (cf-)* Grammatik ist ein 4-Tupel $G = (N, T, P, S)$ mit N, T, S wie in (formalen) Grammatiken und P ist eine endliche Menge von Produktionen der Form:

$X \to Y$ mit $X \in N, Y \in (N \cup T)^*$.

$\Rightarrow, \overset{*}{\Rightarrow}$ sind definiert wie bei regulären Sprachen. Bei cf-Grammatiken nennt man die Ableitungsbäume oft *Parse trees*.

3.3.3.2. Beispiel

$S \to a \mid S + S \mid S * S$
Hier entsteht ein Tafelbild.
Ableitungsbäume für $a + a * a$:

3.3.3.3. Was ist hier los?

Hier entsteht ein Tafelbild.

3.3.3.4. Nicht jede kontextfreie Grammatik ist eindeutig

Def.: Gibt es in einer von einer kontextfreien Grammatik erzeugten Sprache ein Wort, für das mehr als ein Ableitungsbaum existiert, so heißt diese Grammatik *mehrdeutig*. Anderenfalls heißt sie *eindeutig*.

Satz: Es ist nicht entscheidbar, ob eine gegebene kontextfreie Grammatik eindeutig ist.

Satz: Es gibt kontextfreie Sprachen, für die keine eindeutige Grammatik existiert.

3.3.3.5. Kontextfreie Grammatiken und PDAs

Satz: Die kontextfreien Sprachen und die Sprachen, die von PDAs akzeptiert werden, sind dieselbe Sprachklasse.

Satz: Eine von einem DPDA akzeptierte Sprache hat eine eindeutige Grammatik.

Vorgehensweise im Compilerbau: Eine (cf) Grammatik für die gewünschte Sprache definieren und schauen, ob sich daraus ein DPDA generieren lässt (automatisch).

3.3.4. Syntaxanalyse

3.3.4.1. Syntax

Wir verstehen unter Syntax eine Menge von Regeln, die die Struktur von Daten (z. B. Programmen) bestimmen.

3.3.4.2. Ziele der Syntaxanalyse

- Bestimmung der syntaktischen Struktur eines Programms
- aussagekräftige Fehlermeldungen, wenn ein Eingabeprogramm syntaktisch nicht korrekt ist
- Erstellung des AST (abstrakter Syntaxbaum): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. Semikolons, manche Schlüsselwörter)
- die Symboltabelle(n) mit Informationen bzgl. Bezeichner (Variable, Funktionen und Methoden, Klassen, benutzerdefinierte Typen, Parameter, …), aber auch die Gültigkeitsbereiche

3.3.4.3. Was brauchen wir für die Syntaxanalyse von Programmen?

- einen Grammatiktypen, aus dem sich manuell oder automatisiert ein Programm zur deterministischen Syntaxanalyse (= Parser) erstellen lässt
- einen Algorithmus zum Parsen von Programmen mit Hilfe einer solchen Grammatik

3.3.5. Wrap-Up

3.3.5.1. Das sollen Sie mitnehmen

- Die Struktur von gängigen Programmiersprachen lässt sich nicht mit regulären Ausdrücken beschreiben und damit nicht mit DFAs akzeptieren.
- Das Automatenmodell der DFAs wird um einen endlosen Stack erweitert, das ergibt PDAs.
- Kontextfreie Grammatiken (CFGs) erweitern die regulären Grammatiken.
- PDAs akzeptieren kontextfreie Sprachen.
- Deterministisch parsbare Sprachen haben eine eindeutige kontextfreie Grammatik, aber nicht für jede eindeutige kontextfreie Grammatik lässt sich ein deterministischer PDA finden.
- Es ist nicht entscheidbar, ob eine gegebene kontextfreie Grammatik eindeutig ist.
- Syntaxanalyse wird mit (möglichst deterministisch) kontextfreien Grammatiken durchgeführt.
- In der Praxis werden aus kontextfreien Grammatiken Parser automatisch generiert.

Zum Nachlesen

- Aho u. a. (2023)
- Hopcroft u. a. (2003)

Lernziele

- k1: Ich kenne PDAs
- k1: Ich kenne deterministische PDAs
- k1: Ich kenne kontextfreie Grammatiken
- k1: Ich kenne deterministisch kontextfreie Grammatiken
- k2: Ich kann den Zusammenhang zwischen PDAs und kontextfreien Grammatiken an einem Beispiel erklären

- k2: Ich kann PDAs entwickeln
- k2: Ich kann kontextfreie Grammatiken entwickeln

3.4. LL-Parser

Weitere Unterlagen

- Annotierte Folien: LL-Parser

3.4.1. Wiederholung

3.4.1.1. PDAs und kontextfreie Grammatiken

- Warum reichen uns DFAs nicht zum Matchen von Eingabezeichen?
- Wie könnnen wir sie minimal erweitern?
- Sind PDAs deterministisch?
- Wie sind kontextfreie Grammatiken definiert?
- Sind kontextfreie Grammatiken eindeutig?

3.4.2. Motivation

3.4.2.1. Was brauchen wir für die Syntaxanalyse von Programmen?

- einen Grammatiktypen, aus dem sich manuell oder automatisiert ein Programm zur deterministischen Syntaxanalyse (=Parser) erstellen lässt
- einen Algorithmus zum Parsen von Programmen mit Hilfe einer solchen Grammatik

3.4.2.2. Themen für heute

- Automatische Generierung von Top-Down-Parsern aus LL-Grammatiken

3.4.3. Syntaxanalyse

3.4.3.1. Syntax

Wir verstehen unter Syntax eine Menge von Regeln, die die Struktur von Daten (z. B. Programmen) bestimmen.

Syntaxanalyse ist die Bestimmung, ob Eingabedaten einer vorgegebenen Syntax entsprechen.

Diese vorgegebene Syntax wird im Compilerbau mit einer kontextfreien Grammatik beschrieben und mit einem sogenannten **Parser** analysiert.

Wir beshäftigen uns heute mit LL-Parsing, mit dem man eine Teilmenge der eindeutigen kontextfreien Grammatiken syntaktich analysieren kann.

Der Ableitungsbaumwird von oben nach unten aufgebaut.

3.4.3.2. Ziele der Syntaxanalyse

- aussagekräftige Fehlermeldungen, wenn ein Eingabeprogramm syntaktisch nicht korrekt ist
- evtl. Fehlerkorrektur
- Bestimmung der syntaktischen Struktur eines Programms
- Erstellung des AST (abstrakter Syntaxbaum): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. Semikolons, manche Schlüsselwörter)
- die Symboltablelle(n) mit Informationen bzgl. Bezeichner (Variable, Funktionen und Methoden, Klassen, benutzerdefinierte Typen, Parameter, …), aber auch die Gültigkeitsbereiche.

3.4.4. LL(k)-Grammatiken

3.4.4.1. First-Mengen

$S \to A \mid B \mid C$

Welche Produktion nehmen?

Wir brauchen die “terminalen k-Anfänge” von Ableitungen von Nichtterminalen, um eindeutig die nächste zu benutzende Produktion festzulegen. k ist dabei die Anzahl der Vorschautoken.

Def.: Wir definieren $First$ - Mengen einer Grammatik wie folgt:

- $a \in T^*, |a| \leq k : First_k(a) = \{a\}$
- $a \in T^*, |a| > k : First_k(a) = \{v \in T^* \mid a = vw, |v| = k\}$
- $\alpha \in (N \cup T)^* \backslash T^* : First_k(\alpha) = \{v \in T^* \mid \alpha \overset{*}{\Rightarrow} w, \text{mit } w \in T^*, First_k(w) = \{v\}\}$

3.4.4.2. Linksableitungen

Def.: Bei einer kontextfreien Grammatik G ist die *Linksableitung* von $\alpha \in (N \cup T)^*$ die Ableitung, die man erhält, wenn in jedem Schritt das am weitesten links stehende Nichtterminal in α abgeleitet wird.

Man schreibt $\alpha \overset{*}{\Rightarrow}_l \beta$.

3.4.4.3. LL(k)-Grammatiken

Def.: Eine kontextfreie Grammatik *G = (N, T, P, S)* ist genau dann eine *LL(k)*-Grammatik, wenn für alle Linksableitungen der Form:

$S \overset{*}{\Rightarrow}_l \ wA\gamma \Rightarrow_l w\alpha\gamma \overset{*}{\Rightarrow}_l wx$

und

$S \overset{*}{\Rightarrow}_l wA\gamma \Rightarrow_l w\beta\gamma \overset{*}{\Rightarrow}_l wy$

mit $(w, x, y \in T^*, \alpha, \beta, \gamma \in (N \cup T)^*, A \in N)$ und $First_k(x) = First_k(y)$ gilt:

$\alpha = \beta$

3.4.4.4. LL(1)-Grammatiken

Hier entsteht ein Tafelbild.

3.4.4.5. LL(k)-Sprachen

Die von *LL(k)*-Grammatiken erzeugten Sprachen sind eine echte Teilmenge der deterministisch parsbaren Sprachen.

Die von *LL(k)*-Grammatiken erzeugten Sprachen sind eine echte Teilmenge der von *LL(k+1)*-Grammatiken erzeugten Sprachen.

Für eine kontextfreie Grammatik *G* ist nicht entscheidbar, ob es eine *LL(1)* - Grammatik *G'* gibt mit $L(G) = L(G')$.

In der Praxis reichen $LL(1)$ - Grammatiken oft. Hier gibt es effiziente Parsergeneratoren (hier: ANTLR), deren Eingabe eine LL(k)- (meist LL(1)-) Grammatik ist, und die als Ausgabe den Quellcode eines (effizienten) tabellengesteuerten Parsers generieren.

3.4.4.6. Algorithmus: Konstruktion einer LL-Parsertabelle

Eingabe: Eine Grammatik G = (N, T, P, S)

Ausgabe: Eine Parsertabelle *P*

```
for each production X → α
   for each a ∈ First(α)
      add X → α to P[X, a]

   if ε ∈ First(α)
      for each b ∈ Follow(α)
         add X → α to P[X, b]
         if ε ∈ First(α) and ⊥ ∈ Follow(X)
            add X → α to P[X, ⊥]
```

Abbildung 3.2.: Algorithmus zur Generierung einer LL-Parsertabelle

Hier ist $\perp$ das Endezeichen des Inputs. Statt $First_1(\alpha)$ wird oft nur $First(\alpha)$ geschrieben.

3.4.4.7. LL-Parsertabellen

Hier entsteht ein Tafelbild.

3.4.4.8. LL-Parsertabellen

Rekursive Programmierung bedeutet, dass das Laufzeitsystem einen Stack benutzt. Diesen Stack kann man auch “selbst programmieren”, d. h. einen PDA implementieren. Dabei wird ebenfalls die oben genannte Tabelle zur Bestimmung der nächsten anzuwendenden Produktion benutzt. Der Stack enthält die zu erwartenden Eingabezeichen, wenn immer eine Linksableitung gebildet wird. Diese Zeichen im Stack werden mit dem Input gematcht.

3.4.4.9. Algorithmus: Tabellengesteuertes LL-Parsen mit einem PDA

Eingabe: Eine Grammatik G = (N, T, P, S), eine Parsertabelle *P* mit “$w \perp$” als initialem Kellerinhalt

Ausgabe: Wenn $w \in L(G)$, eine Linksableitung von w, Fehler sonst

```
a = next_token()
X = top of stack  // entfernt X vom Stack

while X ≠⊥

   if X = a
      a = next_token()

   else if X ∈ T
      error

   else if P[X, a] leer
      error

   else if P[X, a] = X → Y1Y2 ... Yk
      process_production(X → Y1Y2 ... Yk)
      push(Y1Y2 ... Yk) //Y1 = top of stack

   X = top of stack
```

Abbildung 3.3.: Algorithmus zum tabellengesteuerten LL-Parsen

3.4.4.10. Ergebnisse der Syntaxanalyse

- eventuelle Syntaxfehler mit Angabe der Fehlerart und des -Ortes
- Fehlerkorrektur
- Format für die Weiterverarbeitung:
  - Ableitungsbaum oder Syntaxbaum oder Parse Tree
  - abstrakter Syntaxbaum (AST): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. ;, Klammern, manche Schlüsselwörter, . . .)

- Symboltabelle

3.4.5. Wrap-Up

3.4.5.1. Wrap-Up

- Syntaxanalyse wird mit deterministisch kontextfreien Grammatiken durchgeführt.
- Eine Teilmenge der dazu gehörigen Sprachen lässt sich top-down parsen.
- Ein effizienter LL(k)-Parser realisiert einen DPDA und kann automatisch aus einer LL(k)-Grammatik generiert werden.
- Der Parser liefert in der Regel einen abstrakten Syntaxbaum.

Zum Nachlesen

- Aho u. a. (2023)
- Hopcroft u. a. (2003)

Lernziele

- k1: Ich kenne die Top-Down-Analyse
- k1: Ich kenne LL-Parser
- k2: Ich kann den algorithmischen Ablauf von LL-Parsern an einem Beispiel erklären

4. Sprache L-Int: Integer, Addition, Subtraktion

4.0.1. Teil 1:

- Einführung Lispy (Sprachstufe L-Int)
- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- AST, Values, InterpreterLint

4.0.2. Teil 2:

- Handgeschriebener Lexer, RD-Parser

4.1. L-Int: Die Sprache Lispy

TL;DR

Lispy ist eine für diese Lehrveranstaltung frei erfundene Lisp-ähnliche Programmiersprache. Ihre Ausdrücke werden als S-Expressions in einer Klammer- und Präfixnotation geschrieben.
L-Int ist dabei die Grundstufe von Lipsy und umfasst lediglich Integer-Literale sowie Addition und Subtraktion. Ein Integer-Literal ist bereits ein minimaler Ausdruck, während eine Liste wie `(+ 1 2)` einen Operator und seine Operanden enthält.
Die Syntax legt fest, welche Ausdrücke gültig sind; die Semantik legt fest, welchen Wert ein gültiger Ausdruck ergibt.
Mehrere Operanden können bei der Addition oder Subtraktion als "syntaktischer Zucker" für verschachtelte binäre Ausdrücke behandelt werden.
Kommentare beginnen mit `;;` und reichen bis zum Ende der Zeile. L-Int bildet die Grundlage, um schrittweise Grammatik, Lexer, Parser, Syntaxbaum, AST und Interpreter zu entwickeln.

Videos

Vorlesung [YT], [HSBI]

4.1.1. Problem: Wie verstehen wir `1 + 2 * 3`?

- Eingabe: Text wie `1 + 2 * 3`
- Ziel: Ergebnis berechnen

- Frage:
  - Wie modellieren wir die Struktur solcher Ausdrücke?
  - Wie bringen wir den Rechner dazu, "richtig" zu rechnen?

Starten wir mit einem bewusst einfachen Beispiel: `1 + 2 * 3`. Das kennen alle, aber wir tun jetzt so, als wüssten wir noch nicht, wie ein Rechner das versteht. Die beiden Kernfragen sind: Wie sieht die Struktur dieses Ausdrucks aus? Und wie können wir die Bedeutung so modellieren, dass wir sie programmatisch auswerten können?

Das wird uns durch die gesamte Veranstaltung begleiten: **Vom Text (Eingabe) über Struktur (Syntax) hin zur Bedeutung (Semantik)**.

Important

4.1.1.1. Lehrkonzept

Wir werden im Laufe der Veranstaltung **inkrementell** vorgehen und mit einer minimalen Sprache L-Int (nur Integer, Addition und Subtraktion) starten. Für diese Sprache werden wir die Syntax in Form von Grammatiken festlegen, Lexer und Parser bauen und aus einer Eingabe einen Syntax-Tree (Parse-Tree) und daraus einen abstrakten Syntax-Tree (AST) erzeugen und diesen schließlich mit einem Interpreter evaluieren. (Ein konkreter Syntaxbaum beziehungsweise Parse-Tree bildet die Struktur der konkreten Grammatik ab; der AST lässt syntaktische Details weg, die für die weitere Verarbeitung nicht benötigt werden.) In den Folgewochen erweitern wir L-Int schrittweise um weitere Konzepte. Wir leiten die Java-Implementierung einer Sprachstufe von der vorherigen ab und überschreiben die Methoden, deren Verhalten sich ändert. Mit `super.xyz()` rufen wir die Implementierung der Oberklasse auf, d.h. rufen so das Verhalten der Vorgängerversion ab ohne es erneut zu implementieren.
Wir orientieren uns damit grob an dem inkrementellen Vorgehen in (Siek 2023). Konzeptionell/didaktisch lassen wir uns bei der Modellierung der Bausteine in vielen Fällen ungefähr von der Darstellung in (Nystrom 2021) inspirieren. Es werden sich also immer wieder Hinweise auf diese beiden Werke als Vertiefung finden.

4.1.2. S-Expressions als syntaktische Grundform

Ein Programm in L-Int besteht aus einem **Ausdruck** (*Expression*). Die Ausdrücke haben eine spezielle Form: Sie sind sogenannte **S-Expressions**.

Eine S-Expression ist:

- ein Atom der Form `x`, oder
- ein Ausdruck der Form `(. x y)` (mit `x` u. `y` S-Expressions, `.` eine Operation oder Funktion oder ein Keyword).

Tip

4.1.2.1. Variadische S-Expressions

Die Anzahl der S-Expressions in einem Klammer-Ausdruck ist nicht näher definiert - `x` und `y` sind nur Beispiele. Es könnten auch mehr oder weniger S-Expressions nach der Operation/Funktion/Keyword

auftauchen (vgl. nachfolgende Sprachdefinition).

Important

4.1.2.2. Syntax vs. Semantik

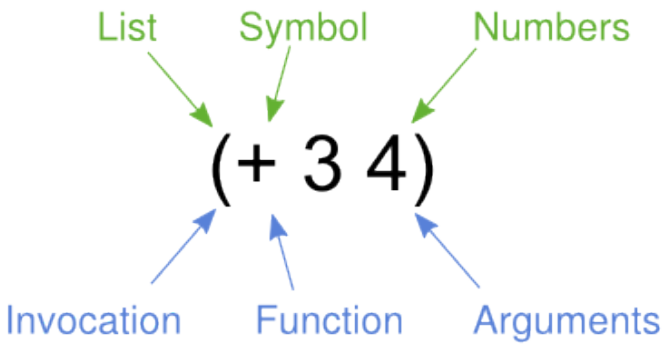
Die Syntax legt fest, welche Ausdrücke gültig sind; die Semantik legt fest, welchen Wert ein gültiger Ausdruck ergibt.
Wir werden zur Beschreibung der Syntax verschiedene Grammatiken entwickeln. Die Auswertung wird über eine semantische Analyse und einen Interpreter erfolgen.

In der *Vorlesung* werden wir für unsere aufeinander aufbauenden Sprachfamilien **S-Expressions** nutzen in Anlehnung an Clojure und Lisp. Das vereinfacht das Parsing deutlich und lässt die Konzepte im Skript und den Folien deutlicher hervortreten. Im Praktikum werden wir diese Konzepte auf einen Subdialekt von C++ anwenden und über das Semester hinweg in modernem Java einen Interpreter für dieses Mini-C++ schreiben.

Lisp ist eine alte Sprachfamilie, die aus dem KI-Umfeld stammt, und ist eine Abkürzung für "*list processing*". In vielen Lisp-Dialekten werden Programme und viele Daten in einer gemeinsamen listenartigen Form (S-Expressions) dargestellt ("Homoikonizität", "*code as data*") - einfach zu merken und gleichzeitig sehr mächtig.

Clojure ist ein moderner Lisp-Dialekt und läuft auf der Java-VM. Der Legende nach stammt der Name "Clojure" von "*closure*" (Programmierkonzept) sowie von "C" für C bzw. C#, "L" für Lisp und "J" für Java (Sprachen, bei denen Konzepte entlehnt wurden). (Quelle: Hickey, Rich (2009-01-05). "meaning and pronunciation of Clojure")

Clojure und Lisp nutzen die "*prefix notation*": `(. x y z)`



Quelle: Clojure.org: Learn Clojure

In grüner Beschriftung ist die Syntax markiert, also die Datenstruktur, die nach dem Einlesen des Codeschnipsels entsteht. Eine S-Expression ist also einfach eine Liste mit einem Symbol und hier im Beispiel zwei Zahlen.

In blauer Schrift ist die Semantik markiert, also das, was die Clojure Runtime dann ausführt. Dabei wird die Liste als Funktionsaufruf interpretiert, das Symbol zu einer Funktion aufgelöst und die beiden Zahlen werden als Argumente übergeben.

Die Verwendung von S-Expressions spielt direkt eine Rolle, warum Lisp und die modernen Varianten wie Clojure und Racket so beliebt sind.

S-Expressions werden in Clojure und Lisp oft auch "*form*" genannt.

Tip

Wir ignorieren die Aspekte "Liste" und "Funktion" vorerst und stellen das für später zurück. Ein `(+ 1 2)` ist für uns einfach eine andere Schreibweise für `1 + 2`.

4.1.3. Lispy: Erste Sprachstufe L-Int

Die Sprache L-Int ist bewusst sehr eingeschränkt, damit wir schnell die Interpreter-Pipeline aufbauen können. Wir werden dann schrittweise weitere Sprachinkremente darauf aufbauen.

In L-Int haben wir:

- Nichtnegative Ganzzahlen (z.B. `0`, `42`, `1337`)
- Arithmetische Operatoren: `+`, `-`
- Klammern: `(`, `)`
- Zeilen-Kommentar: `;;`

Beispiele:

```
1                       ;; 1 (Integer)
(+ 1 2)                 ;; 1 + 2
(+ 1 2 3 4)             ;; 1 + 2 + 3 + 4
(- 10 (+ 3 5))          ;; 10 - (3 + 5)
(- (+ 1 2) (- 3 4))     ;; (1 + 2) - (3 + 4)
```

L-Int ist unsere minimalistische Basissprache für die folgenden Sitzungen. Sie enthält nur das Nötigste: Integer-Literale, Addition und Subtraktion, sowie Klammern. Das ist bewusst klein gehalten, damit wir uns auf die Architektur konzentrieren können: Grammatik, AST, Interpreter. Alles, was wir hier aufbauen, lässt sich (und wird) später systematisch erweitern.

Die einfachste Form sind Integer-Literale mit konkreten Werten.

In der listenartigen Form ist der erste Eintrag der Liste immer eine Operation (oder ein Funktionsname), danach kommen je nach Operation/Funktion (die Arität muss passen!) entsprechende Einträge, die als Parameter für die Operation oder Funktion zu verstehen sind.

Der Ausdruck `(+ 1 1)` ist eine S-Expression, gebildet aus den atomaren Ausdrücken + und 1. Bei der Evaluierung wird das +-Symbol zu einer Additionsfunktion evaluiert, und die 1 steht für sich, also für eine Integerzahl mit dem Wert 1. Bei der Evaluierung der Liste wird die Additionsfunktion mit den beiden Integerzahlen als Argumente aufrufen.

Über `;;` wird ein Kommentar eingeleitet, der bis zum Ende der Zeile geht.

Tip

4.1.3.1. Syntactic Sugar

Die Ausdrücke sind implizit von links nach rechts geklammert, d.h. der Ausdruck `(+ 1 2 3 4)` ist *syntactic sugar* für `(+ (+ (+ 1 2)3)4)`.

Tip

4.1.3.2. Statements vs. Expressions

In vielen Sprachen unterscheidet man zwischen **Statements** und **Expressions** (Anweisungen und Ausdrücke). Dabei haben Statements üblicherweise keinen (Rückgabe-) Wert und verändern den Zustand des laufenden Programms (*state*), während Expressions immer einen Wert ergeben.
In unserem L-Int ist zunächst alles eine Expression, d.h. Ausdrücke ergeben bei der Auswertung immer einen Wert. In späteren Sprachstufen werden wir komplexere Ausdrücke und auch Statements über die Listenform bilden.

4.1.4. Wrap-up

- L-Int besteht aus S-Expressions
- Formen:
  - Integer-Literal als Atom
  - S-Expressions mit + und − als Operatoren

Zum Nachlesen

Einen schönen Überblick über das, was uns in diesem Semester erwartet, gibt Nystrom (2021) in Kapitel 2 “A Map of the Territory”.

Lernziele

- k2: Ich kann den Unterschied zwischen Syntax und Semantik an einem L-Int-Ausdruck erklären.
- k2: Ich kann erklären, wie Integer-Literale und Operatorlisten als S-Expressions dargestellt werden.
- k3: Ich kann einfache L-Int-Ausdrücke entsprechend der festgelegten Auswertungsregeln auswerten.
- k3: Ich kann einen mehrstelligen Additionsausdruck in eine äquivalente Folge binärer Ausdrücke umformen.

Challenges

Form und Bedeutung
Betrachten Sie den Ausdruck:

```
1  (- 10 (+ 3 5))
```

1. Beschreiben Sie seine Struktur in eigenen Worten: Welche Operation steht außen, und welche Operanden hat sie?

2. Berechnen Sie den Wert.
3. Erklären Sie, welche Ihrer Aussagen zur Syntax und welche zur Semantik gehören.

Mehrstellige Subtraktion als syntaktischer Zucker
Der Ausdruck

```
(- 20 5 3)
```

wird laut Skript von links nach rechts geklammert.

1. Schreiben Sie ihn als verschachtelten Ausdruck mit jeweils zwei Operanden um.
2. Berechnen Sie den Wert.
3. Berechnen Sie zum Vergleich auch die rechts geklammerte Variante. Sind die Ergebnisse gleich?

Negative Zahl: Literal oder Ergebnis
Vergleichen Sie diese Schreibweisen:

```
-3
(- 0 3)
(+ 0 (- 5 8))
```

Welche davon sind nach der Beschreibung von L-Int als Eingabeausdrücke vorgesehen? Welche Werte ergeben die Ausdrücke, die Sie auswerten können? Erklären Sie den Unterschied zwischen einem **Integer-Literal** und einem **Auswertungsergebnis**.

4.2. L-Int: Konkrete und abstrakte Syntax

TL;DR

Die **konkrete Grammatik** beschreibt, welche Texte gültige L-Int-Programme sind und welche Struktur ihr **Parse-Tree** hat. Ein L-Int-Ausdruck ist entweder ein Integer-Literal oder eine Addition beziehungsweise Subtraktion mit mindestens zwei Operanden.
Die konkrete Syntax enthält Details wie Klammern und Hilfsregeln, die für die spätere Verarbeitung teilweise nicht mehr benötigt werden. Die **abstrakte Grammatik** beschreibt stattdessen die wesentlichen Ausdrucksformen als `IntLiteral`, `AddExpr` und `SubExpr`. Daraus entsteht ein **AST** (*abstract syntax tree*), in dem beispielsweise Klammern nicht als eigene Knoten vorkommen.
Die AST-Knoten lassen sich in Java durch ein gemeinsames `Expr`-Interface und Records modellieren. Da der AST Addition und Subtraktion binär darstellt, muss festgelegt werden, wie Ausdrücke mit mehr als zwei Operanden in binäre Knoten übersetzt werden - eine einfache Form von “Desugaring”.

Videos

Vorlesung [YT], [HSBI]

4.2.1. Erinnerung Lispy in der Sprachstufe L-Int

```
1  1                       ;; 1 (Integer)
2  (+ 1 2)                 ;; 1 + 2
3  (+ 1 2 3 4)             ;; 1 + 2 + 3 + 4
4  (- 10 (+ 3 5))          ;; 10 - (3 + 5)
5  (- (+ 1 2) (- 3 4))     ;; (1 + 2) - (3 + 4)
```

Lispy wird aus S-Expressions aufgebaut, die die folgende Form haben:

- ein Atom der Form `x`, oder
- ein Ausdruck der Form `(. x y)` (mit `x` u. `y` S-Expressions, `.` eine Operation oder Funktion oder ein Keyword; dabei dürfen beliebig viele S-Expressions als Operanden vorkommen - hier im Beispiel sind nur zwei angedeutet).

In der Sprachstufe L-Int haben wir als Atom nur Integer-Literale, und als Operatoren kennen wir nur + und −.

Frage: Wie können wir das formal definieren?

4.2.2. Konkrete Grammatik für L-Int

```
1  eform ::= '(' '+' eform eform+ ')'
2          | '(' '-' eform eform+ ')'
3          | INT
4  INT   ::= [0-9]+
5
6  lint  ::= eform EOF
```

Diese konkrete Grammatik beschreibt die Struktur (Syntax) der Zeichenketten, die zulässige Ausdrücke in L-Int sind.

Ein Programm in L-Int besteht aus einer S-Expression `eform` (“e” wie “Expression” - diese `eform` sind Ausdrücke und evaluieren zu einem Wert), auf die ein `EOF`-Token folgt. Weitere Eingaben sind nicht zulässig.

Ein `eform` besteht aus drei Alternativen:

- eine S-Expression mit einem `'+'` als Operator (Addition),
- eine S-Expression mit einem `'-'` als Operator (Subtraktion),
- ein Integer-Literal.

Das Integer-Literal ist ein Terminal-Symbol (Token) und wird über den regulären Ausdruck `[0-9]+` beschrieben. Es besteht aus mindestens einer oder beliebig vielen Ziffern. In der Definition von S-Expressions ist das ein “Atom”.

> **Tip**
>
> *Anmerkung*: Man kann hier zwischen der Tokenart (`INT`) und dem Lexem (konkrete Ziffernfolge, die im Lexer durch den regulären Ausdruck gematcht wird) unterscheiden. Das gilt auch für die “impliziten Token”, die in der Grammatik direkt als Literal dargestellt werden, beispielsweise `'('`: Dies ist die Tokenart `LPAREN` mit dem Lexem “(”.

Die beiden Ausdrücke für Addition und Subtraktion haben die Klammern und den Operator als Terminal-Symbol (Token), und bestehen dann aus mindestens zwei weiteren `eform`-Ausdrücken - das sind die Operanden. Über

das `eform`+ erlauben wir auch mehr als zwei `eform` als Operanden, das + ist hierbei ein Wiederholungsoperator der Grammatik (ein- oder mehrfach). Ein * würde "kein- oder mehrfach" bedeuten, das ? entsprechend "ein- oder keinmal". Bitte nicht mit dem `'+'` verwechseln: Dies ist ein "implizites" Token, also ein Token, dessen Lexem hier statt der Tokenart verwendet wird.

Das nennt sich "konkrete Grammatik", weil wir hier ausdrücken, was gültige konkrete Programme sind.

Tip

4.2.2.1. Schreibweise konkrete Grammatik

Wir orientieren uns an einer Mischung aus ANTLR-Grammatik und der Notation in (Siek 2023).

- klein geschrieben: Nichtterminale (Produktionsregeln) der konkreten Syntax, etwa `eform`
- GROSS geschrieben: Terminal-Symbole (Token) des Lexers, etwa `INT` oder `EOF`
- in einfachen 'Anführungszeichen': unbenannte Terminale als konkrete Zeichenketten (implizite Token), etwa `'+'` oder `'('` - für diese gibt es ebenfalls Token im Lexer (`PLUS`, `LPAREN`)
- Wiederholungsoperatoren: beziehen sich auf den Ausdruck davor (ggf. klammern!)
  - ?: "ein- oder keinmal"
  - *: "kein- oder mehrfach"
  - +: "ein- oder mehrfach"

Tip

4.2.2.2. Implizite Token

Wir verwenden in der Grammatik an einigen Stellen aus Gründen der Lesbarkeit "implizite" Token. Das bedeutet, dass wir statt der Tokenart das dazugehörige Lexem angeben. Das kann man immer dann machen, wenn das Lexem fest ist: Für das Token `LPAREN` gibt es nur ein Lexem, nämlich "(".
Beispiel: Wir haben die Form `'('+'eform eform+ ')'`. Statt der impliziten Token könnten wir hier auch wie üblich die Tokenart nutzen: `LPAREN PLUS eform eform+ RPAREN`.
Bitte die impliziten Token nicht mit den Wiederholungsoperatoren der Grammatik verwechseln.

Important

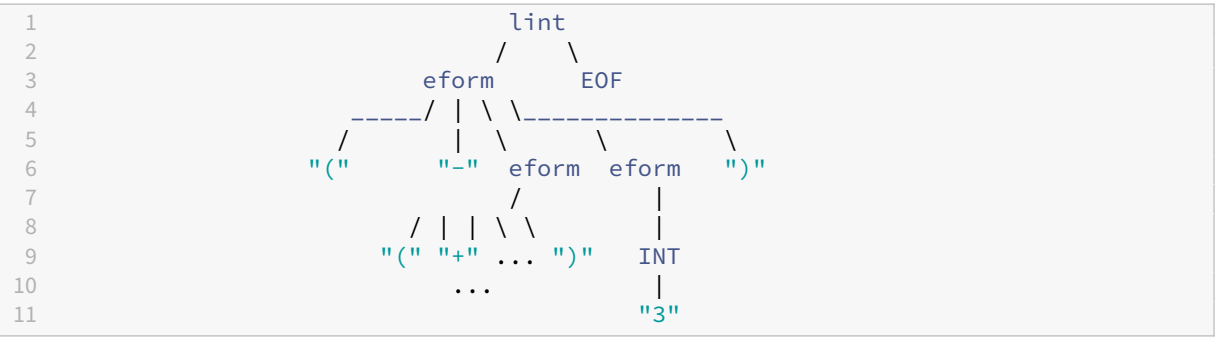
Die Grammatik zeigt die Token und wie der Lexer `INT`, `'+'`, `'('` und `')'` erkennt: Für `INT` wurde ein regulärer Ausdruck definiert, der zum Erkennen von `INT` angewendet wird. Die "impliziten" Token wie `'('` werden direkt mit der angegebenen Zeichenfolge (hier "(") erkannt.
Whitespace (Leerzeichen, Tabs, Zeilenumbrüche) werden vom Lexer ignoriert. Kommentare `;; ...` (bis zum Zeilenende) werden ebenfalls vom Lexer ignoriert. Dies muss man für Tools wie ANTLR explizit formulieren; in der Darstellung der konkreten Syntax in den Vorlesungsunterlagen sparen wir uns dies zugunsten einer besseren Lesbarkeit.
`EOF` ist kein Zeichen in der Eingabe, es markiert lediglich das Ende der Eingabe und wird vom Lexer als End-of-Input-Token produziert.

4.2.3. Beispiel konkrete Syntax

Schauen wir uns an, wie der Ausdruck `(- (+ 1 2)3)` als **Parse-Tree** aussieht.

Text: `(- (+ 1 2)3)`

Parse-Tree:



Wir starten mit der Startregel `lint`. Diese fordert ein `eform` und ein `EOF`, beide bilden die zweite Ebene. Das "(" gefolgt von einem "-" passt zur zweiten Alternative in der Regel `eform`, die beiden Token werden als Kinder des `eform`-Knotens eingetragen. Dann erwarten wir eine `eform eform`+ und die schließende rechte Klammer ")". Der Teilausdruck "(+ 1 2)" passt auf `eform` (hier nur teilweise als Skizze dargestellt). Die "3" passt zu `eform`+, da hier mindestens eine oder beliebig viele `eform`-Ausdrücke erwartet werden und "3" zu der Alternative `INT` in `eform` passt.

> **Tip**
>
> **Beobachtung**: Der Parse-Tree spiegelt die Strukturen der Grammatik wider und wird selbst bei unserer einfachen L-Int-Grammatik und der kleinen Eingabe bereits relativ komplex.

4.2.4. Konkrete Grammatik - Eigenschaften

- Die konkrete Grammatik legt fest:
  - Welche Eingabe-Strings **syntaktisch** gültige Programme sind
  - Struktur des **Parse-Trees**

- Aber:
  - Parse-Tree enthält **"technische" Details**:
    - Klammern, `EOF`, ...
    - Hilfs-Nonterminale (`eform`)
  - Nicht optimal als interne Repräsentation für die Weiterverarbeitung

Die konkrete Grammatik ist gut, um zu entscheiden: Gehört ein String zur Sprache, ja oder nein? Und sie ermöglicht es uns, einen Recursive-Descent-Parser aus der Grammatik abzuleiten. Sie liefert für eine Eingabe eines

syntaktisch korrekt formulierten Programms einen dazu passenden Parse-Tree, der strukturell entsprechend den Regeln in der Grammatik aufgebaut ist.

Aber sie enthält viele technische Hilfskonstrukte, z.B. `eform` oder explizite Klammern, die wir im weiteren Verlauf für die Semantik nicht zwingend brauchen. Für die weitere Verarbeitung ist es oft besser, eine aufgeräumtere und abstraktere Sicht auf das eingegebene Programm zu haben.

4.2.5. Konkrete vs. abstrakte Grammatik

- **Konkrete Grammatik**:
  - Wie der Code "auf dem Papier" (in der Eingabe) aussieht
  - Enthält Syntaxdetails (Klammern, Keywords, Vorrangregeln)

- **Abstrakte Grammatik**:
  - Wie wir den Code **im Compiler** modellieren
  - Fokus auf Bedeutungsträger (z.B. `AddExpr`, `SubExpr`, `IntLiteral`)

Wir orientieren uns an der Darstellung in (Siek 2023) und unterscheiden sehr bewusst zwischen **konkreter** und **abstrakter Grammatik**.

Die konkrete Grammatik beschreibt das, was der Parser sieht: der tatsächliche Text, inklusive aller Klammern und syntaktischen Konstrukte.

Die abstrakte Grammatik hingegen ist das Modell, das wir intern verwenden, um die Semantik zu definieren. Sie reduziert die Sprache auf die wesentlichen Knoten, die Bedeutung tragen.

4.2.6. Abstrakte Grammatik für L-Int

```
1  Expr = IntLiteral(Int)
2       | AddExpr(Expr, Expr)
3       | SubExpr(Expr, Expr)
4
5  Lint = Expr
```

- Keine `eform`-Regel mehr
- Klammern, `EOF`, … werden **nicht** als Knoten repräsentiert

Hier ist die abstrakte Grammatik von L-Int. Sie ist sehr kompakt: Ein Ausdruck ist entweder eine Ganzzahl, eine Addition oder eine Subtraktion. Wir geben dabei auch die Typen der Parameter vor: Die Argumente von `AddExpr` und `SubExpr` sind ihrerseits wieder `Expr`, und das `IntLiteral` wird über einem Integerwert gebildet. Klammern kommen hier nicht mehr vor - sie beeinflussen nur, wie wir beim Parsen den Baum bauen, nicht den Typ der Knoten. Diese abstrakte Grammatik bildet direkt ab, welche Bestandteile der Struktur des Programms wir für die semantische Auswertung unterscheiden wollen.

Mit der abstrakten Grammatik beschreiben wir den *abstract syntax tree* (**AST**).

Tip

4.2.6.1. Schreibweise abstrakte Grammatik

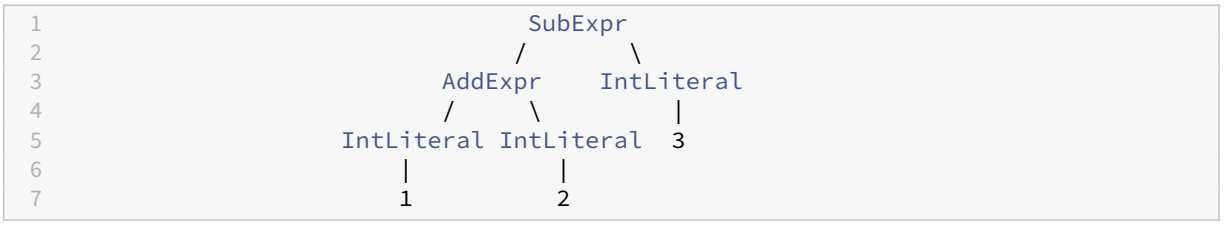
Die abstrakte Grammatik entspricht der Definition von algebraischen Datentypen (**ADT**). Da wir das in die Implementierungssprache Java übertragen, nutzen wir hier die Konventionen aus Java und schreiben alle Datentypen (spätere Interfaces und Records/Klassen) in UpperCamelCase.
Mit `Int` ist ein Integer-Datentyp gemeint, d.h. wir könnten das in Java entweder als **`int`** oder als `Integer` schreiben.
Wir werden später Listen benötigen. Diese werden mit `[]` ausgedrückt, d.h. `[Int]` würde eine Liste mit Integer-Werten bedeuten.

4.2.7. Beispiel abstrakte Syntax (AST)

Schauen wir uns nun an, wie der Ausdruck `(- (+ 1 2)3)` als **AST** aussieht.

Text: `(- (+ 1 2)3)`

AST:



Wir starten mit einer Subtraktion (`SubExpr`). Der linke Operand ist eine Addition (`AddExpr`). Alle Integer werden über `IntLiteral` repräsentiert.

Important

4.2.7.1. Desugaring

Wir haben für den AST nur *binäre* Knoten modelliert, während der Parse-Tree variadisch ist und mehr als zwei Operanden beispielsweise in der Addition erlaubt. Dies muss an einer geeigneten Stelle entsprechend umgeformt werden, beispielsweise noch direkt im Parser oder in einem nachgelagerten Lauf ("*Pass*") über den AST.
Beispiel: Wenn wir den Ausdruck `(+ 1 2 3)` betrachten, würde der Parse-Tree hier einen `eform`-Knoten mit drei Operanden beinhalten. Dies können wir nicht direkt im AST repräsentieren. Dazu müssen wir beim Erzeugen des AST den Ausdruck geeignet klammern (typisch wäre hier eine Klammerung von links nach rechts): `(+ (+ 1 2)3)` und dazu den passenden AST erzeugen:

```
AddExpr(
    AddExpr(
        IntLiteral(1),
        IntLiteral(2)
    ),
    IntLiteral(3)
```

```
)
```

`(+ 1 2 3)` ist also nur "**syntaktischer Zucker**" für `(+ (+ 1 2)3)`. Die gezeigte Vereinfachung ist eine besonders einfache Form der Vereinfachung/Umformung (auch "**Desugaring**" genannt).

4.2.8. AST in Java: Definition

Die abstrakte Grammatik übersetzen wir jetzt direkt in Java-Klassen. Wir modellieren jedes Element der abstrakten Grammatik mit passenden Java-Typen:

```java
public interface Expr {}

public record AddExpr(Expr left, Expr right) implements Expr {}
public record SubExpr(Expr left, Expr right) implements Expr {}
public record IntLiteral(int value) implements Expr {}
```

Die Regel `Expr` in der abstrakten Grammatik kann aus verschiedenen Alternativen bestehen. Wir modellieren `Expr` als Interface, von dem die Alternativen ableiten. Jede dieser Alternativen *ist* eine `Expr`.

Für die Alternativen brauchen wir Datentypen, die ihrerseits Felder speichern können: `AddExpr(Expr, Expr)` muss zwei `Expr` als Kind-Ausdrücke speichern. Jede dieser Formen wird als `record` definiert, was uns Konstruktor, Zugriffsmethoden und `toString` automatisch mitliefert. So bekommen wir eine sehr knappe, typsichere AST-Repräsentation, die gut mit Pattern Matching zusammenspielt. (Wir könnten hier auch normale Klassen nutzen, müssten aber den ganzen Boilerplate-Code selbst schreiben.)

Diese Struktur bildet unseren AST in L-Int und wird in späteren Sprachstufen erweitert.

Die Startregel `Lint = Expr` soll vor allem ausdrücken, dass alle L-Int-Programme aus *einem* `Expr`-Ausdruck bestehen. Diese wird deshalb nicht in den AST-Typen umgesetzt, sondern findet später im Parser und Interpreter Beachtung.

Tip

4.2.8.1. Sealed Interfaces

Normalerweise würde man das gemeinsame Interface als `sealed` Interface modellieren:

```java
public sealed interface Expr permits AddExpr, SubExpr, IntLiteral {}

public record AddExpr(Expr left, Expr right) implements Expr {}
public record SubExpr(Expr left, Expr right) implements Expr {}
public record IntLiteral(int value) implements Expr {}
```

Das hat den Vorteil, dass wir später in den **`switch`**-Expressions keinen **`default`**-Zweig brauchen und der Compiler automatisch darauf achtet, dass wir immer alle Fälle abgedeckt haben.
Da wir unsere Sprachstufen aber inkrementell aufeinander aufbauen, werden wir im Laufe der Zeit immer weitere AST-Knoten hinzufügen. Wenn unser gemeinsames Interface `Expr` nun `sealed` wäre, würden wir im alten Code entsprechende Fehler bekommen, weil wir dort ja noch nicht die neuen Fälle kennen (und auch nicht kennen wollen). D.h. wir müssten dann doch wieder mit **`default`** arbeiten - und damit ist `sealed` nur noch halb so nützlich (schützt nur noch vor unbeabsichtigten Ableitungen).

4.2.9. AST in Java: Beispielkonstruktion

Text: `(- (+ 1 2)3)`

AST:

```
                                SubExpr
                               /       \
                          AddExpr    IntLiteral
                          /     \         |
                  IntLiteral IntLiteral  3
                      |          |
                      1          2
```

AST direkt in Java:

```java
Expr e = new SubExpr(
    new AddExpr(
        new IntLiteral(1),
        new IntLiteral(2)
    ),
    new IntLiteral(3)
);
```

Hier konstruieren wir den AST für `(- (+ 1 2)3)` von Hand in Java. Das ist wichtig, um die Struktur zu verinnerlichen. Später wird der Parser genau solche Bäume für uns erzeugen. Wir können hier gut sehen, wie der abstrakte Syntaxbaum eins zu eins im Code auftaucht. Um die Tests beispielsweise für den Interpreter vom Parser zu entkoppeln, dienen uns genau solche manuell erstellten AST als direkte Eingabe.

4.2.10. Wrap-up

- Zwei Sichtweisen auf L-Int:
  - Konkrete Grammatik: Syntax der Eingabe-Strings
  - Abstrakte Grammatik: AST-Struktur
- Konkrete Grammatik:
  - Regeln und Terminale
  - Akzeptiert eine Eingabe
  - Erzeugt dabei einen Parse-Tree
  - Erlaubt das Ableiten eines Parsers
- Abstrakte Grammatik:
  - Modellierung als algebraische Datentypen
  - Umsetzung in Java: Interfaces und Klassen/Records

Zum Nachlesen

Zum Thema konkrete vs. abstrakte Syntax findet sich eine schöne Darstellung in (Siek 2023): Kapitel 1 “Preliminaries”. In (Nystrom 2021) kann man zum Thema Grammatik und AST in Kapitel 5 “Representing Code” nachlesen.

Lernziele

- k2: Ich kann den Unterschied zwischen konkreter und abstrakter Syntax erklären.
- k2: Ich kann Parse-Tree und AST unterscheiden und erläutern, welche Syntaxdetails im AST weggelassen werden.
- k3: Ich kann für einen gegebenen L-Int-Ausdruck den passenden Parse-Tree beziehungsweise AST skizzieren.
- k3: Ich kann einen L-Int-AST mit den passenden Java-Records konstruieren.

Challenges

Was ist ein gültiger L-Int-Ausdruck?

Ordnen Sie die folgenden Eingaben in zwei Gruppen: nach der konkreten Grammatik gültig und nicht gültig. Begründen Sie Ihre Entscheidung jeweils mit Bezug auf die Grammatik.

```
(+ 1 2)
(+ 1)
(- 1 2 3)
(+ 1 (- 2 3))
(+ 1 2) 3
```

Zusatzfrage: Welche Rolle spielt `EOF` bei der Entscheidung?

Vom Parse-Tree zum AST

Betrachten Sie den Ausdruck:

```
(- (+ 1 2 3) 4)
```

1. Skizzieren Sie den **Parse-Tree**. Achten Sie darauf, welche Klammern, Operatoren und `eform`-Knoten in der konkreten Grammatik vorkommen.
2. Erzeugen Sie den **AST** mit der in der Vorlesung gewählten links-nach-rechts-Klammerung für mehr als zwei Operanden.
3. Markieren Sie, welche Teile der konkreten Syntax im AST nicht als eigene Knoten auftauchen.

Mehrere Operanden, binärer AST

Die konkrete Grammatik erlaubt den Ausdruck:

```
(- 10 3 2)
```

Der AST kann `SubExpr` aber nur mit zwei Operanden darstellen.

1. Schreiben Sie die Klammerung auf, die sich aus der links-nach-rechts-Konvention ergibt.
2. Erstellen Sie den passenden AST mit den Java-Records.
3. Vergleichen Sie ihn mit der alternativen Klammerung von rechts nach links. Sind die beiden ASTs strukturell gleich?

4.3. L-Int: Von Ausdrücken zum Interpreter

TL;DR

Ein AST-basierter Interpreter besteht oft aus einem "Dispatch": Man traversiert mit einer `eval()`-Funktion den AST und ruft je nach Knotentyp die passende Funktion auf. Dabei werden bei Ausdrücken (*Expressions*) Werte berechnet und zurückgegeben ("eval" von "evaluieren" - "auswerten"), d.h. hier hat man einen Rückgabewert und ein entsprechendes **`return`** im **`switch`**/**`case`**, während man bei Anweisungen (*Statements*) keinen Rückgabewert hat.
Der Dispatch (Aufruf der zum Knotentyp passenden Funktion) kann wahlweise per Visitor-Pattern oder manueller Traversierung mit **`switch`**/**`case`** erfolgen.
Der Wert von Literalen ergibt sich direkt durch die Übersetzung des jeweiligen Werts in den passenden Typ der Implementierungssprache. Bei komplexeren Ausdrücken interpretiert man zunächst die Teilausdrücke durch den rekursiven Aufruf von `eval()` für die jeweiligen AST-Kindknoten und berechnet daraus das gewünschte Ergebnis.

Videos

Vorlesung [YT], [HSBI]

4.3.1. Von AST zu Werten: Idee

- Gegeben: Lexer/Parser haben einen AST aufgebaut
- Gesucht: `eval(expr)` berechnet einen **Wert**

- Beispiele:
  - `eval( IntLiteral(1))` $\rightarrow 1$
  - `eval( AddExpr(IntLiteral(1), IntLiteral(2)))` $\rightarrow 3$

- Frage:
  - Wie repräsentieren wir solche Werte sauber im Code?

Der nächste Schritt: Aus dem AST wollen wir konkrete Ergebnisse berechnen. Dazu definieren wir eine Funktion wie `eval()`, die einen `Expr`-Knoten nimmt und einen Wert zurückgibt. Bei L-Int sind das Integer-Werte, aber wir wollen die Architektur so wählen, dass wir später problemlos weitere Typen hinzufügen können. Daher abstrahieren wir Werte ebenfalls in einer kleinen Hierarchie.

4.3.2. Value-Hierarchie: Motivation

Erinnerung: Unsere konkrete Syntax für L-Int sah so aus:

```
1  Expr = IntLiteral(Int) | AddExpr(Expr, Expr) | SubExpr(Expr, Expr)
2  Lint = Expr
```

- L-Int: Nur ganze Zahlen
- Später:
  - Booleans
  - Funktionen
  - Objekte

→ `Value` als gemeinsamer Basistyp für Laufzeitwerte

Aktuell könnten wir streng genommen einfach **`int`** als Rückgabetyp verwenden. Aber wir planen ja schon die nächsten Sprachstufen: Booleans, Funktionen, Objekte, Klassen. Deshalb lohnt es sich, von Anfang an eine `Value`-Abstraktion einzuführen. So müssen wir den Interpreter später nicht grundlegend umbauen, sondern können die Value-Hierarchie einfach erweitern.

4.3.3. Value-Hierarchie in Java

```java
public interface Value {

    static int asInt(Value v) {
        if (v instanceof IntVal(int val)) return val;
        throw new RuntimeException("number expected");
    }
}


public record IntVal(int v) implements Value {}
```

- Klar getrennt von `Expr` (Syntax vs. Laufzeitwerte)

Wir müssen die Typen der Zielsprache (hier Lispy) auf unsere Implementierungssprache mappen (hier Java). D.h. die in der Zielsprache verwendeten (primitiven) Typen müssen auf passende Typen der Sprache, in der der Interpreter selbst implementiert ist, abgebildet werden.

`Value` wird unser Basistyp für Laufzeitwerte, von dem die verschiedenen Typen für Werte ableiten werden. Aktuell betrachten wir nur Integer, die wir über den Subtyp `IntVal` modellieren. Später werden weitere Typen hinzukommen.

> **Important**
>
> **Wichtig**: `Expr` beschreibt die abstrakte Syntax, also die Struktur des Programms, und `Value` beschreibt die Ergebnisse der Auswertung zur Laufzeit. Diese Trennung hilft uns später auch bei Typprüfung und Fehlerbehandlung.

4.3.4. Tree-Walking-Interpreter: Grundidee

- Funktion: `Value eval(Expr expr)`

- Idee:
  - Rekursiv über den AST laufen
  - Pro Knotentyp die passende Operation ausführen:
    * Literal (Integer): Als Integer-Wert zurückgeben
    * Ausdruck (Addition, Subtraktion): linke und rechte Seite zu Integer-Werten evaluieren, die Operation ausführen, Ergebnis als Integer-Wert zurückgeben

Der Tree-Walking-Interpreter ist im Kern eine rekursive Funktion über dem AST. Für jeden Knotentyp - Literal, Addition, Subtraktion - definieren wir, wie er auszuwerten ist. Das ist eine direkte Implementierung der operationalen Semantik unserer Sprache: Wir sagen dem Rechner Schritt für Schritt, wie die Auswertung funktioniert.

4.3.5. Interpreter für L-Int-Expressions in Java (Pattern Matching)

```java
public class InterpreterLint {

    public Value eval(Expr expr) {
        return switch (expr) {
            case IntLiteral(var v) -> new IntVal(v);

            case AddExpr(var l, var r) -> {
                int a = Value.asInt(eval(l));
                int b = Value.asInt(eval(r));
                yield new IntVal(a + b);
            }

            // TODO: SubExpr

            default -> throw new RuntimeException("unexpected expression: " +
                expr);
        };
    }
}
```

Hier sehen wir die (teilweise) Implementierung unseres L-Int-Interpreters.

Die `eval()`-Methode bildet das Kernstück des (AST-traversierenden) Interpreters für Ausdrücke. Hier wird passend zum aktuellen AST-Knoten die passende Methode des Interpreters aufgerufen.

Wir nutzen Pattern Matching in einer **`switch`**-Expression über dem AST-Typ `Expr`. Wir könnten hier auch das Visitor-Pattern nutzen - wichtig ist nur, dass der Baum rekursiv traversiert wird und für jeden Knoten die richtige Aktion durchgeführt wird.

4.3.5.1. Literale auswerten

Das ist der einfachste Teil … Die primitiven Typen der Zielsprache Lispy müssen als Datentyp der Interpreter-Programmiersprache Java ausgewertet werden.

In der abstrakten Grammatik hatten wir definiert:

```
Expr  = IntLiteral(Int)
```

Im AST werden also Knoten vom Typ `IntLiteral` angelegt, um die Integer-Literale zu repräsentieren.

Für jedes `IntLiteral` erzeugen wir nun im L-Int-Interpreter ein `IntVal`.

4.3.5.2. Ausdrücke auswerten

Für Ausdrücke hatten wir in der abstrakten Grammatik für L-Int definiert:

```
Expr  = AddExpr(Expr, Expr) | SubExpr(Expr, Expr)
```

Wir bekommen im AST also Knoten vom Typ `AddExpr` für eine Addition bzw. vom Typ `SubExpr` für eine Subtraktion. Beide Knoten haben jeweils ein linkes und ein rechtes Kind vom Typ `Expr`.

Für `AddExpr` und `SubExpr` werten wir zunächst die beiden Operanden rekursiv aus und kombinieren die Ergebnisse passend. Das entspricht genau der mathematischen Intuition von Addition und Subtraktion, nur eben explizit im Code ausgedrückt.

Wichtig: Die Subtraktion (Auswertung von `SubExpr`) fehlt hier und dient als kleine Übung.

Important

4.3.5.3. Fehlerbehandlung und Fehlermeldungen

Die meisten möglichen Fehlerzustände sind bereits durch den Lexer und Parser (und in späteren Sprachstufen während der semantischen Analyse) abgefangen worden. Falls zur Laufzeit die Auswertung der beiden Summanden keine Zahl ergibt, würde eine Java-Exception geworfen, die der Aufrufer an geeigneter Stelle fangen und behandeln muss. Der Interpreter soll sich ja nicht mit einem Stack-Trace verabschieden, sondern soll eine Fehlermeldung präsentieren und danach normal weiter machen …
Die hier gezeigte Fehlerbehandlung ist nur sehr rudimentär, um nicht zu sehr von den Kernkonzepten für unsere Veranstaltung abzulenken. In der Praxis würde man hier deutlich komplexere Fehlerbehandlung machen, beispielsweise auf die Zeile und Spalte in der Eingabe hinweisen. Gute Fehlerbehandlung ist eine Wissenschaft für sich. Vergleichen Sie einmal die Fehler, die Python ausgibt, und die Meldungen beispielsweise von Rust.

Tip

4.3.5.4. REPL: Read-Eval-Print-Loop

In den Beispielen in dieser Veranstaltung wird immer davon ausgegangen, dass ein komplettes Programm eingelesen, geparst, vorverarbeitet und dann interpretiert wird.
Für einen interaktiven Interpreter würde man in einer Schleife die Eingaben lesen, parsen und vorverarbeiten und dann interpretieren. Dabei würden jeweils der AST und die Symboltabelle und das Environment *ergänzt*, damit die neuen Eingaben auf frühere verarbeitete Eingaben zurückgreifen können. Durch die Form der Schleife “Einlesen - Verarbeiten - Auswerten” hat sich auch der Name “*Read-Eval-Loop*” bzw. “*Read-Eval-Print-Loop*” (**REPL**) eingebürgert.

Tip

4.3.5.5. Exkurs Expressions (Ausdrücke) vs. Statements (Anweisungen)

In Programmiersprachen unterscheiden wir häufig **Expressions** (*Ausdrücke*) und **Statements** (*Anweisungen*).
Expressions sind dabei syntaktische Konstrukte einer Programmiersprache, die (in einem gegebenen Kontext) zu einem Wert **evaluiert** werden können. Typische Expressions sind beispielsweise Ausdrücke wie `(+ 2 3)` oder `(foo 42)` … In manchen Sprachen sind beispielsweise auch Zuweisungen Expressions: `v = 42 + 7;` würde in C der Variablen `v` den Wert 49 zuweisen, dies ist gleichzeitig auch der Wert des gesamten Ausdrucks. Man könnte in C also Dinge formulieren wie `if (v = 42 + 7)...` (wobei das Interpretieren eines Integers in einem bool'schen Kontext nochmal ein anderes Problem ist).
Statements sind syntaktische Konstrukte in Programmiersprachen, die **ausgeführt** werden können und dabei in der Regel einen Zustand im Programm verändern, also einen Seiteneffekt haben. Die Ausführung eines Statements hat normalerweise keinen Wert an sich. Typische Beispiele sind Zuweisungen `v = 7`, Kontrollfluss `if (...)then {...} else {...}`, Schleifen `for x in foo: ...`, `switch`/`case`-Statements. (Es gibt aber auch Programmiersprachen, wo ein `if`/`then`/`else`-Konstrukt eine Expression ist, also bei der Ausführung einen Wert ergibt.) Expressions können in den meisten Programmiersprachen Teile von Statements bilden: In `v = 42 + 7` ist die gesamte Zuweisung eine Anweisung (Seiteneffekt: die Variable `v` hat danach einen anderen Zustand), und der Teil `42 + 7` ist ein Ausdruck, der ausgewertet werden kann und üblicherweise den Wert 49 ergibt (außer man beauftragt ein LLM mit der Auswertung). In C-ähnlichen Sprachen kann durch Hinzufügen eines Semikolons aus dem Ausdruck `42 +7` eine Anweisung gemacht werden …
Vergleiche auch (Nystrom 2021), Kapitel 6 "Parsing Expressions", Kapitel 7 "Evaluating Expressions" und Kapitel 8 "Statements and State", aber auch Wikipedia: Expression und Wikipedia: Statement. Wir werden uns an späterer Stelle noch darum kümmern.

4.3.6. Interpreter in Aktion

Beispiel:

```
// (- (+ 1 2) 3)
Expr expr =
    new SubExpr(
        new AddExpr(new IntLiteral(1), new IntLiteral(2)),
        new IntLiteral(3)
    );

var interp = new InterpreterLint();
Value result = interp.eval(expr);
// result: IntVal(0)
```

Wenn wir den eben gezeigten Ausdruck `(1 + 2)- 3` mit unserem Interpreter auswerten, erhalten wir erwartungsgemäß `0`, verpackt als `IntVal`.

An der Stelle müssen wir den AST noch manuell konstruieren. In Zukunft bekommt der Interpreter diesen AST aber vom Lexer und Parser, und oft werden wir noch weitere Verarbeitungsschritte haben ("Semantische Analyse"), bevor der AST schließlich zum Interpreter gelangt und ausgewertet wird.

4.3.7. Wrap-up

- Interpreter simulieren die Programmausführung
  - Code ausführen: Read-Eval-Loop (REPL)
- Tree-Walking-Interpreter traversieren den AST
- Auswertung der (Expression-) Knoten mit `eval(Expr e)`

Wir haben heute den semantischen Kern von L-Int gebaut. Wir haben eine Value-Hierarchie in Java implementiert und einen Interpreter, der ASTs für L-Int auswertet. In der nächsten Sitzung geht es darum, wie wir diese ASTs automatisch aus Quelltext erzeugen: mit einem handgeschriebenen Lexer und einem Recursive-Descent-Parser.

Zum Nachlesen

Zum Thema Auswertung von Ausdrücken im AST findet sich eine schöne Darstellung in (Nystrom 2021) in Kapitel 7 “Evaluating Expressions”. Siek (2023) beschäftigt sich mit der Interpretation von einfachen Ausdrücken in Kapitel 1 “Preliminaries”.

Lernziele

- k3: Ich kann die Traversierung von ASTs implementieren: Pattern Matching mit **`switch`**-Expresssion oder Visitor-Pattern
- k3: Ich kann bei der Traversierung des AST die dort abgelegten Ausdrücke auswerten

Challenges

Den Interpreter beim Arbeiten beobachten
Gegeben ist folgender AST:

```
Expr expr =
    new SubExpr(
        new AddExpr(new IntLiteral(4), new IntLiteral(2)),
        new AddExpr(new IntLiteral(1), new IntLiteral(3))
    );
```

Verfolgen Sie den Aufruf von `eval(expr)` Schritt für Schritt:

1. In welcher Reihenfolge werden die AST-Knoten ausgewertet?
2. Welcher Wert wird jeweils zurückgegeben?
3. Welchen endgültigen Wert liefert der Interpreter?

Notieren Sie das Ergebnis beispielsweise als **Tabelle** mit den Spalten **Knoten**, **ausgewertete Kinder** und **Rückgabewert**.
Den fehlenden Fall ergänzen
Die im Skript gezeigte `eval()`-Methode behandelt `IntLiteral` und `AddExpr`, aber noch nicht `SubExpr`.
Ergänzen Sie den passenden **`case`**-Zweig. Entwickeln Sie anschließend zwei kleine Tests, die zeigen, dass Ihre Implementierung korrekt ist. Einer der Tests soll insbesondere aufdecken, ob versehentlich die Operanden vertauscht wurden.

Woher kann ein Laufzeitfehler kommen?
In den arithmetischen Fällen wird `Value.asInt(...)` aufgerufen. Diese Methode wirft eine `Exception`, wenn der übergebene Wert kein `IntVal` ist.
Diskutieren Sie:

- Kann `Value.asInt(...)` bei einem AST, der nur aus den aktuell definierten L-Int-Knoten besteht, tatsächlich fehlschlagen?
- Warum könnte die Methode trotzdem einen Laufzeitfehler prüfen und nicht einfach einen Cast verwenden?
- Was ist der Unterschied zwischen einem `Expr`-Knoten und einem `Value`?

4.4. L-Int: Lexer - Handcodierte Implementierung

TL;DR

Der Lexer (auch "Scanner") soll den Zeichenstrom in eine Folge von Token zerlegen. Zur Spezifikation der Token werden reguläre Ausdrücke verwendet.
Von Hand implementierte Lexer arbeiten üblicherweise iterativ und durchlaufen die Eingabe schrittweise von links nach rechts. Dabei verarbeiten wir immer das aktuelle Zeichen im Eingabestrom.
Lexer sollten möglichst effizient sein, da sie noch direkt auf der niedrigsten Abstraktionsstufe arbeiten und u.U. oft durchlaufen werden.
Die Palette an Fehlerbehandlungsstrategien im Lexer reicht von "aufgeben" über den "Panic Mode" ("gobbeln" von Zeichen, bis wieder eines passt) und Ein-Schritt-Transformationen bis hin zu speziellen Lexer-Regeln, die beispielsweise besonders häufige Typos abfangen.

Videos

Vorlesung [YT], [HSBI]

4.4.1. Lexer: Erzeugen eines Token-Stroms aus einem Zeichenstrom

Aus dem Eingabe(-quell-)text

```
;; demo
(+ 1 222 )
```

erstellt der Lexer (oder auch Scanner genannt) eine Sequenz von Token:

```
<LPAREN, "(">   <PLUS, "+">   <INT, "1">   <INT, "222">   <RPAREN, ")">
```

Wir wollen den Eingabetext (Zeichenstrom) in *abstraktere Einheiten* aufsplitten: die **Token**.

Jedes Token hat einen **Typ** und speichert ein **Lexem** - das ist der Teil der Eingabe, der zu dem Token gematcht wurde.

Der Lexer versucht anhand von Regeln (reguläre Ausdrücke) zu erkennen, welche Token in der Eingabe vorkommen. In der Regel geht man zeichenweise von links nach rechts durch die Eingabe und versucht anhand des aktuellen Zeichens zu erkennen, welches Token hier beginnt. Bei Mehrdeutigkeiten muss man entsprechend noch weitere folgende Zeichen mit einbeziehen.

Important

4.4.1.1. Definition wichtiger Begriffe

- **Token**: Tupel (`Tokenname, optional: Wert`)

  Der Tokenname ist ein abstraktes Symbol, welches eine lexikalische Einheit repräsentiert (Kategorie). Die Tokennamen sind die Eingabesymbole für den Parser.

  Token werden i.d.R. einfach über ihren Namen referenziert. Token werden häufig zur Unterscheidung von anderen Symbolen in der Grammatik in Fettschrift oder mit großen Anfangsbuchstaben geschrieben.

  Ein Token kann einen Wert haben, etwa eine Zahl oder einen Bezeichner, der auf das zum Token gehörende Pattern gematcht hatte (also das Lexem). Wenn der Wert des Tokens eindeutig über den Namen bestimmt ist (im Beispiel oben beim Komma oder den Klammern), dann wird häufig auf den Wert verzichtet.

- **Lexeme**: Sequenz von Zeichen im Eingabestrom, die auf ein Tokenpattern matcht und vom Lexer als Instanz dieses Tokens identifiziert wird.

- **Pattern**: Beschreibung der Form eines Lexems

  Bei Schlüsselwörtern oder Klammern etc. sind dies die Schlüsselwörter oder Klammern selbst. Bei Zahlen oder Bezeichnern (Namen) werden i.d.R. reguläre Ausdrücke zur Beschreibung der Form des Lexems formuliert.

- **Lexer**: zerteilt Zeichenstrom (Eingabe) in eine Folge von Wörtern (Token)

- **Parse Tree**: repräsentiert die Struktur der Sätze, wobei jeder Knoten dem Namen einer Regel der Grammatik entspricht. Die Blätter bestehen aus den Token samt ihren Werten.

- **Grammatik**: Regeln für den Aufbau der betrachteten Sprache (formale Beschreibung der Wörter und Sätze)

4.4.2. Modellierung der Token

```
eform ::= '(' '+' eform eform+ ')' | '(' '-' eform eform+ ')' | INT
INT   ::= [0-9]+
```

```
public enum TokenKind {
    PLUS, MINUS, LPAREN, RPAREN, SEMI,
    INT,  // [0-9]+
    EOF
}
```

```
6
7  public record Token(TokenKind kind, String lexeme) {}
```

Tip

4.4.2.1. Design-Diskussion: Modellierung Token

Es gibt verschiedene Möglichkeiten, Token zu modellieren.
Aus der C-Welt stammt die gezeigte Modellierung: Man nutzt ein Enum, um die Tokenarten zu modellieren. (Wir werden dieses Enum im Laufe des Semesters noch deutlich ergänzen.) Für die Repräsentation der Token nutzt man dann eine gemeinsame Daten-Klasse, die einfach die Tokenart und das Lexem kombiniert. Das ist übersichtlich, allerdings muss man für Token wie `PLUS` das Lexem "+" dann unnötigerweise nochmal explizit mit speichern.

Important

4.4.2.2. Weitere Token und Eigenschaften

Token enthalten oft auch Positionen oder Quelltextbereiche (Zeile, Spalte, Start- und Endindex). Das erleichtert anständige Fehlermeldungen erheblich.
`EOF` ist ein synthetisches Token und hat kein Lexem aus dem Quelltext. Das hier verwendete "<EOF>" ist eine mögliche Debug-Darstellung, aber kein tatsächliches Lexem.

4.4.3. Grundstruktur eines Lexers

```
1  public class LispyLexerLint {
2      protected String source;
3      protected int current;
4
5      public List<Token> scan(String input) {
6          source = input;  current = 0;
7          List<Token> tokens = new ArrayList<>();
8
9          do tokens.add(nextToken());
10         while (tokens.getLast().kind() != EOF);
11
12         return tokens;
13     }
14
15     // Arbeitsmethode: nextToken()
16     // Hilfsmethoden:  isAtEnd(), peek(), advance() ...
17 }
```

Der Lexer läuft über den Eingabestring und erzeugt eine vollständige Liste mit Token. Irrelevante Zeichen wie Whitespace und Kommentare werden dabei immer wieder entfernt (s.u.).

Die Einsprungsfunktion `scan()` stützt sich dabei auf der zentralen Hilfsmethode `nextToken()` ab, die das nächste Token an der aktuellen Eingabeposition bildet. Die Implementierung ist algorithmisch einfach: Wir schauen uns das aktuelle Zeichen an, entscheiden, um welchen Token-Typ es sich handelt, und rücken den Positionszeiger entsprechend vor. Falls nötig, ziehen wir weitere Zeichen in diese Entscheidung mit ein.

Anmerkung: In Lispy werden Whitespaces und Kommentare in späteren Verarbeitungsstufen nicht mehr benötigt und deshalb entfernt. In Sprachen wie Python spielen Whitespaces aber eine wichtige syntaktische Rolle und werden deshalb geeignet als Token erfasst und an den Parser weitergegeben.

Tip

4.4.3.1. Design-Diskussion: Schnittstelle Lexer

In der gezeigten Variante verarbeitet der Lexer den *kompletten* Input und erzeugt (falls möglich) eine *vollständige Liste* mit den zur Eingabe gehörenden Token. Der nachfolgende Parser kann damit vom Lexer entkoppelt werden und bekommt als Input einfach eine Liste mit Token und arbeitet auf dieser Liste - die Parserimplementierung muss nicht wissen, dass es eine Klasse `Lexer` gibt und wie diese aussieht. Der Preis ist dabei möglicher Mehrbedarf an Speicher und möglicherweise schlechtere Fehlermeldungen.

4.4.4. Partielle Beispielimplementierung Lexer für L-Int

```java
protected Token nextToken() {
    skipIrrelevant();

    if (!isAtEnd()) {
        char c = advance();

        if (Character.isDigit(c)) {
            String lexeme = c + readWhile(Character::isDigit);
            return Token.from(INT, lexeme);
        }
        return switch (c) {
            case '(' -> Token.from(LPAREN);
            case '+' -> Token.from(PLUS);
            default -> throw new RuntimeException("unexpected character");
        };
    }
    return Token.from(EOF);
}
```

Hier wird eine typische Implementierung von `nextToken()` gezeigt.

Irrelevante Zeichen wie Whitespace und Kommentare werden zu Beginn entfernt (übersprungen). Falls wir danach nicht am Ende der Eingabe sind, können wir das aktuelle Zeichen lesen und entscheiden, welches Token hier vorliegt (oder beginnt). Gelesene Zeichen werden aus der Eingabe "entfernt": Beim Aufruf von `advance()` wird der Index `current` inkrementiert (falls möglich).

Wenn das aktuelle Zeichen eine Ziffer ist, lesen wir so lange weiter, bis keine Ziffer mehr folgt, und erzeugen ein `INT`-Token. Andernfalls unterscheiden wir anhand des Zeichens die Operatoren und Klammern. Unbekannte Zeichen führen zu einem Fehler. Das ist im Kern das, was auch generierte Lexer tun.

Falls wir am Ende der Eingabe angelangt sind, erzeugen wir das synthetische Token `EOF` für den Parser.

Wichtig: Die Subtraktion und die Behandlung der schließenden Klammer fehlen hier und dienen als kleine Übung.

Anmerkung: Häufig findet man im Lexer keinen "schönen" objektorientierten Ansatz. Das hat häufig Performancegründe …

Tip

4.4.4.1. Nützliche Hilfsmethoden für Token

Um beim Erzeugen der Token nicht immer wieder **new** schreiben zu müssen, und um nicht bei Token mit festem Lexem dieses immer wieder extra angeben zu müssen, kann man sich einige statische `from()`-Hilfsmethoden im Enum `Token` definieren, die aus dem Tokentyp das passende Token erzeugen (vgl. Challenge).

4.4.5. Hilfsfunktionen im Lexer: Zeichenhandling

```
protected boolean isAtEnd() { return current >= source.length(); }

protected char peek() { return peek(0); }
protected char peek(int la) {
    if (la < 0 || current + la >= source.length()) return '\0';
    return source.charAt(current + la);
}

protected char advance() { return isAtEnd() ? '\0' : source.charAt(current++);
    }

protected boolean skipIrrelevant() {}  // Homework

protected String readWhile(Predicate<Character> pred) {
    StringBuilder sb = new StringBuilder();
    while (!isAtEnd() && pred.test(peek())) sb.append(advance());
    return sb.toString();
}
```

Um nicht immer wieder manuell mit dem Index `current` und der Eingabe `source` hantieren zu müssen (das hat sich als sehr fehlerträchtig herausgestellt), definiert man sich typischerweise einige Hilfsmethoden. Die hier gezeigten Namen sind tatsächlich relativ verbreitet, aber man findet beispielsweise auch oft `consume()` statt `advance()`.

Wir müssen im Lexer immer wieder erkennen, ob bereits alle Zeichen im Input abgearbeitet sind. Um nicht immer wieder den Index `current` mit der Länge des Inputs `source` vergleichen zu müssen, kapselt man dies in der Hilfsmethode `isAtEnd()`. Diese liefert **`true`**, sobald wir alle Zeichen im Input-String `source` bearbeitet haben.

In der Hauptschleife `nextToken()` müssen wir auf das aktuelle Zeichen schauen. Auch hier bietet es sich an, dies nicht immer wieder manuell über `source.charAt(current)` zu tun, sondern dies über eine Hilfsmethode `peek()` zu kapseln. Im obigen Beispiel ist auch gleich eine Implementierung für ein `peek()` mit Lookahead gezeigt - wir werden später auch das nächste und übernächste Zeichen nach dem aktuellen Zeichen benötigen.

Für das Weiterschalten gibt es die Methode `advance()`. Diese gibt das aktuelle Zeichen zurück und schaltet

auf das nächste Zeichen, d.h. ein nachfolgendes `peek()` liefert dann das nächste Zeichen. Damit wir nicht über den Input-String hinaus laufen, wird die Weiterschaltung nur gemacht, wenn wir noch nicht am Ende des Inputs sind. Anderenfalls liefern wir ein Null-Zeichen (`\0`) zurück.

Der Aufruf von `skipIrrelevant()` "entfernt" vom aktuellen Zeichen an alle nicht benötigten Zeichen wie Whitespace und Kommentare und stoppt, sobald ein benötigtes Zeichen auftaucht. Dabei muss die Funktion selbst prüfen, ob überhaupt noch gelesen werden darf, oder ob bereits das Ende des Inputs erreicht ist. Falls Zeichen entfernt wurden, wird **`true`** zurückgegeben. (In unserer Implementierung springt der Index `current` weiter, es werden keine Zeichen physisch entfernt.) **Wichtig**: Die Implementierung von `skipIrrelevant()` dient als kleine Hausaufgabe (Übungsblatt).

Die Funktion `readWhile()` bekommt einen Test `pred` als Argument und liest so lange, wie das aktuelle Zeichen diesen Test erfüllt und das Ende der Eingabe nicht erreicht ist. Jedes gelesene Zeichen wird an einen StringBuilder angehängt und am Ende als String zurückgeliefert. Diese Funktion ist sehr praktisch, wenn man beispielsweise aufeinander folgende Ziffern einlesen will: `String lexeme = readWhile(Character::isDigit);` liest so lange, wie Ziffern in der Eingabe vorkommen und liefert diese zunächst als String zurück.

Anmerkung: Der gezeigte Lexer verändert den Eingabestring nicht: Er verschiebt lediglich den Index `current`. "Zeichen werden aus der Eingabe entfernt" bedeutet hier also, dass der Index `current` inkrementiert wird.

Tip

4.4.5.1. Design-Diskussion: Index-Handling und Robustheit

Das Hantieren mit dem Index und dem Weiterschalten muss gut aufeinander abgestimmt sein - der Lexer sollte hier nicht in einen ungültigen Zustand geraten können.
Grundsätzlich könnte man es den Usern überlassen, die Aufrufe immer korrekt zu machen und damit nie über das Ende des Inputs hinaus zu lesen. In der Praxis lohnt es sich aber, etwas mehr Robustheit einzubauen.
Im obigen Beispiel habe ich die Rückgabe eines besonderen Null-Zeichens gewählt, dies stammt aus der C-Welt und bedeutet dort "wir sind am Ende des Strings angekommen".
Alternativ könnte man auch eine Exception auslösen beim Versuch, am Ende der Eingabe noch ein weiteres `advance()` zu machen. Dies zieht aber schnell eine relativ umständliche Fehlerbehandlung nach sich …

4.4.6. Typische Muster für die Definition von Token

1. Schlüsselwörter

   - Ein eigenes Token (RE) für jedes Schlüsselwort, oder
   - Erkennung als Name (`ID`) und nachträglich Vergleich mit Wörterbuch sowie Korrektur des Tokentyps

   Wenn Schlüsselwörter über je ein eigenes Token abgebildet werden, benötigt man für jedes Schlüsselwort einen eigenen RE bzw. DFA. Die Erkennung als Bezeichner und das Nachschlagen in einem Wörterbuch (geeignete Hashtabelle) sowie die entsprechende nachträgliche Korrektur des Tokentyps kann die Anzahl der Zustände im Lexer signifikant reduzieren! Wir werden dies später in der Sprachvariante L-Var sehen.

2. Operatoren

- Ein eigenes Token für jeden Operator, oder
- Gemeinsames Token für jede Operatoren-Klasse

3. Bezeichner: Ein gemeinsames Token für alle Namen
4. Zahlen: Ein gemeinsames Token für alle numerischen Konstanten (ggf. Integer und Float unterscheiden)

   Für Zahlen führt man oft ein Token "`NUM`" oder "`INT`"ein. Als Attribut speichert man das Lexem als String. Alternativ kann man das Lexem bereits in eine Zahl konvertieren und als (zusätzliches) Attribut speichern. Dies kann in späteren Stufen viel Arbeit sparen.
5. String-Literale: Ein gemeinsames Token
6. Komma, Semikolon, Klammern, ...: Je ein eigenes Token

7. Regeln für Whitespace und Kommentare etc. ...

   Normalerweise benötigt man Kommentare und Whitespaces in den folgenden Stufen nicht und entfernt diese deshalb aus dem Eingabestrom. Dabei könnte man etwa Whitespaces in den Pattern der restlichen Token berücksichtigen, was die Pattern aber sehr komplex macht. Die Alternative sind zusätzliche Pattern, die auf die Whitespace und anderen nicht benötigten Inhalt matchen und diesen "geräuschlos" entfernen. Mit diesen Pattern werden keine Token erzeugt, d.h. der Parser und die anderen Stufen bemerken nichts von diesem Inhalt.

   Gelegentlich benötigt man aber auch Informationen über Whitespaces, beispielsweise in Python. Dann müssen diese Token wie normale Token an den Parser weitergereicht werden.

Tip

4.4.6.1. Anmerkung: Längster Match

Wenn es mehrere matchende Tokenregeln gibt, wird in der Regel das längste Lexem bevorzugt.
Wenn es mehrere gleich lange Alternativen gibt, muss man mit Vorrangregeln bzgl. der Token arbeiten. Bei ANTLR wäre das die Reihenfolge, in der die Token definiert wurden.
Bei einem handgeschriebenen Lexer muss die Implementierung dieses Verhalten selbst festlegen.

4.4.7. Fehler bei der lexikalischen Analyse

Problem: Eingabestrom sieht so aus: `(fi (== a 42)...)`

Der Lexer kann nicht erkennen, ob es sich bei `fi` um ein vertipptes Schlüsselwort handelt oder um einen Bezeichner: Es könnte sich um einen Funktionsaufruf der Funktion `fi()` handeln ... Dieses Problem kann erst in der nächsten Stufe (Parser) sinnvoll erkannt und behoben werden.

=> Was tun, wenn keines der Pattern auf den Anfang des Eingabestroms passt?

Optionen:

- Aufgeben …

  Eventuell vielleicht sogar die beste und einfachste Variante :-)

- “Panic Mode”: Entferne so lange Zeichen, bis ein Pattern passt.

  Das verwirrt u.U. den Parser, kann aber insbesondere in interaktiven Umgebungen hilfreich sein. Ggf. kann man dem Parser auch signalisieren, dass hier ein Problem vorlag.

- Ein-Schritt-Transformationen:
  - Füge fehlendes Zeichen in Eingabestrom ein.
  - Entferne ein Zeichen aus Eingabestrom.
  - Vertausche ein Zeichen:
    * Ersetze ein Zeichen durch ein anderes.
    * Vertausche zwei benachbarte Zeichen.

  Diese Transformationen versuchen, den Input in einem Schritt zu reparieren. Das ist durchaus sinnvoll, da in der Praxis die meisten Fehler in dieser Stufe durch ein einzelnes Zeichen hervorgerufen werden: Es fehlt ein Zeichen oder es ist eines zu viel im Input. Es liegt ein falsches Zeichen vor (Tippfehler) oder zwei benachbarte Zeichen wurden verdreht …
  Allerdings muss man sich in Erinnerung rufen, dass gerade in dieser ersten Phase eines Compilers die Geschwindigkeit stark im Fokus steht und eine ausgefeilte Fehlerkorrekturstrategie die vielen kleinen Optimierungen schnell wieder zunichte machen kann. Obendrein kann der Lexer ohne weiteren syntaktischen oder semantischen Kontext oft nicht zuverlässig entscheiden, was gemeint war.

- Fehler-Regeln: Matche typische Typos (Fehler-Token)

  Gelegentlich findet man in den Grammatiken für den Lexer extra Regeln, die häufige bzw. typische Typos matchen und dann passend darauf reagieren.

4.4.8. Wrap-up

- Lexer zerlegt den Quelltext von links nach rechts in Token
  - Token: hat Tokenart und Lexem
  - Reguläre Ausdrücke beschreiben die Formen der Token
  - Whitespace und Kommentare überspringen (sofern nicht für spätere Verarbeitungsstufen benötigt)

- Implementierungsansatz: Zeichenweise Iteration durch die Eingabe
  - Entscheidung für ein Token am aktuellen Zeichen (plus ggf. weitere folgende Zeichen (“Lookahead”))
  - Ggf. Lookahead notwendig, z.B. bei “==” vs. “=”
  - Bevorzugung des längsten Matches
  - Ende der Eingabe: Künstliches `EOF`-Token

Zum Nachlesen

Zum Thema Lexing findet sich eine schöne Darstellung in (Nystrom 2021) in Kapitel 4 “Scanning”.

Lernziele

- k1: Ich kenne die Aufgaben eines Lexers
- k2: Ich kann typische Fehler und Lösungsansätze in der lexikalischen Analyse erklären
- k3: Ich kann für ein Problem eine typische Einteilung von Token vornehmen
- k3: Ich kann einen Top-Down-Lexer mit Read-Ahead implementieren

Challenges

Design-Diskussion

1. Modellierung Token

   Welche Möglichkeiten, Token zu modellieren, gibt es noch? Welche Vor- und Nachteile haben die Varianten?

2. Schnittstelle Lexer

   Eine häufige Variante der Lexer-Schnittstelle ist die Methode `nextToken()` als Einsprungmethode in den Lexer: `public Token nextToken()`. Der Parser bekommt dann ein Lexer-Objekt und ruft selbst immer wieder bei Bedarf `nextToken()` auf. Was könnten Vor- und Nachteile zur oben im Skript gezeigten Variante sein?

Token-Strom vorhersagen: Wo endet das Lexem?
Bestimmen Sie für jede Eingabe den Token-Strom, den der Lexer erzeugen sollte. Geben Sie jeweils Tokenart und Lexem an; das synthetische `EOF`-Token können Sie dazuschreiben.

```
(+ 12 345)
(- 7 8)
(+12 3)
(+ 12a 3)
```

Diskussionsfragen:

- Gehört ein Vorzeichen zum Integer-Literal oder ist es ein eigenes Token? Begründen Sie Ihre Entscheidung anhand der Tokenregeln für L-Int.
- Was passiert bei der letzten Eingabe: Erkennt der Lexer “12a” als ein fehlerhaftes Integer-Literal, oder liest er zunächst “12” als `INT`?

Hilfsfunktionen für Token
Jedes Token hat üblicherweise ein Attribut, in dem das Lexem gespeichert wird. Bei eindeutigen Token (etwa bei eigenen Token je Schlüsselwort oder bei den Interpunktions-Token) kann man sich das Attribut auch sparen, da das Lexem durch die Tokenart eindeutig rekonstruierbar ist.
Bei unserer Implementierung in Java müsste man dann entweder zwei verschiedene Klassen zur Modellierung der Token nutzen (was eher umständlich ist), oder man spendiert der `Token`-Klasse statische

Hilfsfunktionen, die ein Token bauen: einmal vollständig (mit Token-Typ und Lexem) und einmal nur mit dem Token-Typ und daraus abgeleitetem festen Lexem.
Wir haben oben zwei Verwendungsmöglichkeiten gesehen:

```
Token.from(INT, lexeme)
Token.from(LPAREN)
```

Implementieren Sie diese Hilfsmethoden. Wie sehen die Signaturen aus? Was wird in `TokenKind` zusätzlich noch benötigt, damit das funktioniert?

`nextToken()` für L-Int vervollständigen

Die Beispielimplementierung erkennt "(", "+" und Integer-Literale. Ergänzen Sie sie so, dass sie auch "-" und ")" als Token erkennt.
Skizzieren Sie Ihren Code. Prüfen Sie anschließend gemeinsam, ob Ihr Lexer bei dieser Eingabe alle Zeichen genau einmal verarbeitet:

```
(- 12 (+ 3 45))
```

Diskussionsfragen:

- Welche Aufgabe übernimmt `advance()` in den einzelnen Fällen?
- Warum wird bei einem Integer-Literal nicht nur ein Zeichen gelesen?

Read-Ahead: Unterscheiden von "=" und "=="

Angenommen, wir müssen die Token `EQEQ` (Lexem "==") und `EQUAL` (Lexem "=") erkennen und voneinander unterscheiden. Beide fangen mit "=" an. Wie könnte man das in der **`switch`**-Expression in `nextToken()` formulieren?

Fehlerfall mit einer klaren Regel behandeln

Der gezeigte Lexer bricht bei einem unbekannten Zeichen mit einer Exception ab. Überlegen Sie sich für L-Int eine einfache alternative Strategie: Der Lexer soll beim ersten unbekannten Zeichen eine verständliche Fehlermeldung erzeugen und anschließend entweder abbrechen oder weiter nach dem nächsten erkennbaren Token suchen.
Verwenden Sie beispielsweise diese Eingabe:

```
(+ 12 @ 3)
```

Legen Sie fest, wie sich der Lexer verhalten soll, und beschreiben Sie die resultierenden Token beziehungsweise die Fehlermeldung.

Wichtig: Falls Ihr Lexer nach einem Fehler weiterläuft: Formulieren Sie eine Regel, die sicherstellt, dass er dabei nicht an derselben Eingabeposition hängen bleibt.

4.5. L-Int: LL-Parser mit Recursive Descent

TL;DR

Ein LL-Parser mit rekursivem Abstieg (*recursive descent*, auch *RD-Parser*) verarbeitet den Tokenstrom von links nach rechts und wählt Alternativen anhand eines begrenzten Lookaheads:

- Für Nichtterminale werden Parsermethoden aufgerufen; Terminale werden mit `match()` geprüft und "konsumiert".
- Alternativen werden anhand ihrer möglichen Anfangstokens ausgewählt. Optionale Teile und Wiederholungen werden typischerweise mit **`if`** und **`while`** umgesetzt.
- Die Grammatik muss eine eindeutige Vorhersage erlauben; bei leeren Alternativen sind dafür auch FOLLOW-Mengen relevant.

Zur Implementierung eines RD-Parsers kann man so vorgehen:

- Zu jeder Produktionsregel erstellt man eine gleichnamige Funktion.
- Wenn in der Produktionsregel andere Regeln "aufgerufen" werden, erfolgt in der Funktion an dieser Stelle der entsprechende Funktionsaufruf.
- Bei Terminalsymbolen wird das erwartete Token mit `match()` geprüft.

Dabei findet man wie bereits im Lexer die Funktionen `match()` und `advance()`, die sich hier aber auf den Tokenstrom beziehen. LL(1)-Parser schauen dabei immer nur das nächste Token an, LL(k) haben ein entsprechendes Lookahead von bis zu k Token.
RD-Parser haben ein Problem mit Linksrekursion in der Grammatik, da hier rekursiv Methoden aufgerufen werden und keine Token entfernt werden; eventuell auftretende Linksrekursion muss zunächst beseitigt werden.

Videos

Vorlesung [YT], [HSBI]

4.5.1. LL-Parser mit Recursive Descent: Ziel

- Aus Tokens einen AST bauen:

```
Expr parse(List<Token> tokens);
```

- Basis konkrete Syntax (Grammatik):

```
eform ::= '(' '+' eform eform+ ')' | '(' '-' eform eform+ ')' | INT
INT   ::= [0-9]+
```

- Zielformat AST (abstrakte Syntax):

```
Expr  = IntLiteral(Int) | AddExpr(Expr, Expr) | SubExpr(Expr, Expr)
```

Mit dem Lexer können wir jetzt eine Tokenliste erzeugen. Der nächste Schritt ist der Parser, den wir als Recursive-Descent-Parser implementieren. Wir verwenden dafür eine leicht umgeformte Grammatik ohne Linksrekursion. Die Methode `parse()` wird unser Einstiegspunkt sein und am Ende einen `Expr`-AST liefern, den wir mit unserem Interpreter auswerten können.

Wir haben in der Vorlesung LL-Parser kennengelernt, die den Parse-Tree von oben nach unten (*top-down*) aufbauen. Es gibt auch die Welt der LR-Parser, die den Baum von unten nach oben (*bottom-up*) aufbauen - diese betrachten wir hier nicht. Bei den LL-Parsern haben wir die Varianten "Recursive Descent" und "Tabellenbasiert" kennengelernt. Heute betrachten wir den Recursive-Descent-Parser (**RD-Parser**) genauer.

Tip

In unserem RD-Parser übergeben wir die Liste aller Token. Das passt zu unserer Modellierung des Lexers und entkoppelt softwaretechnisch beide Bausteine. Der Nachteil ist, dass der Lexer zunächst die komplette Eingabe bearbeitet, auch wenn der Parser vielleicht gleich beim ersten Token einen Fehler finden würde. Außerdem sieht man erst (schrittweise) alle Lexerfehler, bevor man dann (schrittweise) alle Parserfehler findet.
Die traditionelle Formulierung ist, dem RD-Parser einen Lexer zu übergeben. Der Parser muss dann selbst immer wieder `nextToken()` auf diesem Lexer aufrufen. Das sorgt einerseits für eine enge Kopplung zwischen den Bausteinen, hat aber andererseits potentiell leichte Effizienzvorteile und die Fehlermeldungen zwischen Lexer und Parser sind synchron.

Tip

Ein Parser erzeugt zunächst prinzipiell einen **Parse-Tree**. Diesen transformiert man in einem weiteren Schritt in einen **AST** ("AST-Builder").
Wenn man den Parser selbst schreibt, kann man diesen zweiten Schritt auch direkt mit dem Parsing zusammenfassen. Wir werden das tun und in unserem RD-Parser statt einem Parse-Tree direkt den AST ausgeben.
Wenn wir später ANTLR betrachten, werden wir von ANTLR einen Parse-Tree bekommen und müssen diesen noch mit einem AST-Builder in einen AST transformieren.

4.5.2. Grundidee LL-Parser mit Recursive Descent

Die Grammatik (konkrete Syntax) besteht aus einer Menge von Produktionsregeln:

```
r ::= X s
```

Diese Produktionen werden jeweils in eine dazu passende Funktion überführt:

```
Expr parseR() {
    match(X);
    return parseS();
}
```

- Für jedes Nonterminal in der Grammatik wird eine Methode/Funktion mit diesem Namen definiert

- Referenzen auf ein Terminal (Token) `T` werden durch den Aufruf der Methode `match(T)` aufgelöst
  - `match(T)` "konsumiert" das aktuelle Token, falls dieses mit `T` übereinstimmt
  - Anderenfalls löst `match()` eine Exception aus
- Referenzen auf Nichtterminale (Regeln) `s` werden durch Methodenaufrufe `parseS()` aufgelöst

Tip

Wenn die Parserfunktionen ohne Rückgabewert arbeiten, dann prüft der Parser nur, ob die Eingabestruktur passt. Es wird dann kein Parse-Tree o.ä. aufgebaut.
In unserem Fall wollen wir einen Parse-Tree bzw. sogar einen AST erzeugen. Das ist oben noch ziemlich versteckt in dem `return parseS();`. Wir werden später sehen, wie wir den Baum aufbauen können.

4.5.3. Alternative Subregeln

Alternativen in einer Regel:

```
1     a | b | c
```

Auflösen durch Unterscheidung am aktuellen Token (und falls nötig weiterem Lookahead):

```
1 return switch (peek()) {
2     case PredictingA -> parseA();
3     case PredictingB -> parseB();
4     case PredictingC -> parseC();
5     default -> throw new RuntimeException("expected expression");
6 };
```

Anhand des aktuellen Tokens (`peek()`) wird entschieden, welche Alternative vorliegt und welche der `parseXYZ()`-Methoden entsprechend aufgerufen werden soll.

`PredictingA` soll andeuten, dass man mit dem aktuellen Token eine Vorhersage für die Regel `a` versucht (hier kommen die FIRST- und FOLLOW-Mengen ins Spiel …). Wenn das der Fall ist, springt man entsprechend in die Funktion bzw. Methode `a()`. Formal berechnet man die Lookahead-Mengen mit `FIRST` und `FOLLOW`, um eine Entscheidung für die nächste Regel zu treffen. Praktisch betrachtet kann man sich fragen, welche(s) Token eine Phrase in der aktuellen Alternative starten können.

Für LL(1)-Parser betrachtet man immer das **aktuelle** Token (**genau *EIN* Lookahead-Token**), um eine Entscheidung zu treffen. Bei LL(k) nutzt man einen Lookahead von insgesamt bis zu k Token.

Tip

4.5.3.1. Exkurs LL(k)

Die folgende Regel ist für einen LL(1)-Parser nicht deterministisch behandelbar, da die Alternativen mit dem gleichen Token beginnen (die Lookahead-Mengen überlappen sich).

```
1 expr ::= ID '++' | ID '--'
```

Hier benötigt man entweder **zwei Lookahead-Tokens**, also einen LL(2)-Parser, oder man muss die Regel in eine äquivalente LL(1)-Grammatik umschreiben (→ **Linksfaktorisierung**, s.u.):

```
expr ::= ID ('++' | '--')
```

4.5.4. Grundstruktur eines RD-Parsers

```
public class LispyParserLint {
    protected List<Token> tokens;
    protected int current;

    public Expr parse(List<Token> tokenList) {
        tokens = tokenList;  current = 0;

        Expr expr = parseEform();
        if (!isAtEnd()) throw new RuntimeException("unexpected token");

        return expr;
    }

    // Arbeitsmethoden: parseEform(), parseAtom(), parseAddExpr(), parseSubExpr()
    // Hilfsmethoden:   isAtEnd(), peek(), advance(), match(), check() ...
}
```

Der Parser sieht strukturell dem Lexer ähnlich: Er hat eine Eingabefolge - hier die Token-Liste `tokens` - und einen Positionszeiger (aktueller Index `current`).

Den Einstieg in den Parser bildet die Methode `parse()`, die den gesamten Eingabestrom gemäß der Grammatik verarbeitet. Diese Methode bildet die Startregel `lint ::= eform EOF` der konkreten Grammatik ab.

Die einzelnen Regeln der (von direkter Linksrekursion befreiten) konkreten Grammatik bilden wir auf die Arbeitsmethoden `parseEform()`, `parseAtom()`, `parseAddExpr()`, `parseSubExpr()` ab.

Intern werden wir Hilfsmethoden wie `peek()`, `advance()` und `match()` verwenden, um die Tokens bequemer zu handhaben.

4.5.5. Partielle Beispielimplementierung RD-Parser für L-Int

Wir betrachten folgende konkrete Syntax in L-Int:

```
eform ::= '(' '+' eform eform+ ')' | '(' '-' eform eform+ ')' | INT
INT   ::= [0-9]+

lint  ::= eform EOF
```

Der Einstieg ist die Regel `lint`, die eine `eform` erwartet und ein anschließendes `EOF`-Token. Mehr ist in der Sprachstufe L-Int nicht erlaubt und wurde bereits in der Einstiegsmethode `parse()` umgesetzt.

Eine `eform` kann entweder eine Addition, eine Subtraktion oder einfach nur ein Integer-Literal sein. Letzteres ist ein Terminal, d.h. hier können wir einfach mit `match()` das entsprechende Token abfragen.

Die Alternativen für Addition und Subtraktion rufen jeweils die Regel für `eform` rekursiv auf.

Der Parser muss entlang dieser Regeln aufgebaut werden und die Token der Reihe nach über diese Regeln ableiten ("erklären"). Nur wenn dies vollständig gelingt, haben wir eine syntaktisch korrekte Eingabe. Ein Parser kann einen Parse-Tree oder - wie in unserem Beispiel - direkt einen AST erzeugen. Der gezeigte Code erzeugt den AST gemäß der folgenden abstrakten Syntax:

```
Expr  = IntLiteral(Int) | AddExpr(Expr, Expr) | SubExpr(Expr, Expr)
```

Wir haben hier also konkret eine Daten-Klasse `IntLiteral` für die Repräsentation von Integer-Literalen, die jeweils die gefundene Zahl speichern.

Analog für Addition und Subtraktion. Hier ist interessant, dass die vorgesehenen Knoten im AST nur binäre Knoten sind, in der konkreten Syntax aber jeweils zwei oder mehr Argumente für Addition und Subtraktion erlaubt sind. Ein reiner Parse-Tree würde diese Grammatik-Struktur im Baum widerspiegeln (siehe Challenge). Wir wollen hier einen abstrakteren Baum (AST) erzeugen und müssen entsprechend bereits beim Erzeugen des AST die Zielstruktur berücksichtigen.

```
protected Expr parseEform() {
    if (check(LPAREN)) {
        return switch (peek(1).kind()) {
            case PLUS -> parseAddExpr();
            default -> throw new RuntimeException("unexpected token");
        };
    }

    return parseAtom();
}


protected Expr parseAtom() {
    Token t = match(INT);
    return new IntLiteral(Integer.parseInt(t.lexeme()));
}
```

Für die Regel `eform` wird eine `parseEform()` angelegt, die direkt der Grammatikregel entspricht.

Tip

Die Sichtbarkeit ist **`public`** für die Einstiegsmethode `parse()` - diese Methode soll später zum Starten des Parsers genutzt werden. Die anderen (internen) Methoden bekommen die Sichtbarkeit **`protected`**, damit wir sie bei Bedarf in ableitenden Parser-Klassen überschreiben können. Wenn wir uns ganz sicher sind, dass wir eine interne Methode nie überschreiben wollen, können wir natürlich auch wie immer auf **`private`** gehen.

Wir unterscheiden in `parseEform()` anhand des aktuellen Tokens, ob es sich um die Alternativen für Addition (und Subtraktion, nicht gezeigt) handeln könnte (beide starten mit `'('`). Ansonsten muss es sich um die letzte Alternative `INT` handeln:

- Liegt ein `LPAREN` vor, entfällt die letzte Alternative. Es kann sich nur noch um eine Addition oder Subtraktion handeln.

  Da diese Alternativen beide mit `'('` beginnen, schauen wir nun auf das nächste Token:

  - Finden wir ein `ADD`- oder `SUB`-Token, rufen wir entsprechend die Regeln `parseAddExpr()` bzw. `parseSubExpr()` (nicht gezeigt) auf.
  - Anderenfalls melden wir einen Fehler, weil wir `LPAREN` gesehen haben, aber nichts, was laut unserer Grammatik darauf folgen müsste.

- Liegt *kein* `LPAREN` vor, kann es sich nur um die letzte Alternative `INT` handeln.

  Hier handelt es sich um ein Terminal-Symbol, welches wir in `parseAtom()` mit `match(INT)` überprüfen und gleichzeitig aus dem Tokenstrom "entfernen". Wenn das aktuelle Token dabei nicht `INT` ist, würde `match(INT)` einen Fehler auslösen.

Wichtig: Die Subtraktion fehlt hier und dient als kleine Übung.

Wichtig: Wir haben damit in `parseEform()` effektiv ***zwei* Tokens Lookahead** verwendet! Vgl. auch die nachfolgende Diskussion zur Linksfaktorisierung.

Die Methode `parseAtom()` entspricht genau dem weiter oben gezeigten Muster. Wir erwarten ein `INT`-Token, also rufen wir die Hilfsmethode `match()` mit diesem Parameter auf. `match()` prüft, ob das aktuelle Token wirklich diesen Typ hat und "entfernt" es aus dem Tokenstrom (inkrementiert den `current`-Index) und liefert es zurück. Anderenfalls würde `match()` eine entsprechende Exception werfen.

Das Lexem nutzen wir und konvertieren es in einen Java-Integer und erzeugen einen neuen AST-Knoten vom Typ `IntLiteral` - da dieser Typ vom Interface `Expr` ableitet, haben wir hier also den einfachsten möglichen AST gebildet.

Tip

4.5.5.1. Exkurs Linksfaktorisierung

Wir haben in `parseEform()` *implizit* **Linksfaktorisierung** angewendet: Wenn mehrere Alternativen einer Regel den gleichen Anfang haben, kann man diesen "heraus klammern":

```
ruleA ::= a b1 | ... | a bm
```

Hier könnte man den gemeinsamen Start `a` "ausklammern":

```
ruleA ::= a ruleB
ruleB ::= b1 | ... | bm
```

Bei uns im Beispiel war der gemeinsame Start für mehrere Alternativen die Klammer `'('`. Statt die Grammatik umzubauen, hatten wir das über das `if` und das nachfolgende extra Lookahead `peek(1)` gelöst. Mit der Umformung hätten wir eine weitere Hilfsfunktion im Parser, würden dann aber wieder mit `peek()` bzw. `peek(0)` und damit einem Token Lookahead auskommen (statt jetzt zwei).

4.5.6. Details Parser für L-Int: `parseAddExpr()`

Erinnerung an den Teil der Additionsregel in der konkreten Grammatik:

```
'(' '+' eform eform+ ')'
```

```
protected Expr parseAddExpr() {
    match(LPAREN);
    match(PLUS);

    List<Expr> ops = new ArrayList<>();
    ops.add(parseEform());
    while (!check(RPAREN)) ops.add(parseEform());

    match(RPAREN);

    return ops.stream().reduce(AddExpr::new).orElseThrow();
}
```

Auch hier sehen wir wieder das eingangs skizzierte Muster: Für jedes Terminal wird `match()` mit dem gesuchten Token-Typ aufgerufen. Danach müssen wir die Operanden parsen: Solange nicht ein `')'`-Token auftaucht, rufen wir rekursiv die Regel `parseEform()` auf. Jeder Operand muss ja selbst eine `eform` sein. Anschließend matchen wir noch die schließende rechte Klammer, und die Addition ist im Prinzip fertig geparst.

Nun müssen wir aber noch den Baum mit binären `AddExpr`-Knoten bauen. Dazu laufen wir über die Liste der geparsten Operanden und reduzieren den Stream. Dabei würde `(+ 1 2 3)` von links nach rechts geklammert: `(+ (+ 1 2)3)`.

Wichtig: Challenge (s.u.) beachten!

Tip

4.5.6.1. Exkurs Desugaring

Diese Auflösung ist ein einfaches Beispiel für ***Desugaring*** - hier wird aus einer komplexeren Operation mit potentiell beliebig vielen Operanden eine Folge von einfacheren binären Operationen gemacht. Wir werden noch komplexere Beispiele sehen, wenn wir in L-If über Kontrollstrukturen sprechen.

4.5.7. Hilfsfunktionen im Parser: Tokenhandling

```
protected boolean isAtEnd() { return peek().kind() == EOF; }

protected Token peek() { return peek(0); }
protected Token peek(int la) {
    if (la < 0 || current + la >= tokens.size()) return tokens.getLast();  //
        EOF
    return tokens.get(current + la);
}

protected Token advance() {
    return isAtEnd() ? tokens.getLast() : tokens.get(current++);
}

protected boolean check(TokenKind... expected) {
    return Arrays.stream(expected).anyMatch(e -> e == peek().kind());
```

```
15  }
16  protected Token match(TokenKind... expected) {
17      if (check(expected)) return advance();
18      throw new RuntimeException("unexpected token: " + peek());
19  }
```

Man definiert sich gern eine Handvoll von Hilfsfunktionen im Parser, die das Handling mit den Token erleichtern. Im Prinzip können diese Funktionen relativ symmetrisch zu den Pendants im Lexer implementiert werden - allerdings arbeitet man hier im Parser mit Token statt mit Zeichen.

Der auffälligste Unterschied zum Lexer: Wir verlassen uns nicht auf die Länge der Tokenliste, sondern behandeln das `EOF`-Token als Sentinel. Sobald wir dieses Token erreicht haben, brechen wir den weiteren Fortschritt mit `advance()` o.ä. ab.

Zusätzlich zu `isAtEnd()`, `peek()` und `advance()` haben wir noch zwei weitere Hilfsmethoden, die `peek()` und `advance()` unterschiedlich kombinieren. Wir werden in Zukunft häufig wissen wollen, ob das aktuelle Token in einer Menge von erlaubten Token vorkommt, weshalb `check()` und `match()` mit variadischen Parametern realisiert sind: Hier kann man einfach ein oder mehrere Tokentypen übergeben, ohne die Funktionen umständlich überladen zu müssen oder vorher Listen bilden zu müssen. `check()` prüft dabei nur, ob das aktuelle Token vom Typ her in der `expected` ist und liefert einen Boolean zurück, während `match()` im Fall des Zutreffens das aktuelle Token "konsumiert" und anderenfalls einen Fehler wirft.

4.5.8. Auflösen von (links-) rekursiven Grammatiken

Wir wollen die folgenden Ausdrucksvarianten erlauben:

`1`, `1 + 2`, `1 + 2 + 3`, ...

Nun definieren wir eine einfache Grammatik, die diese Ausdrücke akzeptiert:

```
1  expr ::= expr '+' expr | INT
```

Diese intuitiv erscheinende Grammatik erlaubt zwar die gewünschten Ausdrücke, hat aber zwei Probleme!

4.5.8.1. Problem 1: Mehrdeutigkeit

Diese Grammatik ist mehrdeutig!

`1 + 2 + 3` kann mit der Grammatik sowohl als `(1 + 2) + 3` (*linksassoziativ*) als auch als `1 + (2 + 3)` (*rechtsassoziativ*) geparst werden. So etwas wollen wir im Compiler generell vermeiden, auch wenn das hier konkret keine Probleme verursachen würde.

Die folgende umgeformte Grammatik akzeptiert die gleiche Sprache und ist nicht mehr mehrdeutig:

```
1  expr ::= expr '+' INT | INT
```

Hier erzwingen wir die übliche **Klammerung von links** her (**linksassoziativ**): `1 + 2 + 3` kann nur noch als `(1 + 2) + 3` geparst werden.

4.5.8.2. Problem 2: Linksrekursion

Die umgeformte Grammatik ist immer noch **linksrekursiv**: Zu Beginn einer Regel rufen wir die Regel selbst wieder auf und gelangen so in eine Endlosrekursion, da kein Token konsumiert wird.

Das folgende Beispiel versucht die Umsetzung der Grammatik mit Hilfe von Backtracking (teste die erste Alternative, wenn das nicht klappt - einen Fehler wirft - versuche die zweite Alternative). `position()` soll den aktuellen Index holen bzw. wieder zurücksetzen. Dieses Beispiel soll nur das Problem bzgl. der Endlosrekursion verdeutlichen - es funktioniert nicht!

```
1  Expr parseExpr() {
2      int start = position();
3
4      try {
5          // first alternative: expr ::= expr '+' INT
6          Expr left = parseExpr();  // Waily waily!!!
7          match(PLUS);
8          Expr right = parseInt();
9          return new AddExpr(left, right);
10
11     } catch (ParseException e) {
12         // second alternative: expr ::= INT
13         position(start);  // roll back token stream
14         return parseInt();
15     }
16 }
17
18 Expr parseInt() {
19     Token t = match(INT);
20     return new IntLiteral(t.lexeme());
21 }
```

Man kann zwischen **direkter** und **indirekter** Linksrekursion unterscheiden:

- Wenn die Regel sofort beim Start einer Alternative aufgerufen wird, handelt es sich um direkte Linksrekursion (wie im Beispiel).
- Indirekte Linksrekursion hätte man, wenn das erste Symbol einer Regel eine andere Regel ist, die wiederum die erste Regel aufruft: `ruleA ::= ruleB ...` und `ruleB ::= ruleA ...`.

Wenn man eine *direkt* linksrekursive Regel der Form hat:

```
1  ruleA ::= ruleA a1 | ... | ruleA am | b1 | ... | bn |
```

(Dabei dürfen die `bi` nicht mit `ruleA` beginnen.)

Dann kann *direkte* Linksrekursion mit folgender Regel behoben werden:

```
1  ruleA  ::= b1 ruleAA | ... | bn ruleAA
2  ruleAA ::= a1 ruleAA | ... | am ruleAA | eps
```

(Mit `eps` als das leere Wort ϵ.)

Damit können wir unsere Grammatik umschreiben:

```
expr  ::= INT expr'
expr' ::= '+' INT expr' | eps
```

Dies können wir hier noch weiter abkürzen zu:

```
expr  ::= INT ('+' INT)*
```

Mit dieser umgeformten Regel können wir jetzt unseren RD-Parser wie gewohnt aufbauen. Es gibt keine direkte Linksrekursion mehr, mit der der Parser in eine Endlosschleife geraten könnte.

```
Expr parseExpr() {
    Expr e = parseInt();
    while (check(PLUS)) {
        advance();
        e = new AddExpr(e, parseInt());
    }
    return e;
}

Expr parseInt() {
    Token t = match(INT);
    return new IntLiteral(t.lexeme());
}
```

Anmerkung: Normalerweise würden wir für ein Nichtterminal direkt `match()` aufrufen - damit der Code lesbarer bleibt, wurde das `match(INT)` und das Erzeugen des neuen `IntLiteral` in eine Hilfsmethode ausgelagert.

4.5.9. Lispy-Pipeline komplett

```
String source = "(- (+ 1 2) 3)";

List<Token> tokens = new LispyLexerLint().scan(source);
Expr ast = new LispyParserLint().parse(tokens);

Value result = new InterpreterLint().eval(ast);
// result: IntVal(0)
```

4.5.10. Wrap-up

Implementierung als RD-Parser:

- Regeln (Nichtterminale) werden zu Parsermethoden
- Token (Terminals) mit `match()` prüfen (und “konsumieren”)
- Auswahl Alternativen über nächste(s) Token(s) (Lookahead)
- Beachtung und Auflösung von Linksrekursion

Zum Nachlesen

Zum Thema Lexing findet sich eine schöne Darstellung in (Nystrom 2021) in Kapitel 6 “Parsing Expressions”.

Lernziele

- k2: Ich kann den prinzipiellen Aufbau von *recursive-descent* LL-Parsern am Beispiel erklären
- k3: Ich kann LL(1)- und LL(k)-Parser implementieren
- k3: Ich kann mit Linksrekursion umgehen und diese ggf. auflösen

Challenges

Quizzfragen:

- Wie kann man aus einer LL(1)-Grammatik einen LL(1)-Parser mit rekursivem Abstieg implementieren? Wie "übersetzt" man dabei Token und Regeln?
- Wie geht man mit Alternativen um? Wie mit optionalen Subregeln?
- Warum ist Linksrekursion i.A. bei LL-Parsern nicht erlaubt? Wie kann man Linksrekursion beseitigen?
- Wann braucht man mehr als ein Token Lookahead? Geben Sie ein Beispiel an.

Optionale Subregeln

Wie werden optionale Subregeln in RD-Parsern implementiert?

- "Eins oder keins":

```
(T)?
```

- "Mindestens eins"

```
(T)+
```

Desugaring

Bei der Parserregel `parseAddExpr()` haben wir folgenden Code gesehen:

```
protected Expr parseAddExpr() {
    match(LPAREN);
    match(PLUS);

    List<Expr> ops = new ArrayList<>();
    ops.add(parseEform());
    while (!check(RPAREN)) ops.add(parseEform());

    match(RPAREN);

    return ops.stream().reduce(AddExpr::new).orElseThrow();
}
```

1. Was passiert in den Randfällen?
   - Ist `(+ 1)` erlaubt? Falls ja, was produziert der Parser?
   - Ist `(+)` erlaubt? Falls ja, was produziert der Parser?
2. Wird damit der Teil der Regel `eform ::= '('+'eform eform+ ')'` korrekt umgesetzt? Warum (nicht)? Falls "nein", wie müsste der Code korrigiert werden?

Parser-Ablauf nachvollziehen
Verfolgen Sie den Aufruf von `parseEform()` für die Eingabe:

```
(+ 1 (+ 2 3))
```

Erstellen Sie dazu eine Tabelle/Liste mit diesen Spalten: Schritt, Aufgerufene Methode, Aktuelles Token, Aktion. Tragen Sie insbesondere ein, wann `parseEform()` rekursiv aufgerufen wird und welche Methode die einzelnen Tokens konsumiert. Sie müssen noch nicht den vollständigen AST ausrechnen.

Subtraktion ergänzen
Ergänzen Sie den im Skript gezeigten Parser so, dass auch Subtraktionen geparst werden. Skizzieren Sie die nötigen Änderungen in `parseEform()` und eine passende Methode `parseSubExpr()`.
Prüfen Sie Ihren Entwurf gedanklich an diesen Eingaben:

```
(- 8 3)
(- 8 (+ 2 1) 1)
```

Welche AST-Knoten sollten jeweils entstehen?

Den gemeinsamen Anfang ausklammern
Die Alternativen für Addition und Subtraktion beginnen beide mit `'('`. Formulieren Sie die Grammatik mit einer zusätzlichen Regel für den Operator so um, dass die Auswahl zwischen den äußeren Alternativen von `eform` nur **ein** Lookahead-Token benötigt.
Skizzieren Sie anschließend, wie die Parsermethoden zu dieser umgeformten Grammatik aussehen könnten. Die akzeptierte Sprache und die AST-Knoten sollen gleich bleiben.

5. Sprache L-Expr: erweiterte Ausdrücke, Vorrangregeln

5.0.1. Inhalte

- Lispy: Sprachstufe L-Expr <: L-Int
- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- InterpreterLexpr
- Vorrangregeln und Assoziativität für Infix-Notation
- ANTLR: Grammatik, Vorrang mit gestuften Regeln vs. ANTLR4-Precedence
- ANTLR: Visitor mit Rückgabe (zustandslos)

5.1. L-Expr: Multiplikation/Division und unäres Minus

TL;DR

Wir erweitern die Sprachstufe L-Int zu L-Expr, indem wir Multiplikation, Division und ein unäres Minus hinzufügen. In der Lispy-Notation bedeutet das vor allem, dass wir der konkreten Grammatik neue Operatorformen (`* e e`), (`/ e e`) und (`- e`) hinzufügen.
Die abstrakte Syntax (AST) wird um zwei neue Knotentypen `MulExpr` und `DivExpr` erweitert; unäres Minus wird nicht als eigener Knotentyp modelliert, sondern beim AST-Bau auf vorhandene Knoten desugart.
Jede neue Sprachstufe baut auf der vorherigen auf: L-Expr *erweitert* L-Int. Entsprechend leiten in der Implementierung die Bausteine der neuen Stufe von den Bausteinen der alten ab, überschreiben bei Bedarf Methoden und rufen für das bisherige Verhalten `super.XYZ()` auf (z.B. `class InterpreterLexpr extends InterpreterLint`).
Wir verwenden für Division in allen Sprachstufen dieselbe Semantik wie in Java: ganzzahlige Division von `int`-Werten, bei der der Rest verworfen wird.

5.1.1. Rückblick: L-Int

- Konkrete Syntax:

```
1  eform ::= '(' '+' eform eform+ ')' | '(' '-' eform eform+ ')' | INT
2  INT   ::= [0-9]+
3  lint  ::= eform EOF
```

- Abstrakte Syntax:

```
Expr = IntLiteral(Int) | AddExpr(Expr, Expr) | SubExpr(Expr, Expr)
Lint = Expr
```

- AST:

```
public interface Expr {}
public record AddExpr(Expr left, Expr right) implements Expr {}
public record SubExpr(Expr left, Expr right) implements Expr {}
public record IntLiteral(int value) implements Expr {}
```

5.1.2. Erweiterung zu L-Expr: Multiplikation und Division

Wir bleiben bei **Integer-Ausdrücken**, fügen aber Multiplikation und Division hinzu.

- Beispiel:

```
(* 2 (+ 3 4))
```

- **Neue konkrete Syntax in Lispy (Stufe L-Expr)**:

```
eform ::= '(' '+' eform eform+ ')' | '(' '-' eform eform+ ')' | INT
INT   ::= [0-9]+
---

eform ::= '(' '*' eform eform+ ')'
        | '(' '/' eform eform+ ')'
        | '(' '-' eform ')'

lexpr ::= eform EOF
```

Die Grundidee in Lispy ist: Wir fügen einfach zwei weitere Alternativen für die `eform`-Regel mit neuen Operatoren hinzu. Semantisch bleiben wir bei Integer-Werten und mehrstelligen Operationen.

Important

5.1.2.1. Hinweis zum Lesen der Regeln

In dem obigen Beispiel wurden über den `---` die bisherigen Regeln aus L-Int übernommen. Unter dem `---` sehen wir die neuen Regeln, die in L-Expr hinzukommen. Dies soll zum einen die Features der neuen Sprachstufe hervorheben. Zum anderen soll es auch deutlich machen, dass L-Expr tatsächlich L-Int plus ein paar neue Fähigkeiten ist.
Für eine formale Grammatik wären normalerweise zwei Regeln mit dem selben Namen nicht erlaubt. Man muss also die beiden `eform`-Regeln (eine aus L-Int, eine neu in L-Expr) **gedanklich mergen**:

```
eform ::= '(' '+' eform eform+ ')'
        | '(' '-' eform eform+ ')'
        | '(' '*' eform eform+ ')'
        | '(' '/' eform eform+ ')'
        | '(' '-' eform ')'
        | INT
```

```
7
8  INT   ::= [0-9]+
9
10 lexpr ::= eform EOF
```

Dieses Prinzip werden wir ab L-Expr für die konkreten und abstrakten Grammatiken anwenden. Aus Platzgründen werden wir aber irgendwann darauf verzichten, alle bisherigen Regeln erneut über den `---` aufzuschreiben.

5.1.3. Abstrakte Syntax für L-Expr

```
Expr  = IntLiteral(Int) | AddExpr(Expr, Expr) | SubExpr(Expr, Expr)
---

Expr  = MulExpr(Expr, Expr) | DivExpr(Expr, Expr)

Lexpr = Expr
```

```
public interface Expr {}
```

Zusätzlich zu den Klassen aus L-Int

```
public record AddExpr(Expr left, Expr right) implements Expr {}
public record SubExpr(Expr left, Expr right) implements Expr {}
public record IntLiteral(int value) implements Expr {}
```

bekommen wir durch/in L-Expr:

```
public record MulExpr(Expr left, Expr right) implements Expr {}
public record DivExpr(Expr left, Expr right) implements Expr {}
```

Important

5.1.3.1. Hinweis zu den AST-Klassen

Jede Sprachstufe wird auf der Vorgängersprachstufe aufbauen, d.h. jede Stufe übernimmt die Regeln und fügt neue hinzu. Wir haben also eine Art Vererbungsbeziehung: Die aktuelle Stufe erbt von der Vorgängerstufe und fügt neue Features hinzu.
Bei den AST-Klassen bedeutet das, dass wir weitere Klassen ergänzen, um die neuen Features im Baum repräsentieren zu können. D.h. wir bleiben zunächst bei `Expr` als Obertyp für die AST-Knoten und fügen nur die neu benötigten Knoten `MulExpr` und `DivExpr` hinzu.

5.1.4. Interpreter: Erinnerung an L-Int

```
public class InterpreterLint {

    public Value eval(Expr expr) {
        return switch (expr) {
```

```
5               case IntLiteral(var v) -> new IntVal(v);
6
7               case AddExpr(var l, var r) -> {
8                   int a = Value.asInt(eval(l));
9                   int b = Value.asInt(eval(r));
10                  yield new IntVal(a + b);
11              }
12
13              // TODO: SubExpr
14
15              default -> throw new RuntimeException("unexpected expression: " +
                    expr);
16          };
17      }
18  }
```

Tip

Erinnerung: Die Subtraktion war Teil des L-Int-Übungsblattes und ist hier nicht gezeigt.

5.1.5. Interpreter L-Expr: Erben von L-Int und Ergänzen der neuen Funktionalität

```
1   public class InterpreterLexpr extends InterpreterLint {
2
3       @Override
4       public Value eval(Expr expr) {
5           return switch (expr) {
6               case MulExpr(var l, var r) -> {
7                   int a = Value.asInt(eval(l));
8                   int b = Value.asInt(eval(r));
9                   yield new IntVal(a * b);
10              }
11
12              // TODO: DivExpr
13
14              default -> super.eval(expr);
15          };
16      }
17  }
```

Important

5.1.5.1. Hinweis zur Vererbung der Bausteine

L-Expr *ist* L-Int plus Multiplikation, Division und ein unäres Minus.
Unser Interpreter für L-Expr braucht also die Fähigkeiten des L-Int-Interpreters und muss Multiplikation und Division *ergänzen. Dies können wir in objektorientierten Sprachen wie Java leicht erreichen, indem der L-Expr-Interpreter vom L-Int-Interpreter ableitet: `class InterpreterLexpr extends InterpreterLint`. In der Methode `eval()` wollen wir die neuen Fälle ergänzen, deshalb überschreiben wir die geerbte Methode und fügen die neuen Fälle hinzu. Damit die alten Fälle weiterhin funktionieren, rufen wir im `default`-Zweig einfach `super.eval(expr);` auf und bekommen damit das in

`InterpreterLint` implementierte bisherige Verhalten.
Das werden wir von nun an in allen Bausteinen unserer Interpreter-Pipeline so machen! Wir nutzen analog zu (Siek 2023) Vererbung als einfachen Mechanismus zur Wiederverwendung von Verhalten.
Jede neue Sprachstufe "erbt" von der Vorgängersprachstufe:

- In den Grammatiken nutzen wir die alten Regeln weiter und ergänzen neue Regeln/Alternativen (unter dem `---`)
- Bei den Java-Bausteinen (Klassen, Interfaces) leiten die Klassen der aktuellen Sprachstufe jeweils von den Klassen der Vorgängersprachstufe ab und erweitern das Verhalten durch Überschreiben und `super.XYZ()` für das bisherige Verhalten
- Die JUnit-Tests leiten ebenfalls von den Tests der Vorgängerstufe ab, um das bisherige Verhalten immer mitzutesten

5.1.5.2. Unäres Minus

Das unäre Minus taucht **nicht** im AST auf!

Das ist eine Design-Entscheidung, um die Anzahl der AST-Typen möglichst klein und die notwendigen Transformationen möglichst einfach zu halten.

Beim Parsen oder beim Erzeugen des AST aus dem Parse-Tree kann man ein unäres Minus auf den bereits vorhandenen Subtraktionsknoten `SubExpr` "reduzieren". … Ein weiterer Fall von *Desugaring* …

Tip

Hinweis: Das Einbauen des unären Minus bzw. des *Desugaring* ist Teil der Challenges und/oder des L-Expr-Übungsblattes und ist hier nicht gezeigt.

5.1.5.3. Semantik der Division: Integer-Division wie in Java

Wir bleiben in allen Sprachstufen bei **Ganzzahlen** und nutzen überall dieselbe Divisions-Semantik wie in Java:

- Operanden: `int`
- Ergebnis: `int`
- Rest wird **abgeschnitten** (Division mit Abrunden Richtung 0)

Beispiele:

```
(/ 7 3)          ;; => 2
(/ (-7) 3)       ;; => -2
(/ 7 (-3))       ;; => -2
(/ (-7) (-3))    ;; => 2
```

Wichtig:

- In Lispy-Formen (`(/ 7 3)`) und in Java-ähnlichen Ausdrücken (`7 / 3`) verwenden wir dieselbe Integer-basierte Semantik wie in Java.

- Division durch 0 führt zu einem **Laufzeitfehler** (in Java `ArithmeticException`). Wir behandeln diesen Fall nicht besonders in der Sprache.

Tip

Hinweis: Das Einbauen der Division ist Teil der Challenges und/oder des L-Expr-Übungsblattes und ist hier nicht gezeigt.

5.1.6. Wrap-up

- L-Expr = L-Int + Multiplikation, Division und unäres Minus (nur Integer-Ausdrücke)

- Konkrete Syntax: neue Lispy-Formen `(* e1 e2 ...)`, `(/ e1 e2 ...)`, `(- e)`
- Abstrakte Syntax: neue AST-Knoten `MulExpr` und `DivExpr` für binäre Operationen
- Unäres Minus wird beim AST-Bau auf vorhandene Knoten desugart

- Sprachstufen bauen schrittweise aufeinander auf:
  - Grammatiken übernehmen alte Regeln und ergänzen neue Alternativen
  - Java-Klassen leiten von Vorgängerstufe ab, überschreiben Methoden und nutzen `super.XYZ()` für bestehendes Verhalten

Zum Nachlesen

In (Nystrom 2021) kann man in Kapitel 6 “Parsing Expressions” und Kapitel 7 “Evaluation Expressions” nachlesen.

Lernziele

- k2: Ich kann den Unterschied zwischen konkreter und abstrakter Syntax am Beispiel von L-Int und L-Expr erklären.
- k2: Ich kann beschreiben, wie L-Expr die Sprachstufe L-Int um Multiplikation, Division und unäres Minus erweitert.
- k3: Ich kann die konkrete Lispy-Grammatik von L-Int so erweitern, dass Multiplikation, Division und unäres Minus korrekt erkannt werden.
- k3: Ich kann die abstrakte Syntax (AST) von L-Int um neue Knotentypen für Multiplikation und Division ergänzen und den Interpreter entsprechend anpassen.
- k3: Ich kann unäres Minus beim AST-Bau auf vorhandene AST-Knoten desugaren und die Auswirkungen dieser Design-Entscheidung erläutern.
- k4: Ich kann erklären und am Beispiel zeigen, wie Vererbung zwischen Interpreter-Klassen (z.B. `InterpreterLexpr extends InterpreterLint`) die Wiederverwendung von Implementierungen zwischen Sprachstufen ermöglicht.

Challenges

Unäres Minus als Desugaring modellieren
Wir haben in L-Expr eine neue konkrete Form `(- e)` für unäres Minus eingeführt.

1. Skizzieren Sie eine Regel, wie `(- e)` beim AST-Bau in einen vorhandenen AST-Knoten übersetzt werden kann (z.B. in einen `SubExpr`). Schreiben Sie diese Regel textuell oder in einer Art Pseudocode auf.
2. Diskutieren Sie in der Gruppe: Welche Vor- und Nachteile hat es, kein eigenes `UnaryMinusExpr` einzuführen, sondern unäres Minus auf bestehende AST-Knoten zu desugaren?

Mehrstellige Operatoren im AST
In Lispy können Sie `(+ 1 2 3 4)` oder `(* 2 3 4)` schreiben, also Operatoren mit mehr als zwei Operanden. Unsere abstrakte Syntax für `AddExpr`, `MulExpr` usw. ist aber binär (jeweils genau zwei Operanden).

1. Skizzieren Sie, wie der AST für `(+ 1 2 3 4)` mit nur binären `AddExpr`-Knoten aussehen könnte (zeichnen Sie den Baum oder schreiben Sie die verschachtelte Struktur auf).
2. Überlegen Sie, ob Sie diesen Baum eher links-assoziativ `((((1 + 2)+ 3)+ 4))` oder rechts-assoziativ `((1 + (2 + (3 + 4))))` bauen würden. Diskutieren Sie kurz, ob das hier (bei Addition/Multiplikation) semantisch einen Unterschied macht und in welchen Fällen Assoziativität wichtig werden könnte.

Design der Interpreter-Hierarchie
Die Interpreter-Klassen sind so aufgebaut, dass jede neue Sprachstufe von der vorherigen erbt, z.B. `InterpreterLexpr extends InterpreterLint`.

1. Skizzieren Sie in einem Mini-Klassendiagramm, wie die Interpreter-Hierarchie in den ersten beiden Sprachstufen aussieht (nur Klassenname und `eval`-Methode).
2. Diskutieren Sie in der Gruppe:
   - Was sind Vorteile dieser Vererbungsstrategie, wenn wir viele aufeinander aufbauende Sprachstufen haben?
   - Welche Schwierigkeiten könnten auftreten, wenn eine spätere Sprachstufe ein Verhalten der Basisstufe ändern statt nur erweitern möchte?

5.2. L-Expr: Vorrang und Assoziativität

TL;DR

In Lispy mit Präfix-Notation und vielen Klammern gibt es keine Mehrdeutigkeit: Die Struktur des Ausdrucks ist durch die Klammerung eindeutig vorgegeben.
In einer Java- oder C++-ähnlichen Infix-Notation dagegen brauchen wir **Vorrangregeln** und **Assoziativität**, damit Ausdrücke wie `1+2*3` eindeutig interpretiert werden. Multiplikation und Division sollen stärker binden als Addition und Subtraktion, und alle diese Operatoren verhalten sich in unseren Beispielen

linksassoziativ (also `1-2-3 = (1-2)-3`).
Diese Informationen kodieren wir nicht "irgendwo im Parser", sondern **direkt in der konkreten Grammatik**. Dazu verwenden wir eine **gestufte Grammatik** mit mehreren Ebenen wie `expr`, `term` und `atom`, wobei `expr` die Operatoren mit der niedrigsten Priorität und `atom` die mit der höchsten Priorität beschreibt. Die Wiederholung `(...)*` in diesen Regeln erzeugt dabei automatisch **linksassoziative** Parse-Bäume. So erreichen wir, dass z.B. `1+2*3` immer als `1+(2*3)` und nicht als `(1+2)*3` geparst wird.

5.2.1. Java-artige Infix-Syntax: Vorrangregeln

In Lispy gibt es durch die Präfix-Notation und die Klammern **keine Vorrangprobleme**:

- `(* (+ 1 2)3)` ist immer eindeutig

In Java-ähnlicher Infix-Syntax:

- `1+2*3 == 1+(2*3)!= (1+2)*3`

- Multiplikation/Division binden **stärker** als Addition/Subtraktion.

Die Eingabe `1+2*3` muss als `1+(2*3)` interpretiert werden, da `*` Vorrang vor `+` hat.

5.2.2. Assoziativität (Klammerung)

Nicht nur der **Vorrang** (Multiplikation/Division vor Addition/Subtraktion) ist wichtig, sondern auch die **Assoziativität** der Operatoren.

Für unsere Infix-Ausdrücke gilt:

`+`, `-`, `*`, `/` sind **linksassoziativ**: (werden von **links** geklammert)

```
1 - 2 - 3   =   (1 - 2) - 3
8 / 4 / 2   =   (8 / 4) / 2
1 + 2 + 3   =   (1 + 2) + 3
```

5.2.3. Vorrang und Assoziativität: Gestufte Grammatik-Regeln

Um Vorrang und Assoziativität zu berücksichtigen, formuliert man traditionell in der konkreten Grammatik "gestufte" Regeln. Dabei tauchen die Operatoren mit der kleinsten Priorität ganz oben auf, und die Operatoren mit höherer Priorität folgen in den tieferen Stufen, und ganz unten kommen die Operatoren mit der höchsten Priorität. Diese letzte Regel nennt man oft `atom` oder `primary` oder auch `factor`.

```
expr  ::= term ('+' term)*
term  ::= atom ('*' atom)*
atom  ::= '-' atom
        | INT
```

Tip

Hinweis: Das Einbauen von Subtraktion und Division sind aus Gründen der Übersichtlichkeit hier nicht gezeigt. Wir betrachten das als Teil der Challenges und/oder des L-Expr-Übungsblattes.

Tip

Hinweis: Geklammerte Ausdrücke haben ebenfalls wie das unäre Minus die höchste Priorität. Wir werden das als Teil der Challenges und/oder des L-Expr-Übungsblattes einbauen.

Beispiel Parsen “1+2*3” und “1+2+3”

Diese etwas komplex anmutende Struktur ist ein Standardmuster, das Sie in vielen Parsern wiederfinden werden, und soll sicherstellen, dass die Vorrangregeln für Addition und Multiplikation korrekt beachtet werden, d.h. dass `2+3*4` als `2+(3*4)` geparst wird und nicht fälschlicherweise als `(2+3)*4` erkannt wird.

Zusätzlich muss bei LL-Parsern Links-Rekursion vermieden werden: Die Parser-Regeln werden in Funktionsaufrufe übersetzt, d.h. bei einer Links-Rekursion würde man dieselbe Regel immer wieder aufrufen, ohne ein Token aus dem Token-Strom zu entnehmen.

Eigenschaften:

- Vorrang über die **Struktur**:
  - `expr` = schwächste Ebene (+, −)
  - `term` = mittlere Ebene (*, /)
  - `atom` = stärkste Ebene (`INT`, unäres Minus)
- Assoziativität über das `(...)*`: Wiederholung nach rechts → linksassoziative Bäume. Jede Wiederholung von rechts hängt einen neuen Operator rechts an den bereits aufgebauten Teilausdruck - dadurch landen die früheren Operatoren weiter oben im Baum, und der Baum wird links-astig: `((1+2)+3)`.

5.2.4. Wrap-up

- In Infix-Notation brauchen wir:
  - **Vorrang**: * und / binden stärker als + und −
  - **Assoziativität**: +, −, *, / sind linksassoziativ, z.B. `1-2-3 = (1-2)-3`
- **Gestufte konkrete Grammatik** (`expr -> term -> atom`) modelliert Vorrang:
  - `expr`: Ebene mit + und −
  - `term`: Ebene mit * und /
  - `atom`: Ebene mit Literalen, Klammern und unärem Minus
- Muster `X ::= Y (op Y)*` erzeugt **linksassoziative** Bäume

Zum Nachlesen

Im (Nystrom 2021) kann man in Kapitel 6 “Parsing Expressions” und Kapitel 7 “Evaluation Expressions” nachlesen.

Lernziele

- k2: Ich kann erklären, warum Infix-Notation ohne Vorrangregeln und Assoziativität mehrdeutig ist.
- k2: Ich kann beschreiben, wie eine gestufte Grammatik (`expr`/`term`/`atom`) Operatorvorrang modelliert.
- k3: Ich kann eine gegebene flache Grammatik für arithmetische Ausdrücke so umschreiben, dass Vorrang und Linksassoziativität korrekt berücksichtigt werden.
- k3: Ich kann zu einem Ausdruck wie `1+2*3-4` anhand der Grammatik die zugehörige Klammerung und den Parse-Tree konstruieren.
- k2: Ich kann erläutern, wie das Wiederholungsmuster `(...)*` in Grammatikregeln zu linksassoziativen Parse-Bäumen führt.
- k4: Ich kann verschiedene Varianten einer Expression-Grammatik vergleichen und begründen, ob sie mehrdeutig sind oder nicht.

Challenges

Ambiguität der “flachen” Grammatik

Gegeben sei:

```
expr ::= expr '+' expr
       | expr '*' expr
       | INT
```

1. Zeigen Sie, dass diese Grammatik für den Ausdruck `1+2*3` mehrdeutig ist.
2. Zeichnen Sie zwei verschiedene Parse-Trees für `1+2*3`, die aus dieser Grammatik entstehen können.
3. Welche der beiden Klammerungen entspricht der gewünschten Interpretation in Java/C++? Welche nicht?
4. Formulieren Sie in Worten, welches Problem hier auftritt und wie eine gestufte Grammatik dieses Problem löst.

Assoziativität sichtbar machen

Betrachten Sie erneut die Grammatik aus dem Beispiel:

```
expr  ::= term ('+' term)*
term  ::= atom ('*' atom)*
atom  ::= '-' atom
        | INT
```

1. Zeichnen Sie die Parse-Trees für die folgenden Eingaben nach dieser Grammatik:
   - `1 + 2 + 3`
   - `2 * 3 * 4`
   - `1 + 2 * 3`

- `8 * 4 * 2`
- `-3 * 4`

2. Schreiben Sie zu jedem Parse-Tree die explizite Klammerung des Ausdrucks dazu, z.B. `(1 + 2)+ 3`.
3. Diskutieren Sie in der Gruppe:
   - Warum sind die Bäume links-astig?
   - Wie würde eine Grammatik aussehen, die + (oder *) **rechtsassoziativ** parst?

Vorrang durch Umformung der Grammatik
Starten Sie erneut mit der folgenden Grammatik:

```
expr ::= expr '+' expr
       | expr '*' expr
       | INT
```

1. Formulieren Sie eine neue Grammatik, die:
   - Multiplikation und Division stärker bindet als Addition und Subtraktion (wie in Java/C++),
   - alle vier Operatoren linksassoziativ macht, und
   - geklammerte Ausdrücke erlaubt.
2. Verwenden Sie dazu die drei Stufen `expr`, `term`, `atom`.
3. Testen Sie Ihre Grammatik gedanklich an den Ausdrücken:

```
1 + 2 * 3
4 * 5 + 6
7 + 8 + 9
```

4. Notieren Sie für jeden Ausdruck die Klammerung, die Ihre Grammatik erzwingt.

5.3. L-Expr: ANTLR-Grammatik und Vorrangregeln

TL;DR

ANTLR ist ein Parser-Generator, der aus einer Grammatik einen Lexer, einen LL(*)-Parser und Hilfsklassen für die Traversierung des Parse-Trees in verschiedenen Zielsprachen (Java, Python, C++, …) generiert.
In einer ANTLR-Grammatik werden **Lexer-Regeln** mit **Großbuchstaben** geschrieben. Jede Lexer-Regel definiert einen regulären Ausdruck und liefert einen Token-Typ zurück, dessen Name der Regelname ist. Der Lexer wählt immer den **längsten Match**; bei Gleichstand gewinnt die **zuerst definierte** Regel.
Parser-Regeln beginnen mit einem **Kleinbuchstaben** und bestehen aus einem Regelnamen (linke Seite) und einer Produktion (rechte Seite). Auf der rechten Seite können Token-Typen (Lexer-Regeln oder implizite Tokens in `'...'`) und andere Parser-Regeln in beliebiger Reihenfolge kombiniert werden. Parser-Regeln werden in Methoden des generierten Parsers übersetzt (vergleichbar zu einem rekursiv-absteigenden Parser).
Parser-Regeln können mehrere **Alternativen** haben, getrennt durch `|`. Bei Mehrdeutigkeiten entscheidet

ANTLR zugunsten der **ersten passenden Alternative**. Mit ?, * und + lassen sich optionale, beliebig oft oder mindestens einmal wiederholte Teile formulieren.
ANTLR4 erlaubt im Gegensatz zu vielen klassischen LL(1)- bzw. handgeschriebenen rekursiv-absteigenden Parsern auch **direkte Linksrekursion** in Parser-Regeln und nutzt diese u.a. zur automatischen Behandlung von **Operatorpräzedenz** und **Assoziativität**.

5.3.1. Von klassischer CFG zur ANTLR-Grammatik

Erinnerung an “Programmieren 2”:

- ANTLR ist eine Java-Bibliothek
- Generiert aus einer Grammatik einen Lexer und LL(*)-Parser
- Eingabe-String plus Lexer/Parser → Parse-Tree mit Kontextklassen
- Zusätzlich: Klassen für Visitor-Pattern

In Prog2 haben wir bereits ANTLR als Java-Bibliothek kennengelernt. Wir haben uns dort angeschaut, wie wir das Tool in einem Java-Projekt über Gradle einbinden können, wie wir die generierten Lexer und Parser aufrufen, und wir haben sogar die zu den Grammatik-Regeln generierten Kontextklassen betrachtet und das Visitor-Pattern zum Ablaufen der Parse-Trees benutzt.

Wir benutzen hier ANTLR in der Version **4**, manchmal auch als “ANTLR4” geschrieben.

Jetzt wollen wir auf die ANTLR-Grammatik schauen.

5.3.2. Erinnerung: Konkrete Syntax für L-Expr

```
1  eform ::= '(' '+' eform eform+ ')' | '(' '-' eform eform+ ')' | INT
2  INT   ::= [0-9]+
3  ---
4
5  eform ::= '(' '*' eform eform+ ')' | '(' '/' eform eform+ ')' | '(' '-' eform '
       )'
6
7  lexpr ::= eform EOF
```

Im Prinzip sieht unsere konkrete Grammatik bereits der ANTLR-Grammatik recht ähnlich. Wir müssen zunächst die `::=` durch einen einfachen Doppelpunkt `:` ersetzen. Zusätzlich muss jede Regel mit einem Semikolon `;` abgeschlossen werden.

Darüber hinaus kommen noch Namenskonventionen, die die Art der Regel erkennen lassen: Kleingeschriebene Regeln gelten für den Parser, großgeschriebene Regeln sind Lexer-Regeln. Die Reihenfolge der Regeln ist u.a. für Prioritäten bei Mehrdeutigkeiten relevant.

5.3.3. ANTLR-Grammatik für L-Expr (Parser-Teil)

```
grammar Lispy;

lexpr       : eform EOF ;

eform       : atom                              #atomExpr
            | '(' '-' eform ')'                 #unaryMinusExpr
            | '(' '+' eform eform+ ')'          #addExpr
            | '(' '-' eform eform+ ')'          #subExpr
            | '(' '*' eform eform+ ')'          #mulExpr
            | '(' '/' eform eform+ ')'          #divExpr
            ;

atom        : INT ;
```

- `lexpr` ist der Einstiegspunkt
- `eform` entspricht unseren Ausdrücken

Hier sehen Sie den Parser-Teil einer ANTLR-Grammatik für L-Expr.

Als erste Zeile findet man immer die **Deklaration** `grammar <NAME>;`, wobei der Name `<NAME>` auch der Dateiname ist (`NAME.g4`) und später den Präfix für die generierten Lexer-/Parser-/Visitor-Klassen bildet. Unsere Grammatik würde also in der Datei `Lispy.g4` erwartet.

Parser-Regeln beginnen in ANTLR mit einem **Kleinbuchstaben** und können andere Parser-Regeln "aufrufen" und auch Lexer-Regeln beinhalten. Jede Regel endet mit einem Semikolon.

Wir haben eine Parser-Regel `lexpr` als **Einstiegspunkt** (erste Regel), die genau eine `eform` gefolgt vom Token `EOF` erwartet.

Die `eform`-Regel entspricht ziemlich direkt unserer konkreten EBNF-Grammatik. ANTLR4 kommt mit (direkter) Linksrekursion zurecht, weshalb wir die Grammatik relativ intuitiv formulieren können. Eine `eform` kann also entweder ein `atom` sein, oder eine `'(''-'eform ')'` usw.

Nach den Parser-Regeln folgen üblicherweise die Lexer-Regeln (siehe nächste Folie).

5.3.3.1. Formen der Subregeln

Standardregeln

Jede Regel hat einen Kopf (Regelname) und nach dem ":" den Regelkörper. Darin können Token wie `ID` oder Literale (implizite Token) `'='` und `';'` gematcht werden und andere Regeln "aufgerufen" werden wie `expr`. Jede Regel wird mit einem Semikolon abgeschlossen.

```
stmt  : ID '=' expr ';' ;
```

Um die Regel `stmt` anwenden zu können, müssen alle Elemente auf der rechten Seite der Regel erfüllt werden. Dabei müssen die Token wie `ID`, `'='` und `';'` in angegebenen Reihenfolge matchen und die Subregel `expr` muss erfüllt werden können.

Das abschließende Semikolon am Ende einer ANTLR-Regel markiert für ANTLR das Ende der Regel und dient als Hilfe beim Parsen der Grammatik und bezieht sich nicht auf den Input.

Alternativen

Wie auch im RD-Parser können wir in ANTLR-Grammatiken Alternativen ausdrücken:

```
1 stmt  : ID '=' expr ';' | expr ';' ;
```

Alternativen werden durch ein | getrennt. Hier muss genau eine Alternative erfüllt werden. Falls nötig, trennt man die Alternativen durch Einschließung in runden Klammern vom Rest der Regel ab (Gruppierung): `r : a (b | c)d ;`.

Wiederholungen (Multiplizitäten)

Teile der Regeln können wiederholt werden. Die Parser-Regeln nutzen die aus regulären Ausdrücken bekannten Operatoren + (einmal oder öfter) und * (beliebig oft) bzw. ? (einmal oder keinmal: "optional").

```
1 expr  : term ('+' term)* ;
```

Der durch den * gekennzeichnete Teil kann beliebig oft vorkommen oder auch fehlen. Entsprechend würde eine `expr` dann erfüllt sein, wenn ein `term` vorliegt, oder auch wenn ein `term` und ein `'+'` und ein weiterer `term` gematcht wurde, oder auch mit mehrfachen Wiederholungen von `'+'term`.

Bei einem + müsste der Teil mind. einmal vorkommen, und bei einem ? entsprechend einmal oder gar nicht.

(vgl. github.com/antlr/antlr4/blob/master/doc/parser-rules.md)

5.3.3.2. Reihenfolge in Grammatik definiert Priorität

Falls mehr als eine Parser-Regel (oder Alternative) die selbe Input-Sequenz matcht, löst ANTLR diese Mehrdeutigkeit auf, indem es die erste Regel/Alternative nimmt, die an der Entscheidung beteiligt ist.

```
1 start : stmt ;
2
3 stmt  : expr | ID  ;
4 expr  : ID   | NUM ;
```

Bei der Eingabe "foo" würde die Alternative `ID` in der Regel `expr` "gewinnen", weil sie in der Grammatik vor der Alternative `ID` in der Regel `stmt` kommt und damit Vorrang hat.

5.3.3.3. Benannte Alternativen (Label)

Man kann Alternativen in einer Produktionsregel auch individuell benennen über "`#...`". Das # leitet dabei die Benennung ein, gefolgt vom Label: `#foo` wäre also das Label `foo` für eine Alternative. Wenn man das nutzt, müssen immer **alle** Alternativen einer Regel benannt werden.

Benannte Alternativen sind später recht praktisch, da ANTLR benannte Alternativen im generierten Visitor wie normale Regeln behandelt und eigene `visitXYZ`-Methoden für die Label generiert.

Man kann die Label aber nicht in der Grammatik wie andere Regeln verwenden.

5.3.4. ANTLR-Grammatik für L-Expr (Lexer-Teil)

```
...


INT         : [0-9]+ ;

COMMENT     : ';;' ~[\r\n]* -> skip ;
WS          : [ \t\r\n,]+ -> skip ;
```

- Token-Typen: `INT`, `'('`, `'-'`, ...
- Whitespace und Kommentare werden ignoriert

Der Lexer-Teil der Grammatik definiert, wie ANTLR den Eingabetext in Tokens aufteilt. Üblicherweise findet man diese Regeln nach den Parser-Regeln (siehe vorige Folie).

Lexer-Regeln beginnen mit einem **Grossbuchstaben** und definieren einen regulären Ausdruck, der auf den Eingabetext angewendet wird.

```
INT : [0-9]+ ;
WS  : [ \t\n]+ -> skip ;
```

Jede Lexer-Regel entspricht einem (benannten) **Token**. Im Beispiel gibt es die Regel `INT`, die den regulären Ausdruck `[0-9]+` anwendet. Damit matcht das Token `INT` auf alle Zeichenfolgen, die aus einer oder mehreren Ziffern bestehen.

Die Whitespace-Token (`WS`) werden im Beispiel zwar gematcht, aber danach **verworfen** (`-> skip`) - wir müssen diese zwar erkennen und aus der Eingabe entfernen, brauchen sie aber nicht für die weitere Prüfung der Strukturen.

Zusätzlich gibt es auch nicht benannte Token ("**implizite Token**", s.u.), das sind die in `'` eingeschlossenen Zeichenketten, beispielsweise `'='` oder `';'`.

Das Ergebnis ist ein Strom von Tokens, auf dem der Parser aufbaut.

5.3.4.1. Implizite Token

Statt `PLUS` kann man in Parser-Regeln auch direkt `'+'` schreiben, d.h. das Lexem direkt ausschreiben und in **einfache** Anführungszeichen einschließen. ANTLR erzeugt dann automatisch ein unbenanntes Token (deshalb auch "implizites Token" genannt).

Die Länge der Zeichenkette spielt dabei keine Rolle, es werden hier immer einfache Anführungszeichen verwendet.

5.3.4.2. To EOF not to EOF?

Startregeln müssen nicht unbedingt den gesamten Input "konsumieren". Sie müssen per Default nur eine der Alternativen in der Startregel erfüllen.

Ohne EOF: Es darf ein Rest vom Input übrig bleiben

Betrachten wir noch einmal einen leicht modifizierten Ausschnitt aus der obigen Grammatik:

```
start : stmt ;
```

Die Startregel wurde so geändert, dass sie nur noch genau ein Statement akzeptieren soll.

In diesem Fall würde die Startregel bei der Eingabe “aa bb” nur den ersten Teil “aa” konsumieren (als Token `ID`) und das folgende “bb” ignorieren. **Das wäre in diesem Fall aber auch kein Fehler.**

Mit EOF: Der gesamte Input muss passen

Wenn der **gesamte Eingabestrom** durch die Startregel erklärt werden soll, dann muss das vordefinierte Token `EOF` am Ende der Startregel eingesetzt werden:

```
start : stmt EOF;
```

Hier würde die Eingabe “aa bb” zu einem Fehler führen, da nur der Teil “aa” durch die Startregel abgedeckt ist (Token `ID`), und der Rest “bb” zwar sogar ein gültiges Token wäre (ebenfalls `ID`), aber eben nicht mehr von der Startregel akzeptiert. Durch das `EOF` soll die Startregel aber den gesamten Input konsumieren und erklären, was hier nicht geht und entsprechend zum Fehler führt.

(vgl. github.com/antlr/antlr4/blob/master/doc/parser-rules.md)

Tip

5.3.4.3. Reservierte Schlüsselwörter

Die folgenden Strings stellen reservierte Schlüsselwörter von ANTLR dar und dürfen nicht als Token, Regel oder Label genutzt werden:

```
import, fragment, lexer, parser, grammar, returns, locals, throws, catch,
    finally, mode, options, tokens
```

`rule` ist zwar kein Schlüsselwort, wird aber als Methodenname bei der Codegenerierung verwendet. => Wie ein Schlüsselwort behandeln!
(vgl. github.com/antlr/antlr4/blob/master/doc/lexicon.md)

5.3.5. Arbeitsweise von ANTLR

- ANTLR generiert aus der Grammatik: Lexer, Parser, Visitor, …
- Lexer konsumiert Input und erzeugt Tokenstrom
- Parser konsumiert Tokenstrom und erzeugt Parse-Tree

```
var source = "(+ 1 2)";

var input = CharStreams.fromString(source);
var lexer = new LispyLexer(input);
```

```
var tokens = new CommonTokenStream(lexer);

var parser = new LispyParser(tokens);
var tree = parser.lexpr();
```

Web: Lispy (Grammatik und Regeln, Eingabe und Parse-Tree)

5.3.6. Verhalten des Lexers

1. **Längster Match**

   Primäres Ziel: Erkennen der längsten Zeichenkette

   ```
   INT  : [0-9]+ ;
   ID   : [a-zA-Z][a-zA-Z0-9]* ;
   ```

   Eingabe “foo42”: Token `ID` (nicht `ID`, `INT`)

   Die Regel, die den längsten Match für die aktuelle Eingabesequenz produziert, “gewinnt”.

2. **Reihenfolge** in Grammatik definiert Priorität

   ```
   FOR : 'for' ;
   ID  : [a-z][a-zA-Z]* ;
   ```

   Eingabe “for”: Token `FOR`

   Falls mehr als eine Lexer-Regel die selbe Inputsequenz matcht, dann hat die in der Grammatik zuerst genannte Regel Priorität.

3. **Implizite Token**: haben höchste Priorität

   ```
   foo  : '0' | NUM EOF ;

   NUM  : [0-9]+ ;
   WS   : [ \t\r\n]+ -> skip ;
   ```

   Eingabe “0”: Token `'0'` (nicht `NUM`)

   Implizite Tokens (`'...'`) werden intern **vor** expliziten Lexer-Regeln einsortiert und bekommen dadurch eine noch höhere Priorität.

5.3.7. Berücksichtigung von Präzedenz im Parser

Wir können die gestufte konkrete Grammatik, die die Operator-Präzedenz berücksichtigt, direkt nach ANTLR übersetzen:

- **konkrete Grammatik** (gestuft):

```
expr  ::= term ('+' term)*
term  ::= atom ('*' atom)*
atom  ::= '-' atom
        | INT
```

- **ANTLR**-Grammatik:

```
expr  : term ('+' term)* ;
term  : atom ('*' atom)* ;
atom  : '-' atom
      | INT
      ;
```

Die Präzedenz wird dabei durch die Struktur berücksichtigt: `expr` über `term` über `atom`.

Die gewünschte Linksassoziativität für die Operatoren +und * wird durch die Formulierung `(...)*` berücksichtigt.

5.3.8. Direkte Linksrekursion und ANTLR4 - automatische Präzedenz

ANTLR4 kann **direkte Linksrekursion** automatisch auflösen. Die Regel `r : r T U | V ;` kann also in ANTLR4 verarbeitet werden.

ANTLR nutzt beim Auflösen der direkten Linksrekursion die Strategie des *"precedence climbing"* und löst nach der **Reihenfolge der linksrekursiven Alternativen** in einer Regel auf. Dies kann man ausnutzen, um über die Reihenfolge der linksrekursiven Alternativen die Operator-Präzedenz abzubilden:

Aus der Grammatik mit gestuften Regeln zur Berücksichtigung von Präzedenz und (Links-) Assoziativität …

```
expr  : term ('+' term)* ;
term  : atom ('*' atom)* ;
atom  : '-' atom
      | INT
      ;
```

… wird eine Grammatik mit linksrekursiven Alternativen, wo sich der Vorrang über die Reihenfolge der linksrekursiven Alternativen bestimmt (von oben nach unten):

```
expr  : expr '*' expr
      | expr '+' expr
      | '-' atom
      | INT
      ;
```

Unter den beiden linksrekursiven Alternativen `expr '*'expr` und `expr '+'expr` hat die erste Alternative die höhere Priorität und würde beim Parsen wie gewünscht weiter unten im Baum landen.

Beide Varianten sind machbar. Entscheiden Sie selbst, was (für Sie) lesbarer ist.

Caution

5.3.8.1. Indirekte Linksrekursion nicht erlaubt

Mit **indirekter** Linksrekursion kann ANTLR (v4) **nicht** umgehen:

```
1 expr : expM | ... ;
2 expM : expr '*' expr ;
```

Hier würden sich die Regeln `expM` und `expr` gegenseitig aufrufen und kein Token aus dem Tokenstrom entfernen, so dass der generierte LL-Parser hier in einer Endlosschleife stecken bleiben würde. Mit indirekter Linksrekursion kann ANTLR nicht umgehen.
Indirekte Linksrekursion ist nicht erlaubt!

5.3.8.2. Assoziativität

Per Default werden Operatoren wie + von ANTLR automatisch als **linksassoziativ** behandelt, da bei Mehrdeutigkeiten in der Grammatik stets die erste passende Regel/Alternative genommen wird. Dadurch wird die übliche Klammerung von + und * berücksichtigt, d.h. `1+2+3` wird mit der gezeigten Grammatik wie erwartet als `(1+2)+3` geparst.

Die Eingabe `2^3^4` sollte aber als `2^(3^4)` geparst werden. Analog sollte `a=b=c` in C/C++ als `a=(b=c)` verstanden werden. Für *rechts-assoziative* Operatoren muss man ANTLR dies explizit mit `<assoc=right>` in der Grammatik mitteilen:

```
1 expr : <assoc=right> expr '^' expr
2       | INT
3       ;
```

5.3.9. Ausblick

Damit haben wir die sprichwörtliche “Spitze des Eisbergs” gesehen. Mit ANTLR sind noch viele weitere Dinge möglich. Bitte nutzen Sie aktiv die Dokumentation auf github.com/antlr/antlr4.

5.3.10. Wrap-up

- Lexer-Regeln beginnen mit **Großbuchstaben**
  - Längster Match gewinnt, bei Gleichstand: zuerst definierte Regel
  - Implizite Tokens (`'...'`) haben Vorrang vor expliziten Lexer-Regeln
- Parser-Regeln beginnen mit **Kleinbuchstaben**
  - Regeln können Token-Typen (Lexer-Regeln oder `'...'`) und andere Parser-Regeln kombinieren
  - Regeln können Alternativen haben (|) sowie optionale/mehrfache Teile (?, *, +)
  - Bei Mehrdeutigkeit: Vorrang für die erste passende Alternative
  - Startregel + `EOF` erzwingt, dass der **gesamte** Input konsumiert wird

  - ANTLR4 erlaubt **direkte** Linksrekursion (indirekte nicht)

- Vorrang und Assoziativität für Ausdrücke in ANTLR:
  - Gestufte Grammatik-Regeln (`expr` / `term` / `atom`) kodieren Präzedenz strukturell
  - Linksrekursive Regel mit geordneter Alternativenliste (Reihenfolge der linksrekursiven Alternativen steuert den Vorrang der Operatoren)
  - Linksassoziativität ist Standard; Rechtsassoziativität kann mit `<assoc=right>` deklariert werden

Zum Nachlesen

Sie finden auf der ANTLR-Homepage Links zur Dokumentation und auch zum ANTLR-Projekt auf GitHub. Eine ältere, aber dennoch interessante Quelle vom "Kopf" hinter ANTLR (Terence Parr) ist (Parr 2014), die auch online verfügbar ist.
Die ANTLR-Dokumentation und die ANTLR-FAQ bieten ebenfalls interessante Einstiegspunkte zur Vertiefung für Fortgeschrittene.

Lernziele

- k2: Ich kann den grundsätzlichen Aufbau einer ANTLR-Grammatik (Lexer-Teil, Parser-Teil, Startregel) erklären.
- k2: Ich kann das Verhalten des ANTLR-Lexers bzgl. längstem Match, Regelreihenfolge und impliziten Tokens beschreiben.
- k3: Ich kann eine gegebene EBNF-Grammatik für arithmetische Ausdrücke in eine äquivalente ANTLR-Grammatik übersetzen.
- k3: Ich kann in ANTLR Parser-Regeln mit Alternativen sowie optionalen und wiederholten Teilen formulieren.
- k3: Ich kann Operatorpräzedenz und Linksassoziativität entweder über gestufte Regeln (`expr`/`term`/`atom`) oder über eine linksrekursive `expr`-Regel in einer ANTLR-Grammatik umsetzen.
- k4: Ich kann für eine gegebene ANTLR-Grammatik begründen, wie eine konkrete Eingabe (z.B. `2+3*4`) geparst wird, und den entstehenden Parse-Tree grob skizzieren.

Challenges

Von EBNF zu ANTLR mit impliziten Tokens
Gegeben sei die folgende EBNF-Grammatik für einfache logische Ausdrücke mit C-ähnlicher Präzedenz (`not` bindet am stärksten):

```
1  expr   ::= disj
2  disj   ::= conj ('or' conj)*
3  conj   ::= atom ('and' atom)*
4  atom   ::= 'not' atom
5           | '(' expr ')'
6           | ID
```

Dabei soll eine `ID` mit einem kleinen Buchstaben beginnen und danach beliebig viele kleine/große Buchstaben und Ziffern beinhalten. Eine ID besteht dabei aus mindestens einem Zeichen.
Übersetzen Sie diese Grammatik in eine ANTLR-Grammatik (`Logic.g4`) mit:

- einer Startregel `expr`
- impliziten Tokens für die Schlüsselwörter und Klammern
- einer expliziten Lexer-Regel für `ID`,
- einer Lexer-Regel für Whitespace; dieser soll verworfen werden

Präzedenzfehler finden und korrigieren

Betrachten Sie die folgende ANTLR-Grammatik für arithmetische Ausdrücke:

```
grammar ArithWrong;

expr : term (('+' | '-') term)* ;
term : atom (('*' | '/') atom)* ;
atom : '-' expr
     | INT
     | '(' expr ')'
     ;
```

mit `INT` ein oder mehrere Ziffern (wie üblich).

1. Überlegen Sie, wie der Ausdruck `-1*2` mit dieser Grammatik geparst wird. Skizzieren Sie grob den Parse-Tree (nur mit den Regeln `expr`, `term`, `atom`).
2. Diskutieren Sie: Entspricht das der üblichen Bedeutung von `-1*2` in C/Java?
3. Ändern Sie nur die Definition von `atom` so, dass das unäre Minus mit höherer Präzedenz als `*` und `/` behandelt wird (und der Rest weiterhin funktioniert).

Implizite Tokens und Lexer-Priorität

Gegeben sei ein Ausschnitt einer ANTLR-Grammatik:

```
grammar Mini;

start : 'if' '(' ID ')' EOF ;
```

Dabei soll eine `ID` mit einem kleinen Buchstaben beginnen und danach beliebig viele kleine/große Buchstaben und Ziffern beinhalten. Eine ID besteht dabei aus mindestens einem Zeichen.

1. Welche Tokenfolge erzeugt der Lexer für die Eingabe `if(x)`?
2. Diskutieren Sie: Was würde passieren, wenn Sie statt `'if'` in der Startregel eine explizite Lexer-Regel `IF : 'if';` und die Startregel als `start : IF '('ID ')'EOF ;` schreiben würden?
3. Variieren Sie nun die Reihenfolge der Lexer-Regeln: `antlr ID : [a-z][a-zA-Z0-9]* ; IF : 'if';` Diskutieren Sie:
   - Welche Tokenfolge erzeugt der Lexer nun für die Eingabe `if(x)`?
   - Warum macht es aus Sicht der ANTLR-Spezifikation trotzdem einen Unterschied, ob Sie `'if'` als implizites Token oder als explizite Lexer-Regel schreiben?

Linksrekursion und Vorrang

Betrachten Sie die folgende typische ANTLR-Grammatik für Expressions:

```
program : expr EOF ;

expr
  : expr ('*'|'/') expr
  | expr ('+'|'-') expr
  | INT
  ;
```

In der gezeigten Schreibweise wird implizit ein Vorrang unter den Alternativen dieser direkt links-rekursiven Regel `expr` definiert und die Eingabe `2+3*4` würde korrekt als `2+(3*4)` geparst.
Was passiert, wenn man nun versucht, die Operatoren auszuklammern und in eine eigene Regel zu verlagern?
Betrachten Sie die folgende leicht geänderte Grammatik:

```
program : expr_ EOF ;

expr_
  : expr_ operator expr_
  | INT
  ;
operator
  : ('*'|'/')
  | ('+'|'-')
  ;
```

Was passiert hier bei der Eingabe von `2+3*4`?

5.4. L-Expr: ANTLR-Visitoren und Rückgabewerte

TL;DR

ANTLR erzeugt aus der Eingabe auf Basis der Grammatik einen Parse-Tree: Token sind die Blätter, jede angewendete Parser-Regel liefert einen inneren Knoten. Dieser Parse-Tree ist noch kein AST, eignet sich aber als Grundlage, um daraus einen AST zu konstruieren.
Für die Verarbeitung des Parse-Tree stellt ANTLR generierte Visitor-Klassen bereit. Die Traversierung steuern wir beim Visitor selbst: Die geerbte Methode `visit()` übernimmt nur den Dispatch und ruft je nach Knotentyp die passende `visitRegelName()`-Methode auf. Dort müssen wir explizit entscheiden, welche Kindknoten besucht werden und wie aus deren Ergebnissen ein AST-Knoten aufgebaut wird.
ANTLR übergibt an jede `visitRegelName()`-Methode ein Kontextobjekt `RegelNameContext`, das den Zugriff auf die Bestandteile der jeweiligen Regel erlaubt (z.B. weitere Kontexte oder Token über Methoden wie `ctx.eform()` oder `ctx.INT()`). Benannte Alternativen (`#label`) und Regel-Elemente mit Namen (`x=expr`) sind hilfreich, weil sie zu eigenen Kontextklassen `LabelContext` bzw. Zugriffsmethoden `visitLabel()` führen und so den Visitor-Code übersichtlicher machen.
Der generische Parameter `<T>` der Visitor-Klassen legt den Rückgabetyp aller `visitXYZ()`-Methoden fest (z.B. `Expr` für einen AST-Builder oder `Object` für flexibel erweiterbare Builder-Hierarchien). Die Default-Implementierung in `BaseVisitor` ruft die vordefinierte Methode `visitChildren(ctx)` auf, das alle Kinder besucht und typischerweise das Ergebnis des letzten Kindes zurückgibt.

> Für Regeln mit eigener Semantik (Operatoren, Kontrollstrukturen, Klammern entfernen, …) müssen wir `visitXYZ()` überschreiben und selbst festlegen, wie die Ergebnisse der Kindknoten kombiniert werden. Nur bei reinen Durchleitungsregeln reicht meist die Default-Implementierung.

5.4.1. Erinnerung: ANTLR-Toolchain: Von String zum Parse-Tree

```
var source = "(+ 1 2)";

var input = CharStreams.fromString(source);
var lexer = new LispyLexer(input);
var tokens = new CommonTokenStream(lexer);

var parser = new LispyParser(tokens);
var tree = parser.lexpr();
```

Ergebnis: ein **Parse-Tree**, noch kein AST

5.4.2. ANTLR: Generierte Klassen

- Grammatik: `Lispy.g4`
  - mit Regel `eform : '('+'eform eform+ ')'#addExpr ;`
- ANTLR generiert:
  - `LispyLexer`, `LispyParser`
  - `LispyBaseVisitor<T>` mit

```
T visitAddExpr(LispyParser.AddExprContext ctx) {
    return visitChildren(ctx);
}
```

Wir haben bereits in Prog2 darüber gesprochen: Einführung in ANTLR

Hier nur kurz zur Erinnerung: ANTLR generiert uns aus der Grammatik `Lispy.g4` einen Visitor `LispyVisitor<T>` mit einer “leeren” Basisimplementierung `LispyBaseVisitor<T>`. Für jede Regel und benannte Alternative finden wir da eine `visitName()`-Methode, die in der Basisimplementierung einfach nur die Kinder der Kontextklasse (also die Bestandteile der Grammatikregel) rekursiv besucht. Dabei wird das letzte Ergebnis dann zurückgegeben.

Wenn wir unseren AST-Builder schreiben wollen, können wir genau hier ansetzen. Wir leiten von `LispyBaseVisitor<T>` ab und überschreiben alle relevanten `visitName()`-Methoden. Für jede Regel im Parser, die für unsere abstrakte Grammatik relevant ist, überschreiben wir die passende `visit()`-Methode und konstruieren dort den entsprechenden AST-Knoten.

Der generische Parameter `T` sollte dem gewünschten Rückgabetyp der `visitName()`-Methoden entsprechen. In unserem Fall auf der Sprachstufe L-Expr wäre das ein `Expr`. In den nächsten Sprachstufen wollen wir von

unserem AST-Builder aber später erneut ableiten und die neuen Regeln überschreiben und wir werden dann andere/weitere Rückgabetypen benötigen. Deshalb ist `Object` die passende Wahl (auch wenn wir dann explizit casten müssen).

5.4.3. AST-Builder: Beispiel Int-Literal und Addition

```java
public class LispyAstBuilderLexpr extends LispyBaseVisitor<Object> {

    @Override
    public Expr visitLexpr(LexprContext ctx) { return asExpr(visit(ctx.eform())
        ); }

    @Override
    public Expr visitIntLit(IntLitContext ctx) {
        int value = Integer.parseInt(ctx.INT().getText());
        return new IntLiteral(value);
    }

    @Override
    public Expr visitAddExpr(AddExprContext ctx) {
        List<Expr> ops = ctx.eform().stream()
                            .map(this::visit)
                            .map(this::asExpr).toList();
        return ops.stream().reduce(AddExpr::new).orElseThrow();
    }
}
```

Wir leiten von `LispyBaseVisitor` ab und setzen den generischen Typ-Parameter auf `Object`. Wie oben diskutiert, wäre für L-Expr der Typ `Expr` die bessere Wahl - aber wir wollen später von diesem AST-Builder weiter ableiten (`class LispyAstBuilderLvar extends LispyAstBuilderLexpr`) und brauchen noch weitere Rückgabetypen. Deshalb nehmen wir in Kauf, dass wir entsprechend casten müssen. Im obigen Beispiel wurde dazu eine Hilfsmethode `Expr asExpr(Object o)` definiert und eingesetzt. (**Hinweis**: Implementierung von `asExpr()` siehe Challenges.)

Für jede Regel und für jedes Label haben wir eine `visitName()`-Methode in `LispyBaseVisitor`. Wir überschreiben die für uns relevanten Methoden, um den AST zu bauen.

Zu jeder Regel und jedem Label wird ein passendes Kontextobjekt generiert, mit dem man die Bestandteile der Regel abfragen kann. Zur Regel `s : expr ;` würde also ein `SContext`-Objekt erzeugt bzw. der `visitS()`-Methode übergeben, und man könnte darauf über `expr()` den `ExprContext` abfragen. Ein `eform : '('+eform eform+ ')'#addExpr ;` generiert analog zum Label `addExpr` eine `visitAddExpr()`-Methode und eine Klasse `AddExprContext`.

Wir betrachten im Folgenden exemplarisch das Behandeln der Startregel, der Addition und der Regel für Integer-Literale.

5.4.3.1. Start: visitLexpr

Wir hatten die Startregel `lexpr : eform EOF ;`. In der zugehörigen Methode `visitLexpr()` bekommen wir den Kontext dieser Regel `LexprContext`. Auf das Element `eform` können wir hier mit `ctx.eform()` zugreifen.

Um die rekursive Traversierung anzustoßen, rufen wir die geerbte Methode `visit()` auf dieser `eform` auf und liefern das Ergebnis als `Expr` zurück. Die geerbte Methode `visit()` erledigt den Dispatch für die Parse-Tree-Knoten und sorgt dafür, dass jeweils die richtige `visitXYZ()`-Methode aufgerufen wird.

Tip

5.4.3.2. Stilfrage: Kovariante Rückgabetypen

Eigentlich ist wg. `LispyBaseVisitor<Object>` der Rückgabetyp aller `visitXYZ()`-Methoden ebenfalls `Object`. Wir liefern hier in der ableitenden Klasse `LispyAstBuilderLexpr` mit `Expr` einen spezialisierteren Typ (`Expr` leitet implizit von `Object` ab) zurück, und Java erlaubt uns, dies auch in der Signatur der Methode kenntlich zu machen. Das nennt man auch "**kovariante Rückgabetypen**".
Hinweis: Wir könnten andersherum natürlich auch eine gemeinsame Oberklasse für alle (auch zukünftige) AST-Knoten definieren und diese hier statt `Object` als generischen Typparameter und damit für die Rückgabetypen nutzen. Letztlich müssten wir dann aber ebenfalls immer wieder nach dem dynamischen Typ prüfen …

5.4.3.3. Blätter: Int-Literale

In `visitIntLit` bekommen wir ein `IntLitContext` für die Regel `atom : INT #intLit ;` (also für das Label).

Das einzige Kind in dieser Regel ist das Token `INT`, welches wir über `ctx.INT()` abfragen und das Lexem mit `getText()` holen. Dann müssen wir nur noch einen Java-Integer daraus machen, um ein neues `IntLiteral` für unseren AST erzeugen zu können.

Eine weitere Traversierung ist hier nicht notwendig, die Int-Literale bilden die Blätter im Parse-Tree.

5.4.3.4. Zwischenknoten: Addition

Für die Addition überschreiben wir die Methode `visitAddExpr`. Die Regel war `eform : '(''+'eform eform+ ')'#addExpr ;`, weshalb im `AddExprContext` die Zugriffsmethode `eform()` vorhanden ist - allerdings haben wir mehrere `eform`-Kinder in dieser Regel, entsprechend liefert `eform()` eine `List<EformContext>`.

Was müssen wir nun tun? Wir wollen für immer zwei Summanden ein neues `AddExpr` als AST-Knoten erzeugen (d.h. von links klammern). Entsprechend machen wir eine Schleife über alle `eform`-Kinder der Regel/des Parse-Tree-Knotens, stoßen für diese jeweils die rekursive Verarbeitung mit `visit()` an und bekommen so für jedes Kind einen AST, der in der Ergebnisliste `List<Expr>` gesammelt wird. In einem zweiten Schritt iterieren wir über diese Liste und erzeugen von links her immer aus dem letzten `AddExpr` und dem aktuellen Eintrag ein

neues `AddExpr` (dabei ruft `AddExpr::new` den Konstruktor `AddExpr(Expr left, Expr right)` auf) - dies entspricht in der funktionalen Programmierung einer `reduce`-Operation. Im ersten Schritt haben wir noch kein voriges `AddExpr` und würden entsprechend die ersten beiden Listeneinträge in ein `AddExpr` verpacken. Falls es keine Kinder gegeben haben sollte, würden wir hier eine Exception auslösen.

Caution

5.4.3.5. `visit()` vs. `visitChildren()` vs. `visitXYZ()`

Innerhalb unserer überschriebenen `visitXYZ()`-Methoden sollten wir für das Traversieren von Kindknoten in der Regel **immer `visit(...)` verwenden**.

- `visit(node)` ist der zentrale Einstiegspunkt des Visitor-Patterns: Es führt den dynamischen Dispatch aus und ruft je nach tatsächlichem Knotentyp automatisch die passende `visitXYZ()`-Methode für das Argument `node` auf.
- `visitChildren(ctx)` ist eine **fertige Default-Strategie**: Sie besucht *alle* **Kinder** des Knotens nacheinander und kombiniert deren Rückgabewerte (in der Standardimplementierung meist: "nimm das letzte Ergebnis"). Diese Methode ist dafür gedacht, in der Basisklasse als Default zu dienen - nicht als allgemeiner Ersatz für gezielte Kind-Verarbeitung in unseren eigenen `visitXYZ()`-Methoden.
- Ein **direkter Aufruf** von konkreten Methoden wie `visitAddExpr(ctx)` innerhalb anderer `visitXYZ()`-Methoden ist unflexibel: Er umgeht den normalen Dispatch-Mechanismus und koppelt den Code unnötig eng an konkrete Kontextklassen. Spätere Erweiterungen oder Umbenennungen der Grammatik werden so schwerer.

Faustregel: Für rekursive Aufrufe auf Kindknoten immer **`visit(kind)`** benutzen.

5.4.4. ANTLR-Pipeline komplett

```
var source = "(+ 1 2)";

var input = CharStreams.fromString(source);
var lexer = new LispyLexer(input);
var tokens = new CommonTokenStream(lexer);

var parser = new LispyParser(tokens);
var tree = parser.lexpr();

var builder = new LispyAstBuilderLexpr();
var ast = builder.visitLexpr(tree);
```

Ab hier sind wir jetzt synchron zu unserem AST, den wir manuell über den handgeschriebenen Lexer und unseren RD-Parser erzeugt haben! D.h. wir können uns spontan aussuchen, welche dieser beiden Varianten wir nutzen möchten …

Für die Exploration von Sprachen und schrittweise wachsende Grammatiken ist das Arbeiten mit ANTLR sehr angenehm, weil man nur die Grammatik und den AST-Builder anpassen muss.

Sobald die Sprache allerdings fix ist, gehen viele Projekte dazu über, ihre Lexer und Parser selbst zu implementieren.

5.4.5. Default-Implementierung: Wann reichen die Default-`visitXYZ()`-Methoden?

Default in `LispyBaseVisitor<T>`:

```
1 T visitAddExpr(AddExprContext ctx) { return visitChildren(ctx); }
```

Hinweis: Eigener Code, der die in ANTLR enthaltene/generierte Methode `visitChildren()` in der Wirkung zusammenfasst. (Der gezeigte Code ist stark vereinfacht, die tatsächliche ANTLR-Implementierung ist generischer und arbeitet u.a. mit `defaultResult()`/`aggregateResult()`.)

```
1 public T visitChildren(RuleNode node) {
2     return node.childs().stream()
3             .map(c -> c.accept(this))
4             .reduce((left, right) -> right).orElse(null);
5 }
```

Die Methode `visitChildren()` wird in der Defaultimplementierung in `LispyBaseVisitor` für alle `visitXYZ()`-Methoden aufgerufen. Sie geht über alle Kinder des Knotens (Bestandteile der Grammatik-Regel), stößt jeweils die rekursive Verarbeitung an und liefert das **letzte** Ergebnis (bzw. `null`) zurück.

Das ist einerseits sehr angenehm: Wir müssen nur noch diejenigen `visitXYZ()`-Methoden überschreiben, wo wir Dinge aktiv bearbeiten wollen. Sobald uns die rekursive Traversierung und das Ergebnis des letzten Kindes reicht, müssen wir diese `visitXYZ()`-Methode nicht überschreiben.

Andererseits ist dies auch gefährlich. Das letzte Kind kann ein Token sein. Die "Traversierung" eines Tokens liefert oft nur `null` - und das wäre dann das Gesamtergebnis für den jeweiligen Knoten.

Konsequenz:

- **Überschreiben**, wenn:
  - Semantik / Auswertung benötigt: Beispiel `visitAddExpr()`

    ```
    1 @Override
    2 public Expr visitAddExpr(AddExprContext ctx) {
    3     List<Expr> ops = ctx.eform().stream().map(this::visit).map(this::
          asExpr).toList();
    4     return ops.stream().reduce(AddExpr::new).orElseThrow();
    5 }
    ```

  - Standardlogik falsch: `parens : '('expr ')';`

    ```
    1 @Override
    2 public T visitParens(ParensContext ctx) {
    3     return visit(ctx.expr());  // ignore parenthesis
    4 }
    ```

- **Defaultverhalten** reicht, wenn
  - die Regel nur "durchgereicht" werden soll

    ```
    1 expr    : addExpr ;
    2 addExpr : multExpr (('+' | '-') multExpr)* ;
    ```

Wenn wir hier `visitMultExpr()` und `visitAddExpr()` implementiert haben, reicht für `visitExpr()` das Defaultverhalten.

Das gilt auch, wenn eine Regel wie `expr` einfach nur mehrere Alternativen auflistet: `expr : foo | bar | wuppie | fluppie ;` und wir im Visitor einfach nur die passende Sub-Regel besuchen wollen, ohne für `expr` noch zusätzliche Verarbeitung zu benötigen.

5.4.6. Wrap-up

- ANTLR-Visitoren sind generisch: `<T>` → Rückgabetyp aller `visitXYZ()`
- `BaseVisitor` implementiert `visitChildren()` als Default: Besuch aller Kinder, meist Rückgabe des letzten Kind-Ergebnisses
- Kontextobjekte (`...Context`) kapseln Bestandteile einer Grammatikregel
- Benannte Alternativen (`#label`) erzeugen eigene Kontextklassen bzw. Zugriffsmethoden

- Faustregel:
  - Für **semantisch relevante** Regeln `visitXYZ()` überschreiben (Operatoren, Kontrollstrukturen, unäres Minus, …)
  - Für reine **Durchleitungs-/Klammer-Regeln** kann der Default genügen

Zum Nachlesen

Sie finden auf der ANTLR-Homepage Links zur Dokumentation und auch zum ANTLR-Projekt auf GitHub. Eine ältere, aber dennoch interessante Quelle vom "Kopf" hinter ANTLR (Terence Parr) ist (Parr 2014), die auch online verfügbar ist.
Die ANTLR-Dokumentation und die ANTLR-FAQ bieten ebenfalls interessante Einstiegspunkte zur Vertiefung für Fortgeschrittene.

Lernziele

- k2: Ich kann den Unterschied zwischen Parse-Tree und AST in eigenen Worten erklären.
- k2: Ich kann erläutern, wie ANTLR-Kontextobjekte (`...Context`) aus Parser-Regeln entstehen und wie ich damit auf Regelbestandteile zugreife.
- k2: Ich kann erklären, wie der generische Rückgabetyp `<T>` der ANTLR-Visitoren die Signatur der `visitXYZ()`-Methoden bestimmt.
- k3: Ich kann für eine gegebene ANTLR-Grammatik geeignete benannte Alternativen und Regel-Elemente definieren, um den späteren Visitor-Code zu vereinfachen.
- k3: Ich kann einen AST-Builder als ANTLR-Visitor implementieren, der aus einem Parse-Tree einen typisierten AST erzeugt.
- k3: Ich kann entscheiden, für welche Regeln die Default-Implementierung `visitChildren()` ausreicht und für welche Regeln ich `visitXYZ()` überschreiben muss, und dies am Beispiel begründen.
- k4: Ich kann eine gegebene Visitor-Implementierung kritisch analysieren und Verbesserungsvorschläge bezüglich Struktur, Wiederverwendung und Typisierung machen.

Challenges

Casten im AST-Builder

Im Beispiel in der Vorlesung wurde die Hilfsmethode `Expr asExpr(Object o)` definiert, um nicht manuell casten zu müssen. Wie sieht die passende Implementierung in Java dazu aus?

Kontextobjekte und benannte Alternativen nutzen

Betrachten Sie die beiden Varianten der Grammatik für `eform`:

- Variante A:

```
eform
  : '(' '+' eform eform+ ')'
  | '(' '*' eform eform+ ')'
  | INT
  ;
```

- Variante B:

```
eform
  : '(' '+' eform eform+ ')'  # addExpr
  | '(' '*' eform eform+ ')'  # mulExpr
  | INT                       # intLit
  ;
```

1. Überlegen Sie, welche Kontextklassen und `visitXYZ()`-Methoden ANTLR bei Variante A bzw. B generiert (nur qualitativ, keine exakte Namensliste nötig).
2. Skizzieren Sie, wie sich der AST-Builder für L-Expr in Variante A vs. Variante B unterscheidet:
   - Wo wird der Code einfacher / übersichtlicher?
   - Wo müssten Sie in Variante A zusätzliche **`instanceof`**-Checks oder **`if`**/**`switch`**-Logik einbauen?
3. Formulieren Sie eine kurze "Daumenregel", wann sich benannte Alternativen in einer Grammatik lohnen.

Wann reicht `visitChildren()`?

Gegeben sei der folgende Ausschnitt einer (vereinfachten) Grammatik für arithmetische Ausdrücke:

```
expr     : addExpr ;
addExpr  : multExpr (('+' | '-') multExpr)* ;
multExpr : atom (('*' | '/') atom)* ;
atom     : INT
         | '(' expr ')'   # parens
         ;
```

1. Markieren Sie für jede Regel (`expr`, `addExpr`, `multExpr`, `atom`, `parens`), ob die Default-Implementierung `visitChildren()` für einen AST-Builder ausreicht oder ob Sie `visitXYZ()` überschreiben würden.
2. Begründen Sie Ihre Entscheidungen jeweils in ein bis zwei Sätzen.
3. Diskutieren Sie in der Gruppe, welche typischen Fehler entstehen, wenn man sich zu sehr auf `visitChildren()` verlässt.

Rückgabetyp wählen: `Expr` vs. `Object`

Sie implementieren einen AST-Builder für L-Expr als ANTLR-Visitor:

```
public class LispyAstBuilderLexpr extends LispyBaseVisitor<Object> {
    // ...
}
```

1. Diskutieren Sie in der Gruppe die Vor- und Nachteile der Wahl von `Object` als generischem Visitor-Typ gegenüber einem spezifischen Typ wie `Expr`.
2. Überlegen Sie, wie sich Ihre Entscheidung auf folgende Szenarien auswirkt:
   - Sie wollen in der nächsten Sprachstufe L-Var neue AST-Knotentypen einführen und den Builder vererben.
   - Sie möchten zusätzlich zu Ausdrücken auch ganze Programme oder Funktionsdeklarationen als AST-Knoten zurückgeben.

Lexer und Parser mit ANTLR

Betrachten Sie den folgenden Programmschnipsel:

```
result99 = acc_2*ACC_2 + spillover7 + bonus_1*3 + inc_0;
calc_42 = __9 * zZ_1 + 5 + FooBar_42 * bar_7 + q0;
_ExprLine + A_1 * bB_2 + cc3 * 7 +      11;
```

Erstellen Sie für diese fiktive Sprache einen Lexer+Parser mit ANTLR. Implementieren Sie mit Hilfe des Parse-Trees und der Visitoren einen einfachen Pretty-Printer.

6. Sprache L-Var: Variablen, Statments, nested Scopes

6.0.1. Inhalt

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- Nested Scopes, Resolving
- ResolverLVar, InterpreterLVar

7. Sprache L-If: Datentyp Boolean, Vergleiche, Kontrollstrukturen (if/else, while)

7.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- ResolverLIf, TypeCheckerLIf, InterpreterLIf
- Nano-Pass mit Desugaring: for, +=, … (??)

8. Sprache L-Fun: Funktionen (Definition, Aufruf)

8.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- ResolverLFun, TypeCheckerLFun, InterpreterLFun
- native Funktionen vs. geparste Funktionen

9. Sprache L-Class: Klassen (Felder, Methoden, Objekte)

9.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- Klassen als Environment für Methoden, Instanzen als Environment für Felder, Auflöse-Strategien
- ResolverLClass, TypeCheckerLClass, InterpreterLClass

10. Sprache L-Inherit: Einfachvererbung und dynamischer Dispatch

10.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- ResolverLInherit, TypeCheckerLInherit, InterpreterLInherit

11. Sprache L-Self: native Klassen (inkl. Vererbung)

11.0.1. Inhalte

- Einbinden von nativen Klassen
- InterpreterLSelf, TypeCheckerLSelf/ResolverLSelf
- Vorbereitung für Snake

Teil III.

Praktikum

Hier finden Sie die Übungsblätter.

12. Blatt 01: Reguläre Sprachen

12.1. A1.1: Sprachen von regulären Ausdrücken

Welche Sprache wird von dem folgenden regulären Ausdruck beschrieben?

$a + a\,(a + b)^*\,a$

12.2. A1.2: Bezeichner in Programmiersprachen

Die Referenz einer Programmiersprache beschreibt zugelassene Bezeichner (= Namen für Variablen, Funktionen, Klassen, Methoden, …) folgendendermaßen:

- Alle Variablennamen beginnen mit **V** oder **v**
- Handelt es sich um globale Variablen, beginnen Sie mit **V**, lokale beginnen mit **v**
- Funktions- und Methodenparameter beginnen mit **p**, KLassenparameter (bei der Definition von Vererbung) beginnen mit **P**
- Weitere Bezeichner müssen mit einem Buchstaben (a-z, A-Z) beginnen
- Die folgenden Zeichen dürfen Buchstaben, Ziffern und ein Unterstrich sein
- Bezeichner dürfen nicht mit einem Unterstrich enden
- Alle Bezeichner müssen aus mindestens zwei Zeichen bestehen

Entwickeln Sie **einen** regulären Ausdruck, der den Aufbau aller Bezeichner beschreibt.

Beachten Sie, dass Ihr regex alle zulässigen Bezeichner beschreiben muss, aber keinen einzigen unzulässigen beschreiben darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie, wie sie vom regex gematcht werden.

Entwickeln Sie einen DFA, der diese Bezeichner akzeptiert. Beachten Sie, dass Ihr DFA alle zulässigen Bezeichner akzeptieren muss, aber keinen einzigen unzulässigen akzeptieren darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie, wie sie vom Automaten zeichenweise gelesen und akzeptiert werden.

Bitte beachten Sie, dass die Schreibweise a-z nicht unserer Definition von DFAs genügt. Eigentlich müsste jedes Zeichen aufgeführt werden.

Anmerkung: Diese Darstellung wird ab jetzt akzeptiert.

Entwickeln Sie eine reguläre Grammatik, die diese Bezeichner generiert. Beachten Sie, dass Ihre Grammatik alle zulässigen Bezeichner generieren können muss, aber keinen einzigen unzulässigen generieren darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie die Ableitungsbäume dazu.

12.3. A1.3: Gleitkommazahlen in Programmiersprachen

Recherchieren Sie zunächst den Aufbau von Gleitkommazahlen in Python und Java.

Erstellen Sie für jede der beiden Programmiersprachen reguläre Ausdrücke, DFAs und reguläre Grammatiken wie in Aufgabe A1.2. Verifizieren Sie Ihre Lösungen wie in Aufgabe A1.2. Vorgaben, die sich auf Längen oder Werte von Teilen der Zahlen beziehen, ignorieren Sie bitte.

12.4. A1.4: Mailadressen?

Warum ist der folgende regex ungeeignet für die Verarbeitung von Mailadressen?

$(a-z)^+@(a-z).(a-z)$

Verbessern Sie den gegebenen regulären Ausdruck.

12.5. A1.5: Der zweitletzte Buchstabe

Entwickeln Sie einen DFA, der nur Wörter über $\Sigma = \{1, 2\}$ akzeptiert, deren zweites und zweitletztes Zeichen eine **1** ist.

12.6. A1.6: Sprache einer regulären Grammatik

Welche Sprache generiert die folgende Grammatik?

$$\begin{aligned} S &\rightarrow aA \mid bB \\ A &\rightarrow bS \mid c \\ B &\rightarrow aS \mid b \end{aligned}$$

Können Sie einen regulären Ausdruck oder einen DFA dafür angeben?

13. Blatt 02: CFG

13.1. A2.1: Akzeptierte Sprache

Ist der folgenden PDA deterministisch? Warum (nicht)?

q_4 sei der akzeptierende Zustand.

$$
\begin{aligned}
\delta(q_0, a, \bot) &= (q_0, A\ \bot) \\
\delta(q_0, a, A) &= (q_0, AA) \\
\delta(q_0, b, A) &= (q_1, BA) \\
\delta(q_1, b, B) &= (q_1, BB) \\
\delta(q_1, c, B) &= (q_2, \epsilon) \\
\delta(q_2, c, B) &= (q_2, \epsilon) \\
\delta(q_2, d, A) &= (q_3, \epsilon) \\
\delta(q_3, d, A) &= (q_3, \epsilon) \\
\delta(q_3, d, A) &= (q_3, AA) \\
\delta(q_3, \epsilon, \bot) &= (q_4, \epsilon)
\end{aligned}
$$

Zeichnen Sie den Automaten. Geben Sie das 7-Tupel des PDa an. Welche Sprache akzeptiert er?

13.2. A2.2: Kontextfreie Sprache

Welche Sprache generiert die folgende kontextfreie (Teil-) Grammatik?

$$G = (\{\text{Statement}, \text{Condition}, \ldots\}, \{\text{"if"}, \text{"else"}, \ldots\}, P, \text{Statement})$$

mit

$$
\begin{aligned}
P = \{ & \\
& \text{Statement} \quad \rightarrow \text{"if" Condition Statement} \mid \text{"if" Condition Statement "else" Statement} \\
& \text{Condition} \quad \rightarrow \ldots \\
\} &
\end{aligned}
$$

Ist die Grammatik mehrdeutig? Warum (nicht)?

13.3. A2.3: Kontextfreie Grammatik

Entwickeln Sie eine kontextfreie Grammatik für die Sprache

$$L = \{a^i b^j c^k \mid i = j \vee j = k\}$$

Zeigen Sie, dass die Grammatik mehrdeutig ist. Entwickeln Sie einen PDA für diese Sprache.

13.4. A2.4: PDA

Erstellen Sie einen deterministischen PDA, der die Sprache

$$L = \{w \in \{a, b\}^* \mid w \text{ hat genauso viele a's wie b's}\}$$

akzeptiert.

Beschreiben Sie Schritt für Schritt, wie der PDA die Eingaben *baba* und *bba* abarbeitet.

14. Blatt 03: L-Int und C-Int (Integer, Addition, Subtraktion)

> **Important**
>
> Für die erfolgreiche Bearbeitung dieses Übungsblattes müssen mindestens alle mit * und ** markierten Aufgaben gelöst werden.
> Die Aufgaben mit *** sind als Vertiefung gedacht.

Dieses Blatt knüpft an folgende Vorlesungseinheiten an:

- L-Int: Die Sprache Lispy
- L-Int: Konkrete und abstrakte Syntax
- L-Int: Lexer - Handcodierte Implementierung
- L-Int: LL-Parser mit Recursive Descent
- L-Int: Von Ausdrücken zum Interpreter

14.1. A3.1: L-Int: Subtraktion verstehen und ergänzen (*)

L-Int unterstützt Integer-Literale, Addition und Subtraktion. Im Skript wurde die Implementierung nur für Integer und Addition im Detail gezeigt.

14.1.1. 1. Sprache und Semantik nachvollziehen

- Welche Eingaben sind **gültige** L-Int-Ausdrücke?
- Wie werden mehrere Operanden einer Addition oder Subtraktion gruppiert?
- Welche Werte ergeben die folgenden Ausdrücke?

```
(+ 1 2)
(- 8 3)
(- 20 5 3)
(- 10 (+ 3 5))
```

Notieren Sie mindestens zwei **ungültige** L-Int-Eingaben. Mindestens eine davon soll eine Operation mit weniger als zwei Operanden sein.

14.1.2. 2. Auswertung der Subtraktion skizzieren

Skizzieren Sie in Java, wie die Auswertung einer `SubExpr` aussehen könnte. Geben Sie dabei an, welche Teil-Ausdrücke ausgewertet und wie ihre Ergebnisse kombiniert werden.

14.2. A3.2: L-Int: Funktion `skipIrrelevant()` im Lexer ergänzen (**)

Im Skript wurde die Funktion `skipIrrelevant()` eingeführt. Sie überspringt nicht benötigte Zeichen wie Whitespace und Kommentare und wird zu Beginn von `nextToken()` aufgerufen:

```
protected boolean skipIrrelevant() {}  // Homework
```

Gegeben: Betrachten Sie den folgenden Code-Rahmen für diese Aufgabe und erstellen Sie damit ein neues Java-Projekt.

```
public enum TokenKind { PLUS, MINUS, LPAREN, RPAREN, SEMI, INT,  EOF }
public record Token(TokenKind kind, String lexeme) {}

public class LispyLexerLint {
    protected String source;
    protected int current;

    public List<Token> scan(String input) {
        source = input;  current = 0;
        List<Token> tokens = new ArrayList<>();

        do tokens.add(nextToken());
        while (tokens.getLast().kind() != EOF);

        return tokens;
    }

    protected Token nextToken() {
        skipIrrelevant();

        if (!isAtEnd()) {
            char c = advance();

            if (Character.isDigit(c)) {
                String lexeme = c + readWhile(Character::isDigit);
                return Token.from(INT, lexeme);
            }
            return switch (c) {
                case '(' -> Token.from(LPAREN);
                case '+' -> Token.from(PLUS);
                default -> throw new RuntimeException("unexpected character");
            };
        }
        return Token.from(EOF);
    }

    protected boolean isAtEnd() { return current >= source.length(); }

```

```
39      protected char peek() { return peek(0); }
40      protected char peek(int la) {
41          if (la < 0 || current + la >= source.length()) return '\0';
42          return source.charAt(current + la);
43      }
44
45      protected char advance() { return isAtEnd() ? '\0' : source.charAt(current
            ++); }
46
47      protected boolean skipIrrelevant() {}  // Homework
48
49      protected String readWhile(Predicate<Character> pred) {
50          StringBuilder sb = new StringBuilder();
51          while (!isAtEnd() && pred.test(peek())) sb.append(advance());
52          return sb.toString();
53      }
54  }
```

1. Zeichnen Sie den DFA für `skipIrrelevant()`.
2. Implementieren Sie `skipIrrelevant()`.
3. Erstellen Sie mindestens vier Testfälle. Berücksichtigen Sie dabei mindestens:
   - eine Eingabe ohne Whitespace oder Kommentare
   - eine Eingabe mit Whitespace vor einem Token
   - eine Eingabe mit einem Kommentar
   - eine Eingabe, die nach Whitespace oder einem Kommentar das Dateiende erreicht

14.3. A3.3: C++ ausprobieren: Compiler und ein erstes Programm (*)

Wir werden in dieser Veranstaltung Schritt für Schritt C++ kennenlernen und einen Interpreter in Java für einen kleinen C++-Ausschnitt (“Mini-C++”) entwickeln.

In dieser Aufgabe richten Sie zunächst einen C++-Compiler ein, schreiben ein kleines Programm und übersetzen sowie starten es mit `clang++` oder `g++`.

14.3.1. Schritt 1: Compiler überprüfen und installieren

Prüfen Sie in einem Terminal, ob ein C++-Compiler verfügbar ist:

```
1  clang++ --version
2  g++ --version
```

Falls kein Compiler gefunden wird:

- Unter **macOS**: Installation der Xcode Command Line Tools.
- Unter **Linux**: Installation von clang++ oder g++ über die Paketverwaltung.
- Unter **Windows**: Nutzen Sie beispielsweise WSL mit Ubuntu und installieren Sie dort `clang++` oder `g++`, oder verwenden Sie eine andere C++-Toolchain.

Tip

Für C++ verwenden wir **clang++** beziehungsweise **g++**, nicht clang beziehungsweise gcc. Umgangssprachlich meint "gcc" in diesem Kontext "g++", und "clang" meint tatsächlich "clang++". Bei der Installation von clang/gcc sind in der Regel beide Varianten (C und C++) dabei.

14.3.2. Schritt 2: Erstes Programm übersetzen und ausführen

Speichern Sie das folgende Programm als `hello.cpp`:

```cpp
#include <iostream>

int main() {
    int result = ((1 + 2) - 3);
    std::cout << "hello world: ";
    std::cout << result << '\n';
    return 0;
}
```

Übersetzen und starten Sie das Programm mit clang++:

```
clang++ hello.cpp -o hello
./hello
```

Oder mit g++:

```
g++ hello.cpp -o hello
./hello
```

Unter Windows heißt die ausführbare Datei meist `hello.exe`. Starten Sie diese in PowerShell mit:

```
.\hello.exe
```

Variieren Sie den arithmetischen Ausdruck in `result` und überprüfen Sie die Ausgabe für mindestens drei Beispiele.

Tip

14.3.3. Mini-Waschzettel: `hello.cpp` im Vergleich zu L-Int / C-Int / Java

Zum Verständnis von `hello.cpp` benötigen Sie hier nur wenige Konstrukte:

Zweck	C++	Java	L-Int	C-Int
Integer-Variable anlegen	`int result = 42;`	`int result = 42;`	n.a.	n.a.
Addition	`1 + 2`	`1 + 2`	`(+ 1 2)`	`1 + 2`
Subtraktion	`7 - 2`	`7 - 2`	`(- 7 2)`	`7 - 2`
Klammerung	`(1 + 2)`	`(1 + 2)`	`(+ 1 2)`	`(1 + 2)`

Ausgabe einer Variablen	`std::cout << result << '\n';`	`IO.println(result)`	n.a.	n.a.
Zeilenkommentar	`// Kommentar`	`// Kommentar`	`;; Kommentar`	`// Kommentar`
Import von Bibliotheken	`#include <iostream>`	`import ...;`	n.a.	n.a.

14.4. A3.4: C-Int: Addition und Subtraktion für Integer in C++ (**)

Wir legen nun mit **C-Int** die Basis für den Mini-C++-Interpreter, den wir über das Semester hinweg erweitern.

C-Int erlaubt analog zu L-Int **Integer-Literale, Addition und Subtraktion**, verwendet aber die in C++ übliche **Infix-Notation**.

Important

14.4.1. Waschzettel: In C++ sieht es so aus ...

Sprachelement	L-Int (Lispy)	C-Int (Mini-C++)
Integer-Literal	`42`	`42`
Addition	`(+ 1 2)`, `(+ 1 2 3 4)`	`1 + 2`, `1 + 2 + 3 + 4`
Subtraktion	`(- 7 2)`, `(- 7 2 1)`	`7 - 2`, `7 - 2 - 1`
Klammerung	`(+ 1 2)`	`(1 + 2)`
Zeilenkommentar	`;; Kommentar`	`// Kommentar`

Beispiele (mit Ergebnis im Kommentar):

```
2           // => 2

1+2         // => 3
(1+2)       // => 3

7-2         // => 5
7-2-1       // => 4

1+2+3       // => 6
(1+2)+3     // => 6
((1+2)+3)   // => 6
```

Ein L-Int-Ausdruck wie `(- (+ 1 2)3)` entspricht in C-Int beispielsweise:

```
((1 + 2) - 3)
(1 + 2) - 3
1 + 2 - 3
```

Addition und Subtraktion sind *linksassoziativ*. Ohne zusätzliche Klammern wird also von links nach rechts gruppiert.
Wichtig: Damit ein C++-Compiler diese Ausdrücke übersetzen kann, müssen sie in einen passenden Programmrahmen eingebettet werden, beispielsweise in eine `main()`-Funktion und geeignete Statements für Zuweisung und Ausgabe.
Die im Semester betrachteten Sprachstufen entwickeln sich schrittweise weiter. Die genaue Reihenfolge und der Umfang der späteren Sprachstufen werden im Skript eingeführt.
Syntaktisch und semantisch betrachten wir im Laufe des Semesters folgende Abstufung:

```
C++ <: C-Self <: C-Inherit <: C-Class <: C-Fun <: C-If <: C-Expr <: C-Int
```

In diesem Semester enden wir (voraussichtlich) bei C-Self.

14.4.2. Schritt 1: Java-Projekt für C-Int einrichten

Legen Sie ein Java-Projekt für **C-Int** an. Sie können dafür ein eigenes Projekt erstellen oder Ihre Projektorganisation so wählen, dass sie den fortlaufenden Ausbau im Semester unterstützt. Falls gewünscht, können Sie eines der folgenden Templates als Ausgangspunkt nutzen:

- Gradle: https://github.com/Compiler-CampusMinden/student-support-code-template
- Maven: https://github.com/Compiler-CampusMinden/student-support-code-template-maven

Legen Sie eine für Sie nachvollziehbare Paket- und Verzeichnisstruktur fest. Sie können diese in der Übung besprechen und bei Bedarf anpassen.

14.4.3. Schritt 2: Konkrete Syntax für C-Int festlegen

Definieren Sie analog zur Vorlesung die **konkrete Syntax für C-Int**. Sie legen damit fest, welche Programme von Ihrem Lexer und Parser akzeptiert werden.

Berücksichtigen Sie dabei folgende Anforderungen:

- Whitespace darf zwischen den Bestandteilen eines Ausdrucks stehen und wird übersprungen.
- Zeilenkommentare `// ...` bis zum Zeilenende werden unterstützt und übersprungen.
- Addition und Subtraktion verwenden jeweils zwei Integer-Operanden, z.B. `1-2`.
- Mehrfach verkettete Ausdrücke wie `1+2+3+4` sind erlaubt.
- Ausdrücke dürfen geklammert sein.
- Nicht geklammerte Ausdrücke werden implizit **von links nach rechts** gruppiert.

Notieren Sie Ihre konkrete Syntax, zum Beispiel als Grammatik. Halten Sie außerdem fest, welche Token, Schlüsselwörter und Symbole Ihr Lexer erkennen muss.

14.4.4. Schritt 3: Abstrakte Syntax für C-Int definieren

Definieren Sie analog zur Vorlesung die **abstrakte Syntax** (AST) für **C-Int** und repräsentieren Sie diese in Java.

Berücksichtigen Sie dabei folgende Fragen:

- Welche Informationen aus dem Parse-Tree müssen im AST erhalten bleiben?
- Warum müssen Klammern im AST nicht zwingend als eigene Knoten vorkommen?
- Wie wird die Linksassoziativität in der Struktur des AST dargestellt?

14.4.5. Schritt 4: Interpreter-Pipeline für C-Int implementieren

Erstellen Sie analog zum Vorgehen im Skript für C-Int:

- eine Tokendefinition,
- AST-Klassen und -Interfaces,
- einen Lexer,
- einen handgeschriebenen Recursive-Descent-Parser,
- einen Interpreter `InterpreterCint`.

Verwenden Sie in dieser Aufgabe **keinen** generierten ANTLR-Parser. Implementieren Sie die Pipeline wie in der Vorlesung mit handgeschriebenem Java-Code.

14.4.6. Schritt 5: Tests für C-Int schreiben

Schreiben Sie JUnit-Tests für Ihre C-Int-Pipeline. Testen Sie mindestens:

- ein Integer-Literal,
- eine Addition,
- eine Subtraktion,
- einen geschachtelten Ausdruck,
- Whitespace an unterschiedlichen Stellen,
- ein zusätzliches Token nach einem vollständigen Ausdruck,

- eine fehlende schließende Klammer,
- einen ungültigen Operator oder ein unerwartetes Zeichen.

Ein weiterer Test soll aufdecken, ob bei der Subtraktion versehentlich linker und rechter Operand vertauscht wurden. Testen Sie daher beispielsweise sowohl `(- 7 2)` als auch `(- 2 7)`.

Testen Sie nach Möglichkeit nicht nur die Auswertung, sondern auch die erzeugten Token und ASTs. Dadurch können Sie Fehler in den einzelnen Phasen der Pipeline besser voneinander unterscheiden.

14.4.7. Schritt 6: C-Int-Ergebnisse mit nativem C++ vergleichen

Schreiben Sie zu den folgenden Ausdrücken jeweils ein vollständiges C++-Programm mit `main()`, das den Ausdruck berechnet und das Ergebnis ausgibt:

```
42
(1 + 2)
((1 + 2) - 3)
(10 - (3 + 5))
```

Übersetzen und starten Sie Ihre Programme mit clang++ oder g++.

Vergleichen Sie die Ausgaben mit den Ergebnissen Ihres C-Int-Interpreters und notieren Sie kurz:

- Zwei Gemeinsamkeiten zwischen der Auswertung im nativen C++-Programm und der Auswertung durch Ihren Interpreter.
- Zwei Unterschiede zwischen den beiden Verarbeitungswegen (z.B. Laufzeit, Fehlermeldungen, Komfort).

14.5. A3.5: Vertiefung: Interpreter für L-Int mit Visitor-Pattern (***)

Im Skript wurde die `eval()`-Funktion im L-Int-Interpreter mit Pattern Matching und **`switch`**-Expressions implementiert.

```
public interface Expr {}
public record IntLiteral(int value) implements Expr {}
public record AddExpr(Expr left, Expr right) implements Expr {}
public record SubExpr(Expr left, Expr right) implements Expr {}


public interface Value {
    static int asInt(Value v) {
        if (v instanceof IntVal(int val)) return val;
        throw new RuntimeException("number expected");
    }
}
public record IntVal(int v) implements Value {}


public class InterpreterLint {
    public Value eval(Expr expr) {
        return switch (expr) {
```

```
19              case IntLiteral(var v) -> new IntVal(v);
20
21              case AddExpr(var l, var r) -> {
22                  int a = Value.asInt(eval(l));
23                  int b = Value.asInt(eval(r));
24                  yield new IntVal(a + b);
25              }
26
27              // TODO: SubExpr
28
29              default -> throw new RuntimeException("unexpected expression: " +
                    expr);
30          };
31      }
32  }
```

1. Übertragen Sie dies auf das Visitor-Pattern.
2. Skizzieren Sie Vor- und Nachteile des Visitor-Patterns im Vergleich zur **`switch`**-Variante aus der Vorlesung.

15. Blatt 04: L-Expr und C-Expr (Multiplikation/Division, Vorrang und ANTLR)

Important

Für die erfolgreiche Bearbeitung dieses Übungsblattes müssen mindestens alle mit $\star$ und $\star\star$ markierten Aufgaben gelöst werden.
Die Aufgaben mit $\star\star\star$ sind als Vertiefung gedacht.

Dieses Blatt knüpft an folgende Vorlesungseinheiten an:

- L-Expr: Multiplikation/Division und unäres Minus
- L-Expr: Vorrang und Assoziativität
- L-Expr: ANTLR-Grammatik und Vorrangregeln
- L-Expr: ANTLR-Visitoren und Rückgabewerte

15.1. A4.1: L-Expr: Division ergänzen (*)

Im Skript haben Sie L-Expr als Erweiterung von L-Int um Multiplikation und Division kennengelernt. In der Vorlesung wurde die Multiplikation im Detail behandelt.

Wenden Sie das gezeigte Muster auf die Division an und skizzieren Sie in Java, wie die Auswertung einer `DivExpr` aussehen könnte. Geben Sie dabei an, welche Teil-Ausdrücke ausgewertet und wie ihre Ergebnisse kombiniert werden.

Gegeben sei dazu der folgende Code-Rahmen.

```
1  public interface Expr {}
2  public record IntLiteral(int value) implements Expr {}
3  public record AddExpr(Expr left, Expr right) implements Expr {}
4  public record SubExpr(Expr left, Expr right) implements Expr {}
5  public record MulExpr(Expr left, Expr right) implements Expr {}
6  public record DivExpr(Expr left, Expr right) implements Expr {}
7
8  public interface Value {
9      static int asInt(Value v) {
10         if (v instanceof IntVal(int val)) return val;
11         throw new RuntimeException("number expected");
12     }
13 }
14 public record IntVal(int v) implements Value {}
15
```

```java
public class InterpreterLint {
    public Value eval(Expr expr) {
        return switch (expr) {
            case IntLiteral(var v) -> new IntVal(v);

            case AddExpr(var l, var r) -> {
                int a = Value.asInt(eval(l));
                int b = Value.asInt(eval(r));
                yield new IntVal(a + b);
            }

            // TODO: SubExpr

            default -> throw new RuntimeException("unexpected expression: " +
                expr);
        };
    }
}


public class InterpreterLexpr extends InterpreterLint {

    @Override
    public Value eval(Expr expr) {
        return switch (expr) {
            case MulExpr(var l, var r) -> {
                int a = Value.asInt(eval(l));
                int b = Value.asInt(eval(r));
                yield new IntVal(a * b);
            }

            // TODO: DivExpr

            default -> super.eval(expr);
        };
    }
}
```

Weitere Aufgabenteile:

1. Wie behandeln Sie Vorrangregeln und die Assoziativität im Interpreter für L-Expr?
2. Notieren Sie, wie der mehrstellige Ausdruck `(/ 12 3 2)` gruppiert wird und welches Ergebnis er nach dieser Gruppierung hat.
3. Wie wird die Division durch null behandelt? Brauchen wir hier eine explizite Fehlerbehandlung im Interpreter?
4. Erstellen Sie außerdem mindestens zwei Testfälle:
    - einen Test, der die Java-Integer-Division sichtbar macht
    - einen Test für eine verschachtelte Division

15.2. A4.2: C-Expr: Vorrangregeln in einer gestuften Grammatik (**)

Wir erstellen nun **C-Expr**, indem wir **C-Int vom letzten Blatt** um Multiplikation, Division, unäres Minus und geklammerte Ausdrücke erweitern.

C-Expr soll analog zu L-Expr **Multiplikation und Division und unäres Minus und geklammerte Ausdrücke** erlauben, aber in C++-üblicher Infix-Notation und unter Beachtung der üblichen Vorrang- und Assoziativitätsregeln.

Important

15.2.1. Waschzettel: In C++ sieht es so aus ...

Zweck	L-Expr (Lispy)	C-Expr (Mini-C++)
Multiplikation	`(* 2 3)`	`2 * 3`
Division	`(/ 8 3)`	`8 / 3`
Unäres Minus	`(- 3)`	`-3`
Klammerung	`(* (+ 1 2)3)`	`(1 + 2)* 3`

Während in Lispy (L-Int, L-Expr) die Operatoren für `+`, `-`, `*`, `/` mehrstellig sein können, sind sie in C++ (C-Int, C-Expr) **binär** (bzw. `-` zusätzlich unär).

15.2.2. Schritt 1: C-Expr ausprobieren: Vorrang und unäres Minus live testen

Wir wollen zunächst die Vorrangregeln und unäres Minus **direkt in C++** ausprobieren.

Schreiben Sie ein C++-Programm `expr.cpp`, das folgende Ausdrücke auswertet und jeweils das Ergebnis über `std::cout` ausgibt:

```
1  1 + 2 * 3
2  (1 + 2) * 3
3  8 / 4 / 2
4  (8 / 4) / 2
5  8 / (4 / 2)
6  7 - 2 - 1
7  7 - (2 - 1)
8  -3 * 4
9  -(3 * 4)
10 -(1 + 2) * 3
```

Übersetzen Sie es mit g++/clang und prüfen Sie, ob jeweils das Ergebnis Ihren Erwartungen entspricht.

15.2.3. Schritt 2: Konkrete Syntax für C-Expr (gestufte Variante)

Definieren Sie eine **konkrete Grammatik für C-Expr** (in EBNF), die folgende Anforderungen erfüllt:

- Zulässige Ausdrücke enthalten:
  - Integer-Literale
  - binäre Operatoren `+`, `-`, `*`, `/`
  - unäres Minus: `-x`
  - Klammern `(` und `)`
- Vorrang:
  - `*` und `/` **binden stärker** als `+` und `-`
  - Unäres Minus und geklammerte Ausdrücke haben höchsten Vorrang
- Alle vier Operatoren sind **linksassoziativ**
- Whitespace darf an beliebigen sinnvollen Stellen stehen und wird ignoriert
- Zeilenkommentare werden ab dem `//` bis zum Zeilenende ignoriert

Skizzieren Sie Ihre Grammatik ausdrücklich mit **gestuften Regeln**, z.B. mit den Ebenen `expr`/`term`/`atom`.

Überprüfen Sie mit Ihrer Grammatik:

- Wie wird `1 + 2 * 3` geparst und ausgewertet?
- Wie wird `8 / 4 / 2` geparst und ausgewertet?
- Wie wird `-3 * 4` geparst und ausgewertet?
- Wie werden Klammern berücksichtigt, z.B. `(1 + 2)* 3`, `-(1 + 2)* 3`?

Zeichnen Sie jeweils den Parse-Tree. Entspricht das dem Verhalten von C++ aus der vorigen Aufgabe?

15.2.4. Schritt 3: ANTLR-Grammatiken für C-Expr

Definieren Sie nun ausgehend von der konkrete Grammatik aus der vorigen Aufgabe zwei **ANTLR-Grammatiken** für **C-Expr**:

- einmal mit **gestuften Regeln**, und
- einmal mit **linksrekursiven Alternativen**.

Berücksichtigen Sie dabei die für C-Expr formulierten Operatoren und Vorrangregeln und Assoziativitäten.

15.2.5. Schritt 4: Abstrakte Syntax für C-Expr

Definieren Sie nun die **abstrakte Syntax für C-Expr**. Orientieren Sie sich an der Vorlesung und bauen Sie auf Ihrer abstrakten Syntax von C-Int auf.

Diskutieren Sie: Welche neuen AST-Knoten sind notwendig und warum?

15.2.6. Schritt 5: Parsen von C-Expr und bauen des AST

1. Implementieren Sie für Ihre konkrete Grammatik (mit den gestuften Regeln) für **C-Expr einen Lexer**. Erweitern Sie dazu die Tokendefinitionen passend.

2. Implementieren Sie einen **RD-Parser für C-Expr**, der als Ausgabe einen AST erzeugt. Ergänzen Sie dazu die AST-Klassen entsprechend Ihrer abstrakten Syntax.
3. Lassen Sie sich von **ANTLR** für die beiden Grammatiken aus der vorigen Aufgabe je einen **Lexer, Parser und Visitor generieren**.
4. Erstellen Sie **zwei AST-Builder**, die aus den beiden unterschiedlichen ANTLR-Parse-Trees je einen AST entsprechend Ihrer abstrakten Syntax erzeugen. Nutzen Sie das **Visitor-Pattern**. Diskutieren Sie:
   - Welchen generischen Typ brauchen Sie für das Ableiten des generierten `XyzBaseVisitor`?
   - Welche der `visitXYZ()`-Methoden sollten Sie überschreiben, welche nicht? Begründen Sie Ihre Aussage.
5. Testen Sie die neue C-Expr-Pipeline: Implementieren Sie die **obigen Beispiele in JUnit** und führen Sie diese auf **allen drei Varianten** aus (RD-Parser, ANTLR mit gestuften Regeln, ANTLR mit linkrekursiven Alternativen). Was beobachten Sie?

Tip

Hinweis: Sie können die Funktionalität aus dem Lexer und RD-Parser für C-Int als Basis per Copy&Paste übernehmen und für C-Expr ergänzen/umbauen. Die gestuften Regeln vertragen sich nicht gut mit dem sonst in diesem Kurs praktizierten Ableiten `class RdParserCexpr extends RdParserCint`, weshalb wir hier an dieser Stelle darauf verzichten.

15.2.7. Schritt 6: Interpreter-Pipeline für C-Expr

Implementieren Sie nun den **Interpreter für C-Expr** in einer Klasse `InterpreterCexpr`. Dabei soll wie üblich in diesem Kurs gelten `class InterpreterCexpr extends InterpreterCint`.

Ergänzen Sie die neuen Fälle für C-Expr durch Überschreiben von `eval()`. Rufen Sie für die bisherigen C-Int-Fälle `super.eval(expr)` auf.

Testen Sie Ihren Interpreter mit JUnit mit den obigen Beispielen. Dabei soll wie bei L-Expr gelten: `class InterpreterCexprTest extends InterpreterCintTest`.

Important

Der Interpreter bekommt als Eingabe den AST und ist damit unabhängig vom konkreten Parser!
Testen Sie Ihren Interpreter sowohl mit dem handgeschriebenen RD-Parser als auch mit den beiden ANTLR-Parsern plus AST-Buildern.

Vergleichen Sie die Resultate der Testfälle von `InterpreterCexpr` mit dem mit g++/clang übersetzten nativen C++-Code. Gibt es Unterschiede? Woran liegt das?

15.3. A4.3: Vertiefung: ANTLR und Stolperfallen (***)

Betrachten Sie die folgende ANTLR-Grammatik:

```
foo  :  (ID '(' ')' '=' '0' | ID | NUM) EOF ;

ID       : [a-z][a-zA-Z]* ;
NUM      : [0-9]+ ;
WS       : [ \t\n]+ -> skip ;
```

Was passiert bei der Eingabe von `10` und `01` und `a()= 0`, wie sieht der Parse-Tree aus? Was passiert bei der Eingabe von `0`?

Erklären Sie die Resultate.

Quellen

Aho, A. V., M. S. Lam, R. Sethi, J. D. Ullman, und S. Bansal. 2023. *Compilers: Principles, Techniques, and Tools, Updated 2nd Edition by Pearson*. Pearson India. https://learning.oreilly.com/library/view/compilers-principles-techniques/9789357054881/.

Grune, D., K. van Reeuwijk, H. E. Bal, C. J. H. Jacobs, und K. Langendoen. 2012. *Modern Compiler Design*. Springer.

Hopcroft, J. E., R. Motwani, und J. D. Ullman. 2003. *Einführung in die Automatentheorie, formale Sprachen und Komplexitätstheorie*. I theoretische informatik. Pearson Education Deutschland GmbH.

Nystrom, R. 2021. *Crafting Interpreters*. Genever Benning. https://github.com/munificent/craftinginterpreters.

Parr, T. 2014. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf. https://learning.oreilly.com/library/view/the-definitive-antlr/9781941222621/.

Siek, J. G. 2023. *Essentials of Compilation: An Incremental Approach in Python*. The MIT Press. https://github.com/IUCompilerCourse/Essentials-of-Compilation.

Torczon, L., und K. Cooper. 2012. *Engineering a Compiler*. Morgan Kaufmann. https://learning.oreilly.com/library/view/engineering-a-compiler/9780080916613/.



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- Clojure.org: Learn Clojure
- A Map of the Territory (mountain.png) by Bob Nystrom on Github.com (MIT)
- Abzählreim “99 Bottles of Beer” nach “Lyrics of the song 99 Bottles of Beer” on 99-bottles-of-beer.net
- “Language Java” by Sean Russell on 99-bottles-of-beer.net
- “Language Prolog” by M@ on 99-bottles-of-beer.net
- “Language C” by Bill Wein on 99-bottles-of-beer.net
- Screenshot of “Language Brainfuck” by Michal Wojciech Tarnowski on 99-bottles-of-beer.net
- “Language Haskell” by Iavor on 99-bottles-of-beer.net

> **Last modified:** 30368eb 2026-11-12 markdown: add more space in formula